

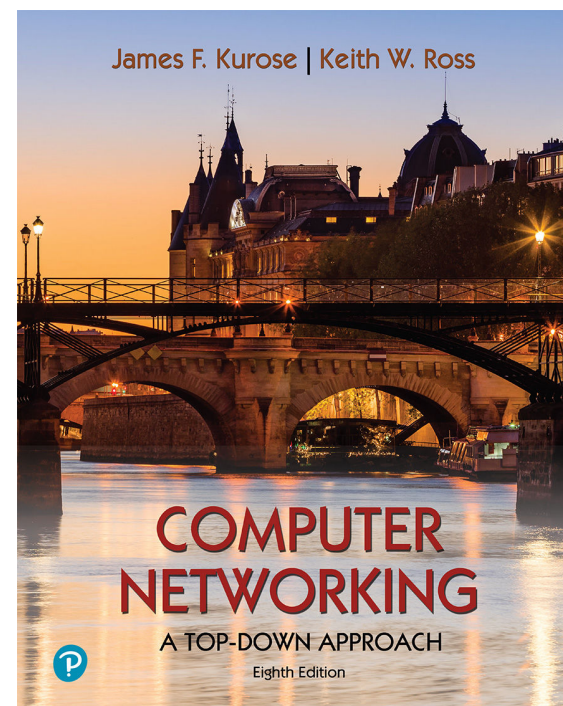
第 5 章

网络层:控制平面

中国科学技术大学

自动化系 郑烱

改编自 Jim Kurose, Keith Ross



*Computer Networking: A
Top-Down Approach*

8th edition

Jim Kurose, Keith Ross

Pearson, 2020

网络层控制平面：目标

- 理解网络控制平面背后的原理：
 - 传统路由算法
 - SDN控制器
 - 网络管理、配置
- 互联网网络层控制平面的实例和实现
 - OSPF, BGP
 - OpenFlow, ODL和ONOS控制器
 - Internet Control Message Protocol: ICMP
 - SNMP, YANG/NETCONF

网络层：“控制平面”提纲

- 引论
- 路由算法
 - 链路状态link state
 - 距离矢量distance vector
- ISP内部的路由协议: RIP,OSPF
- ISP之间的路由: BGP
- SDN控制平面
- Internet Control Message Protocol
- 网络管理，配置
 - SNMP
 - NETCONF/YANG



网络层功能

- **转发**: 将分组从路由器的一个输入端口转移到合适的输出端口
- **路由**: 决定分组从源到目标的路径

数据平面

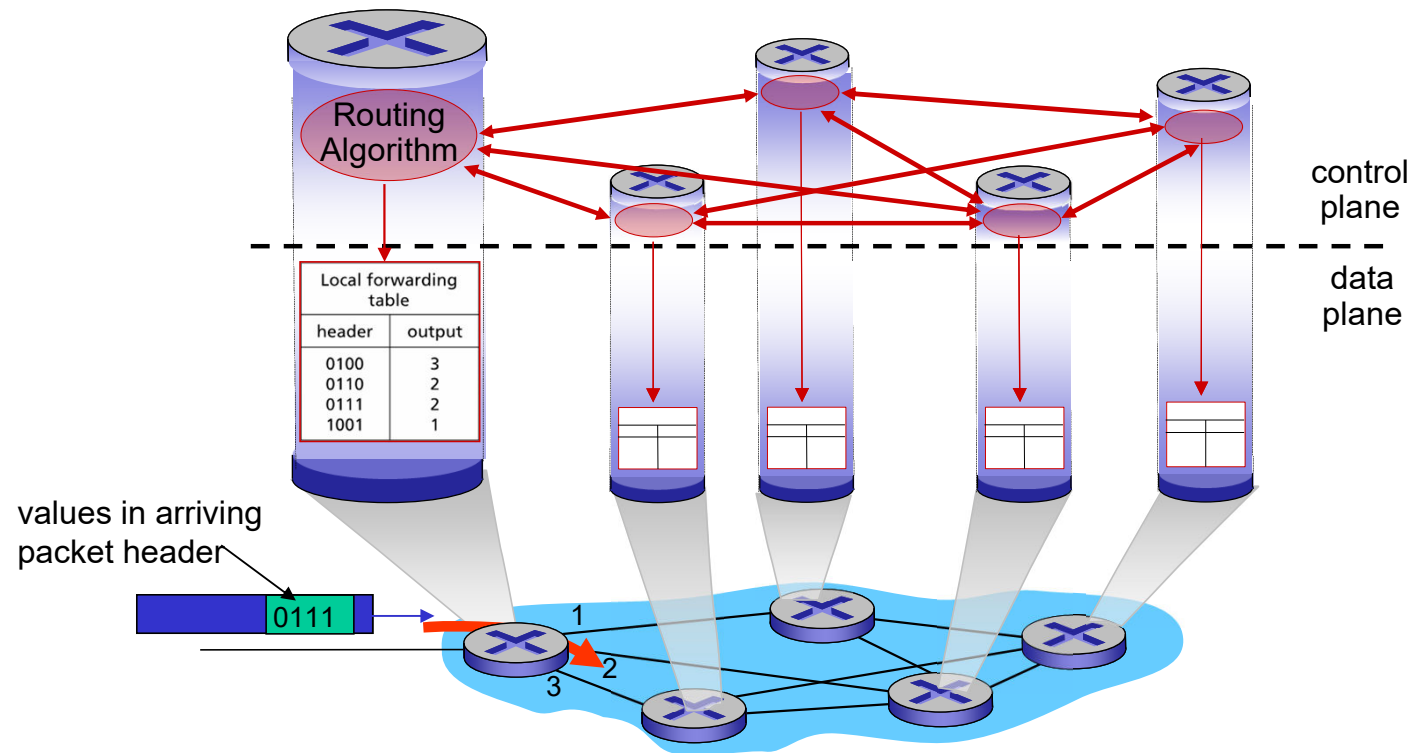
控制平面

2种构建网络层控制平面的方法:

- 每-路由器控制（传统方法）
- 逻辑上集中式控制（软件定义网络）

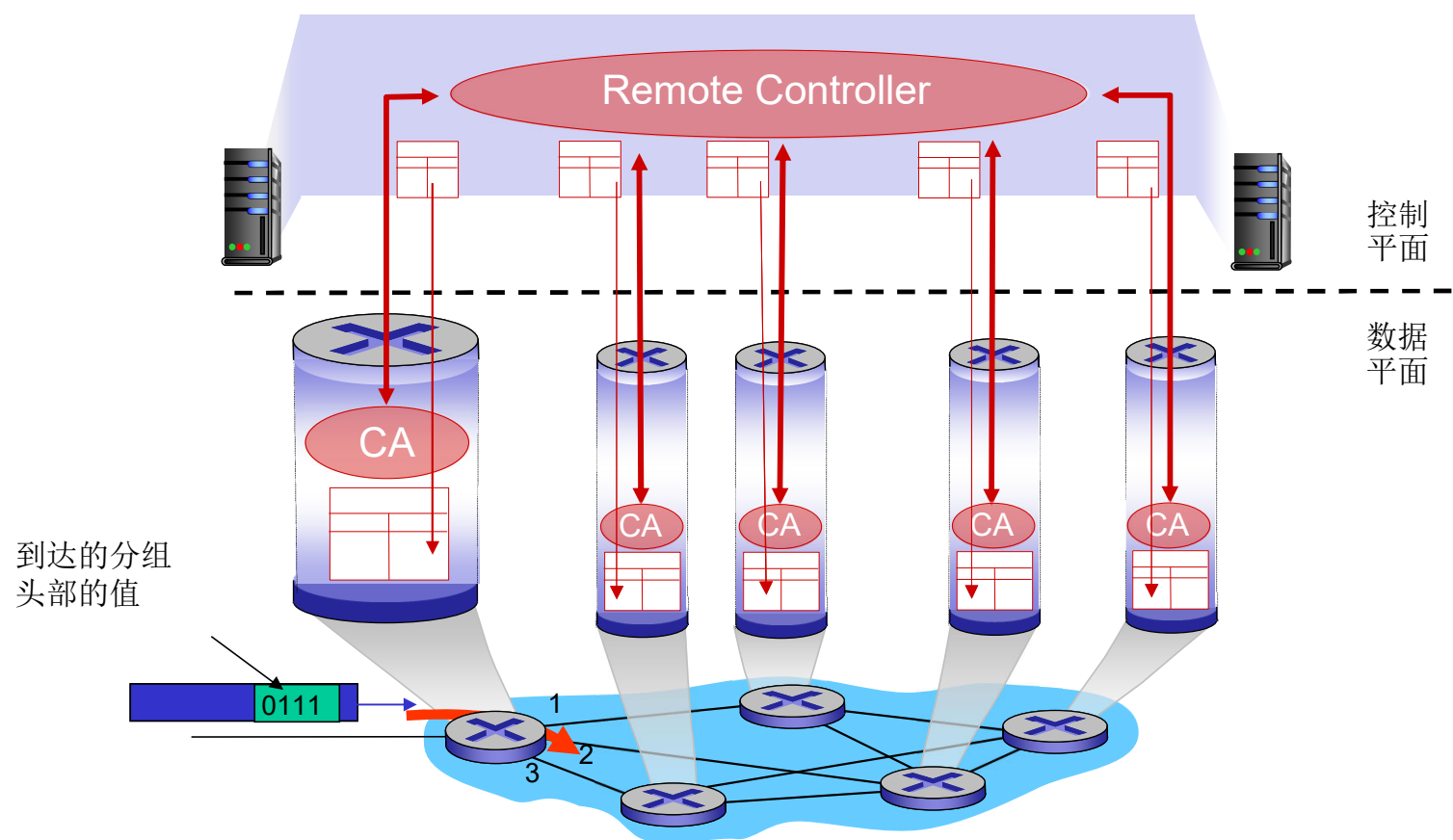
每路由器控制平面

独立的路由算法元件运行在**每个路由器**中，路由器在控制平面进行交互



Software-Defined Networking (SDN) 控制平面

远程的控制器计算并在交换设备安装转发表



网络层：“控制平面”提纲

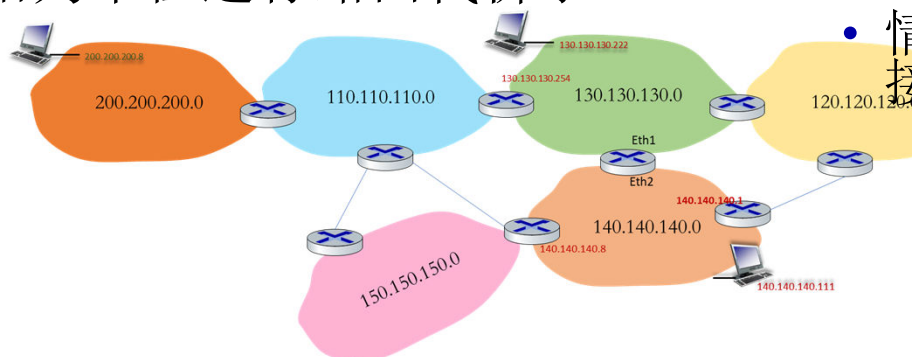
- 引论
- 路由算法
 - 链路状态link state
 - 距离矢量distance vector
- ISP内部的路由协议: RIP,OSPF
- ISP之间的路由: BGP
- SDN控制平面
- Internet Control Message Protocol
- 网络管理，配置
 - SNMP
 - NETCONF/YANG



以网络（子网）为单位进行路由信息通告和计算

■ 互联网以网络为单位组织

- 一个网络内子网前缀相同，子网内IP之间一跳可达
 - 主机到路由器
 - 路由器到路由器
 - 路由器比较特殊，分属多个网络，可在多个网络间转发分组
- 把该网络任何IP作为目标，互联网其他网络通往这些IP的路径都一样
- 以网络为单位进行路由代价小



■ 路由结果：以子网为单位的指针

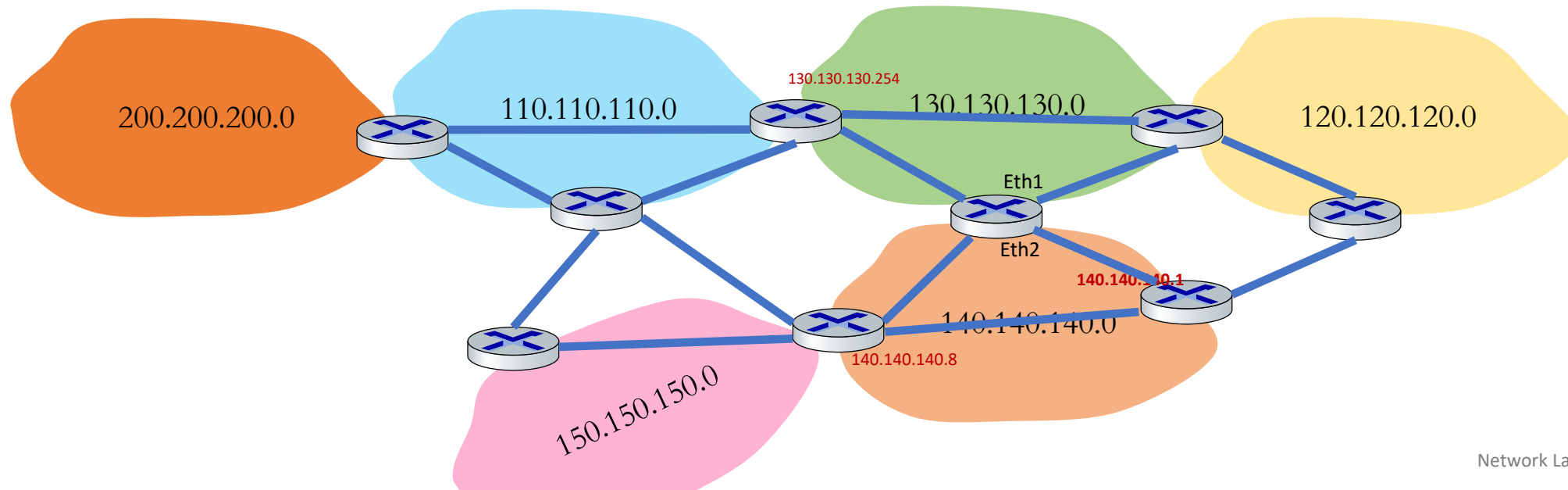
- 该路由器往目标网络的下一跳路由器（与本路由器同在一个网络）
- 实际上是指向目标网络的路径上下一跳指针

■ 使用：以子网为单位的匹配和逐跳转发

- 路由器从某个接口对应的某个网络A收到一个分组，匹配
- 情况1：通过另一接口，转到同属于B网络的另外一个下一跳路由器
- 情况2：通过另外一个接口B，可直接到达主机，下一跳为-

路由器为主体进行路由计算

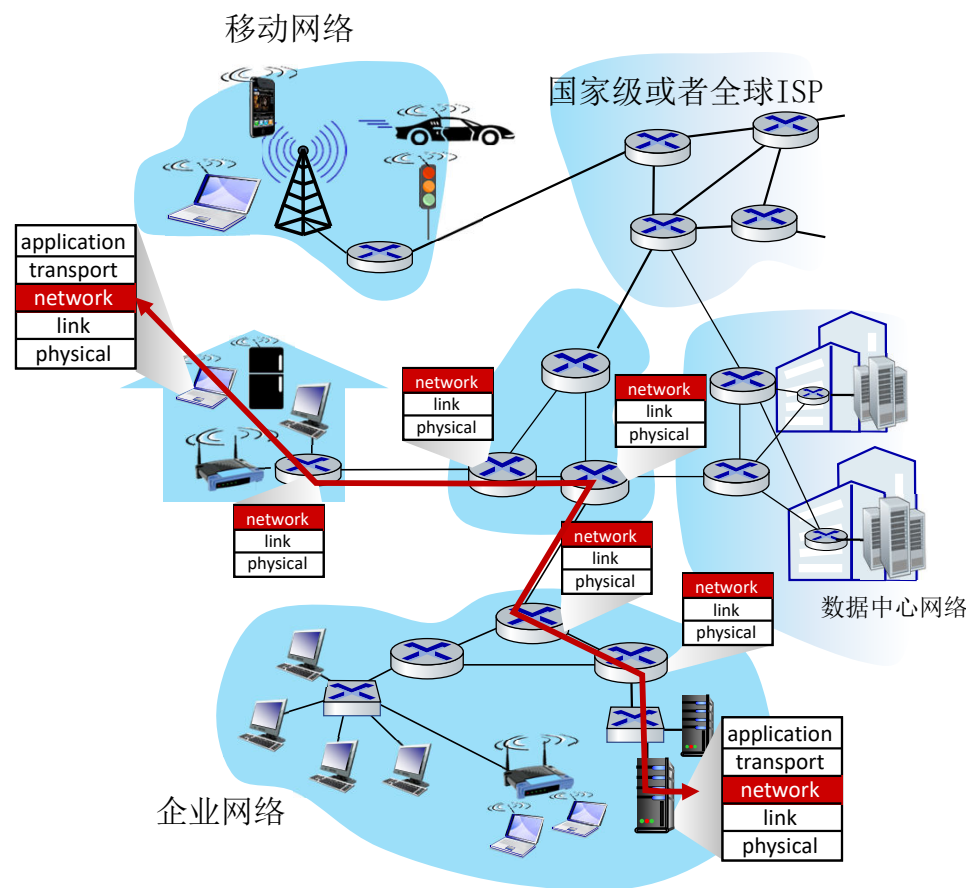
- 路由器作为主体进行往各个子网的路由计算
- 从路由器的视角来看：网络间路由= 路由器间路由
 - 主机所在网络间的路由（主机-主机的路由）= 路由器之间的路由
 - 第一跳：源主机所在网络的默认出口路由器到目标网络路由器
 - 最后一跳：到了目标网络所连的路由器就是到了这个网络



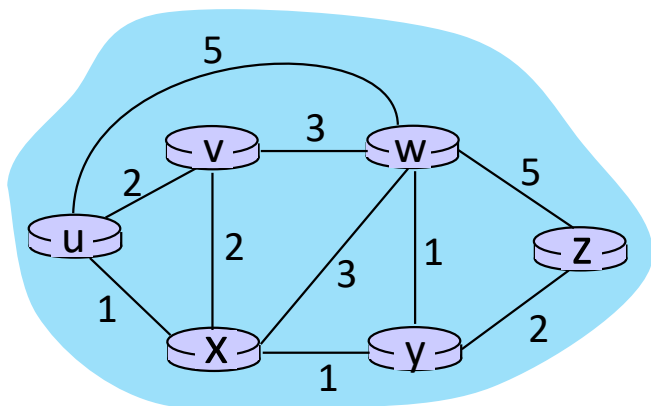
路由协议

路由： 决定从源网络到目标源网络、通过路由器网络的“较好”路径

- **路径：** 给出源到目的所穿过的路由器序列
- **“较好”：**按照某种指标较小的路径
 - 跳数、代价、时间、拥塞程度，队列长度等，或是一些单纯指标的加权平均
 - 采用什么样的指标，表示网络使用者希望网络在什么方面表现突出，什么指标网络使用者比较重视
- 路由： 一个“top 10”的网络挑战！



图抽象：链路和路径的代价



$c_{a,b}$: 直接连接a和b的链路代价

e.g., $c_{w,z} = 5$, $c_{u,z} = \infty$

代价被网络运营者所定义：可能永远为1，或者与带宽成反比，或者与拥塞程度成反比

图: $G = (N, E)$

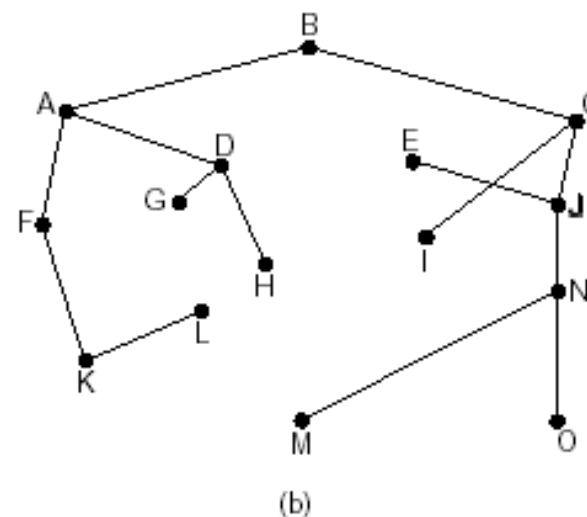
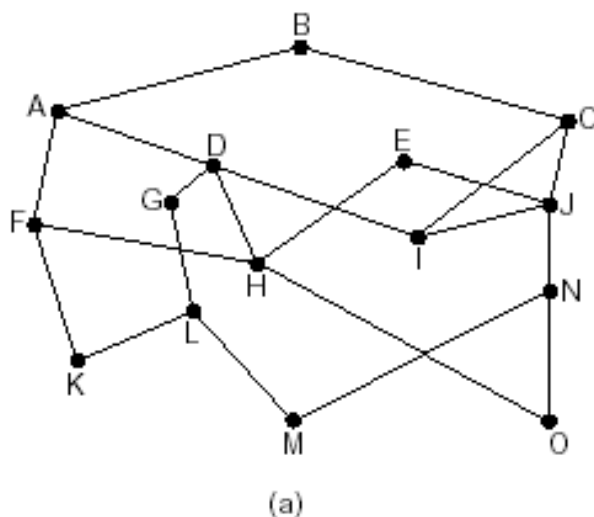
N : 路由器的集合 = $\{ u, v, w, x, y, z \}$

E : 链路的集合 = $\{ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$

路径代价: $(x_1, x_2, x_3, \dots, x_p) = c(x_1, x_2) + c(x_2, x_3) + \dots + c(x_{p-1}, x_p)$

最优化原则(optimality principle)

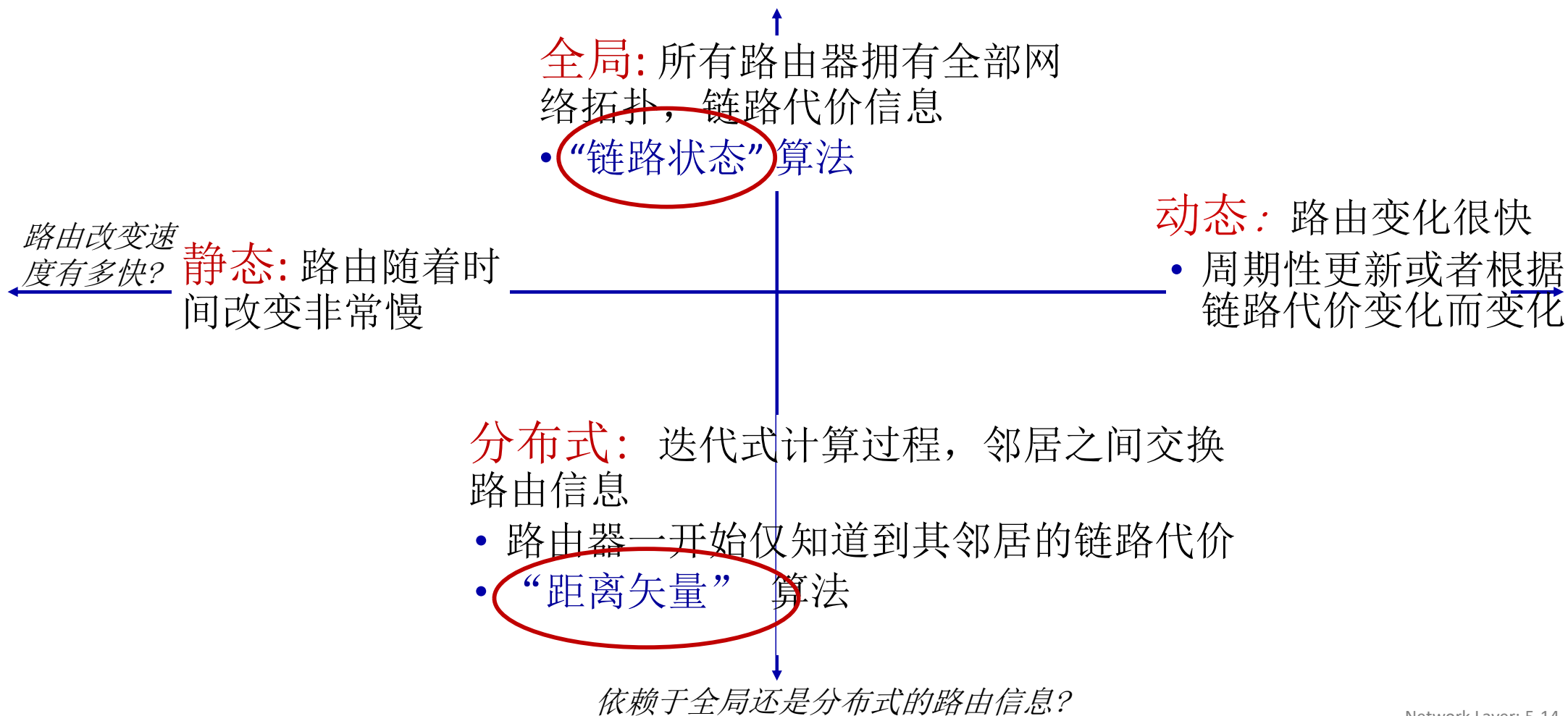
- 某节点的汇集树(sink tree)
 - 此节点到所有其它节点的最优路径所形成的树
 - 路由选择算法就是为所有路由器找到汇集树



路由的原则

- 正确性(correctness): 算法必须是**正确**的和**完整**的
 - 完整: 没有处理不了的目标
- 简单性(simplicity): 算法在实现上应简单
- 健壮性(robustness) : 算法应能适应**通信量**和**网络拓扑**的变化
- 稳定性(stability): 产生的路由不应该摇摆
- 公平性(fairness): 对每一个目标都公平
- 最优性(optimality): 某一个指标的最优;
 - 实际上获取最优路由代价较通常高, 通常可以是次优的

路由算法分类



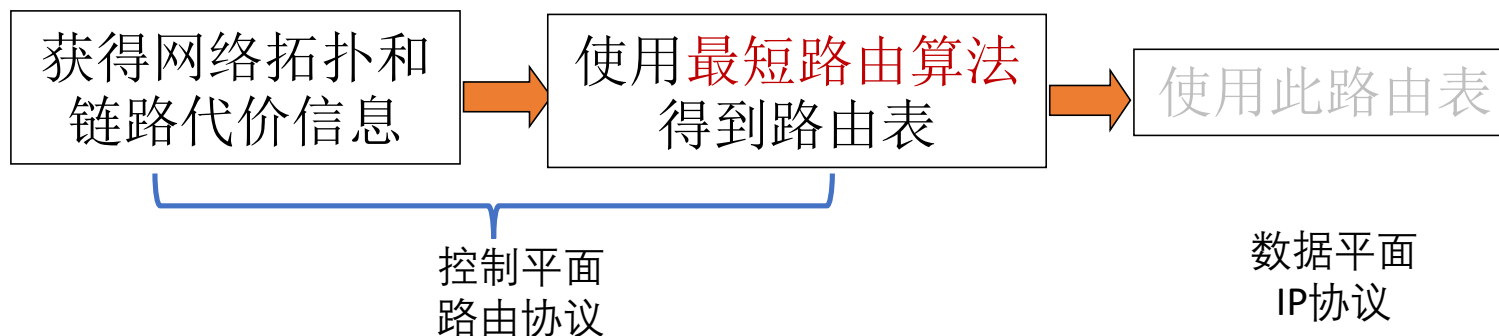
网络层：“控制平面”提纲

- 引论
- 路由算法
 - 链路状态link state
 - 距离矢量distance vector
- ISP内部的路由协议: RIP,OSPF
- ISP之间的路由: BGP
- SDN控制平面
- Internet Control Message Protocol
- 网络管理，配置
 - SNMP
 - NETCONF/YANG



LS路由的工作过程

- 各节点通过路由协议交换、测量路由信息，获得**整个网络拓扑**，链路**代价**等信息（这部分和算法没关系，属于协议及其实现）
- 使用**LS路由算法**，计算本路由节点到其它路由节点的最优路径(汇集树)，得到路由表
- 按照此路由表转发分组(datagram方式)-数据平面的功能
 - 严格意义上说不是路由的一个步骤

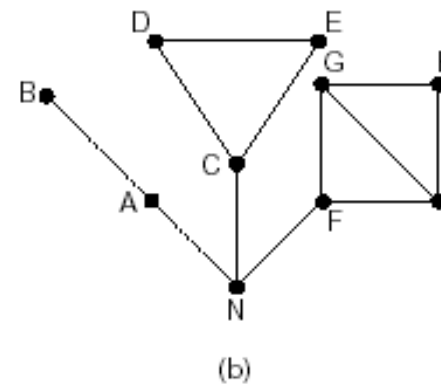
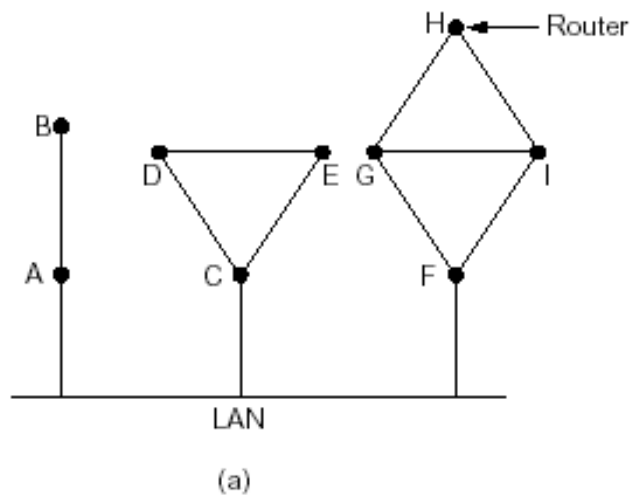


链路状态路由选择(link state routing)

1. 发现相邻节点，获知对方网络地址
2. 测量到相邻节点的代价(延迟，开销)
3. 组装一个LS分组，描述它到相邻节点的代价情况
4. 将分组通过扩散的方法发到所有其它路由器
以上4步让每个路由器获得拓扑和边代价
5. 通过Dijkstra算法算出最短路径（这才是路由算法）
 - a) 每个节点独立算出来到其他节点的最短路径
 - b) 迭代算法：每个节点第k步能够知道本节点到k个其他节点的最短路径

第1步.发现相邻节点获知对方网络地址

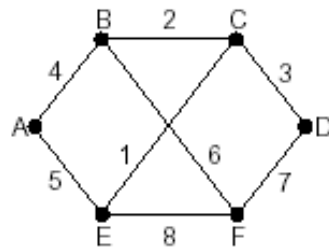
- 一个路由器上电之后，向所有接口发送HELLO分组
- 其它路由器收到HELLO分组，回送应答，在应答分组中，告知自己的名字(全局唯一，网络号)
- 如果在LAN，通过广播HELLO分组，获得其它路由器的信息，可以认为引入一个人工节点



LS 路由的第2. 第3.步

2.测量到相邻节点的代价(延迟, 开销等)

- 实测法，发送一个分组要求对方立即响应
- 回送一个ECHO分组
- 通过测量时间可以估算出延迟情况等



(a)

3.组装一个分组，描述到相邻节点的情况

- 发送者名称
- 序号、年龄
- 列表：给出它的邻居列表，和到这些邻居的延迟等代价

Link		State		Packets	
A	B	C	D	E	F
Seq.	Seq.	Seq.	Seq.	Seq.	Seq.
Age	Age	Age	Age	Age	Age
B 4	A 4	B 2	C 3	A 5	B 6
E 5	C 2	D 3	F 7	C 1	D 7
	F 6	E 1		F 8	E 8

(b)

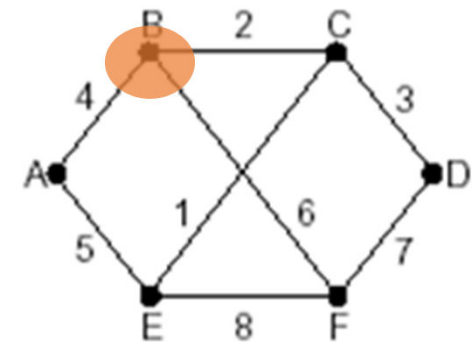
第4步.将LS分组扩散到所有其它路由器

- **顺序号**：用于控制无穷的扩散，每个路由器都记录(源路由器,顺序号)，发现重复的或老的就不扩散
 - 问题1: 循环使用问题
 - 问题2: 路由器崩溃之后序号从0开始
 - 问题3: 序号出现错误
- 解决问题的办法：**年龄字段(age)**
 - 生成一个分组时，年龄字段不为0
 - 每隔一个时间段，AGE字段减1
 - AGE字段为0的分组将被抛弃

第4步.将LS分组扩散到所有其它路由器

■ 关于扩散分组的数据结构：实现可靠泛洪

- ✓ Source :从哪个节点收到LS分组
- ✓ Seq, Age: 序号, 年龄
- ✓ Send flags: 发送标记, 必须向指定的哪些相邻站点转发LS分组
- ✓ ACK flags: 本站点必须向哪些相邻站点发送应答
- ✓ DATA: 来自source站点的LS分组



(a)

- ✓ 节点B的数据结构

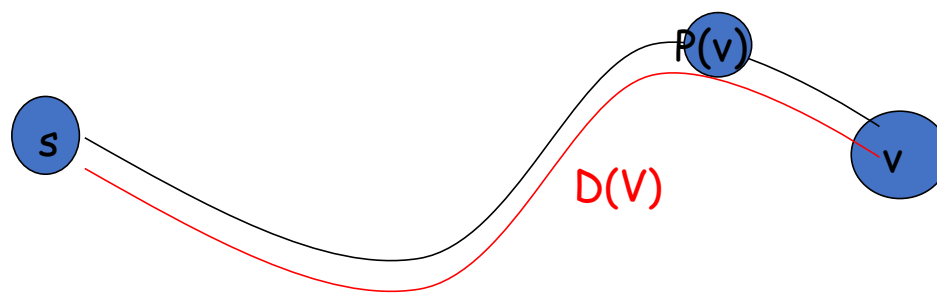
Source	Seq.	Age	Send flags			ACK flags			Data
			A	C	F	A	C	F	
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

第5步.使用Dijkstra计算路由表

- **集中式**: 网络拓扑和链路代价被所有节点获知
 - 通过“LS分组的广播”来完成
 - 所有节点拥有同样的信息
- 节点计算自己（“作为源”）到所有其他节点的最小代价路径
 - 得到该节点的**转发表**
- **迭代**: 每个节点在计算时经过k次迭代，知道k个目标的最小代价路径

标记

- $c_{x,y}$: x到y的直接邻居代价;
= ∞ 如果是非直接邻居
- $D(v)$: 从源到目标v的当前最优路径的代价估计值
- $p(v)$: 从源到v节点路径的前序节点
- N' : 已知具有最小代价路径节点的集合

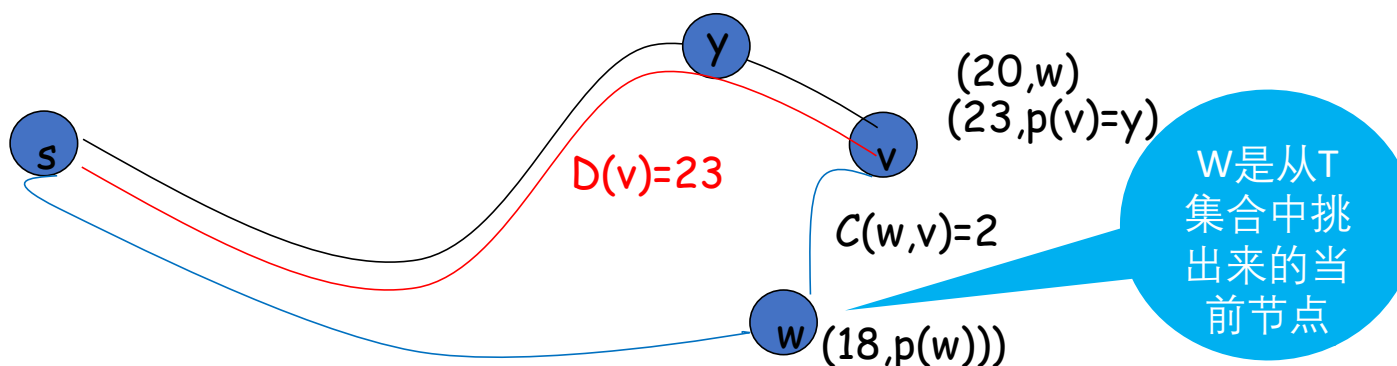


第5步.使用Dijkstra计算路由表

- 节点标记: 每一个节点使用 $(D(v), p(v))$ 如: $(3, B)$ 标记
 - $D(v)$ 从源节点由当前已知最优路径到达本节点的距离
 - $P(v)$ 前序节点
- 2类节点
 - 临时节点(tentative node): 还未找到从源节点到此节点的最优路径
 - 永久节点(permanent node) N' : 已经找到了从源节点到此节点的最优路径

第5步.使用Dijkstra计算路由表

- 初始化
 - 除了源节点外，所有节点都为临时节点
 - 标记邻居节点：除了与源节点代价相邻的节点外，节点代价都为 ∞
- 从所有临时节点中找到一个节点代价最小的临时节点，将之变成永久节点(当前节点) w
- 对此节点 w 的所有在临时节点集合中的邻节点(v)
 - 如 $D(v) > D(w) + c(w,v)$, 则重新标注此点: $(D(w)+C(w,v), w)$
 - 否则不重新标注
- 开始一个新的循环



Dijkstra's 链路状态路由选择算法

1 *Initialization:*

2 $N' = \{u\}$

/* 计算u到所有其他节点的最小代价路径 */

3 for all nodes v

4 if v adjacent to u

/* u 一开始仅知道到其直接相联邻居的直联边的代价 */

5 then $D(v) = c_{u,v}$

/*但是可能不是最小代价!

*/

6 else $D(v) = \infty$

7
8 *Loop*

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

11 update $D(v)$ for all v adjacent to w and not in N' :

12 $D(v) = \min (D(v), D(w) + c_{w,v})$

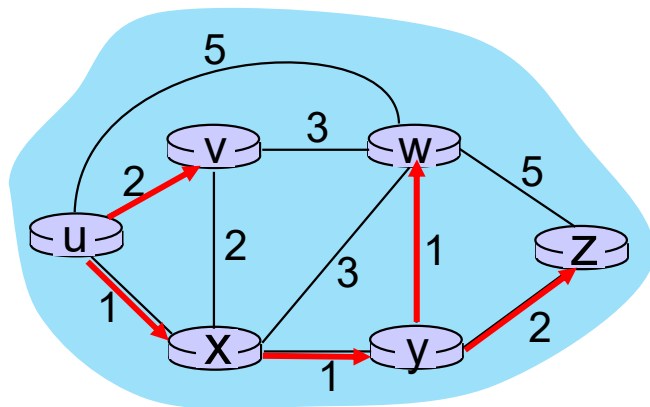
13 /* new least-path-cost to v is either old least-cost-path to v or known

14 least-cost-path to w plus direct-cost from w to v */

15 *until all nodes in N'*

Dijkstra算法: 一个实例

Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



初始化 (步骤 0): 对于所有的 a , 如果 a 与之相邻, 则标记 $D(a) = c_{u,a}$

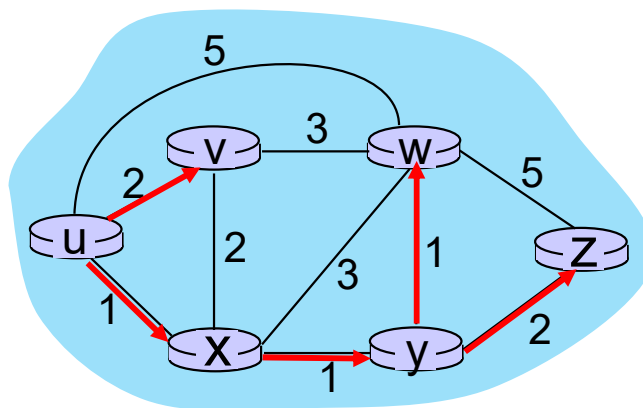
在 N' 中找到具有最小 $D(a)$ 的 a

将 a 移动到 N'

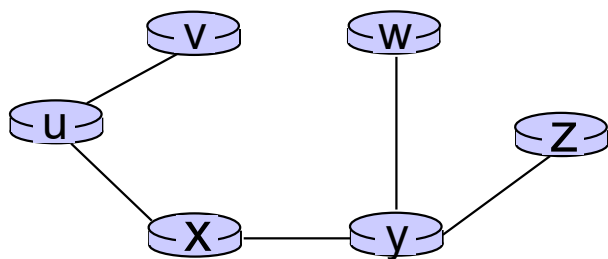
对于所有的与 a 相邻的不在 N' 的 b , 更新其 $D(b)$:

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

Dijkstra算法: 一个实例



路由计算结果:
源u的汇集树



在u中的结果转发表:

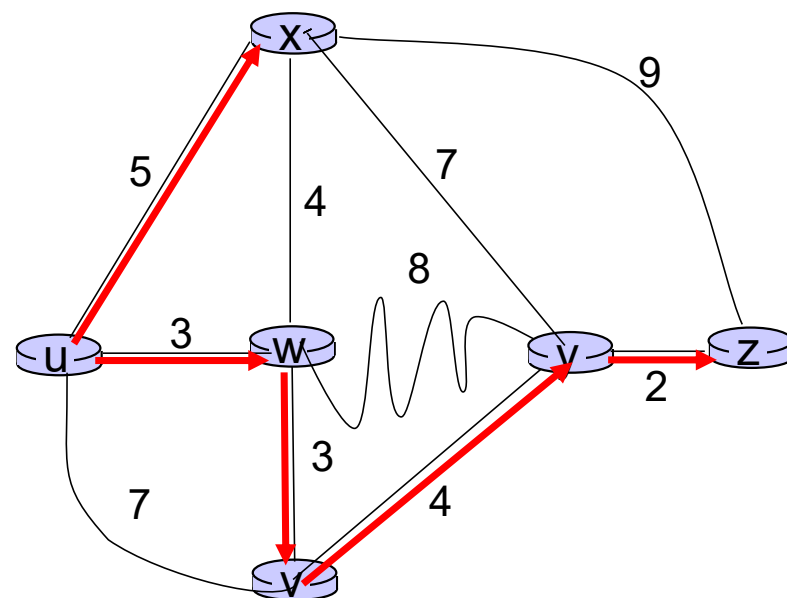
目标	输出链路
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

直接从u到路由到v

从u到所有其他目标都通过x

Dijkstra算法: 另外一个实例

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					



注:

- 通过最终前序节点来构建最小代价路径树
- 节点标记中的前序节点，构成连接（可以被任意断开）

Dijkstra算法: 讨论

时间复杂度: n 节点

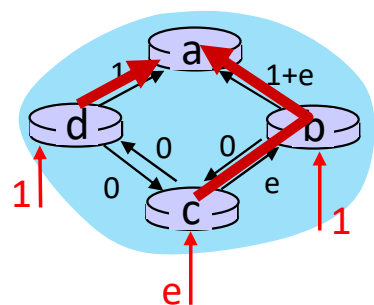
- 每个 n 的迭代: 都需要检测所有不在 N' 中的 w 节点
- $n(n+1)/2$ 比较: $O(n^2)$ 复杂度
- 可能存在更加高效的实现: $O(n \log n)$

消息复杂度:

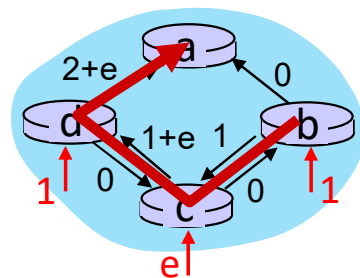
- 每个路由器都必须广播它的链路状态到其他 n 个路由器
- 有效（且有趣!）的广播算法: 经过 $O(n)$ 链路来散播从一个源的广播消息
- 每个节点（路由器）的消息都要经过 $O(n)$ 链路: 所有节点的消息复杂度为 $O(n^2)$

Dijkstra 算法: 可能的路由振荡

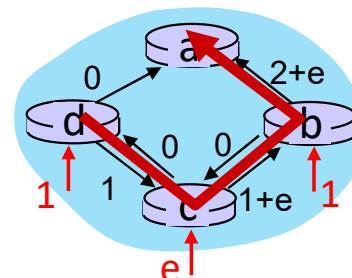
- 当链路代价依赖于通信流量，可能会发生路由振荡
- 简单场景:
 - 路由到目标a，流量从d, c, e进入，值分别是 1, e ($e < 1$), 1
 - 链路代价是该链路上与流量相关的值



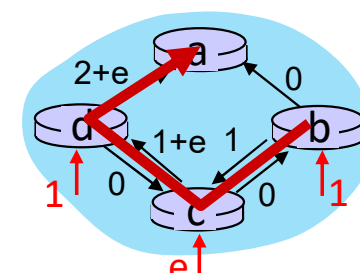
一开始



给定这些代价，
找到新路径。
路由结果有新的代价



给定这些代价，
找到新路径..
路由结果有新的代价



给定这些代价，
找到新路径..
路由结果有新的代价

网络层：“控制平面”提纲

- 引论
- 路由算法
 - 链路状态link state
 - 距离矢量distance vector
- ISP内部的路由协议: RIP,OSPF
- ISP之间的路由: BGP
- SDN控制平面
- Internet Control Message Protocol
- 网络管理，配置
 - SNMP
 - NETCONF/YANG

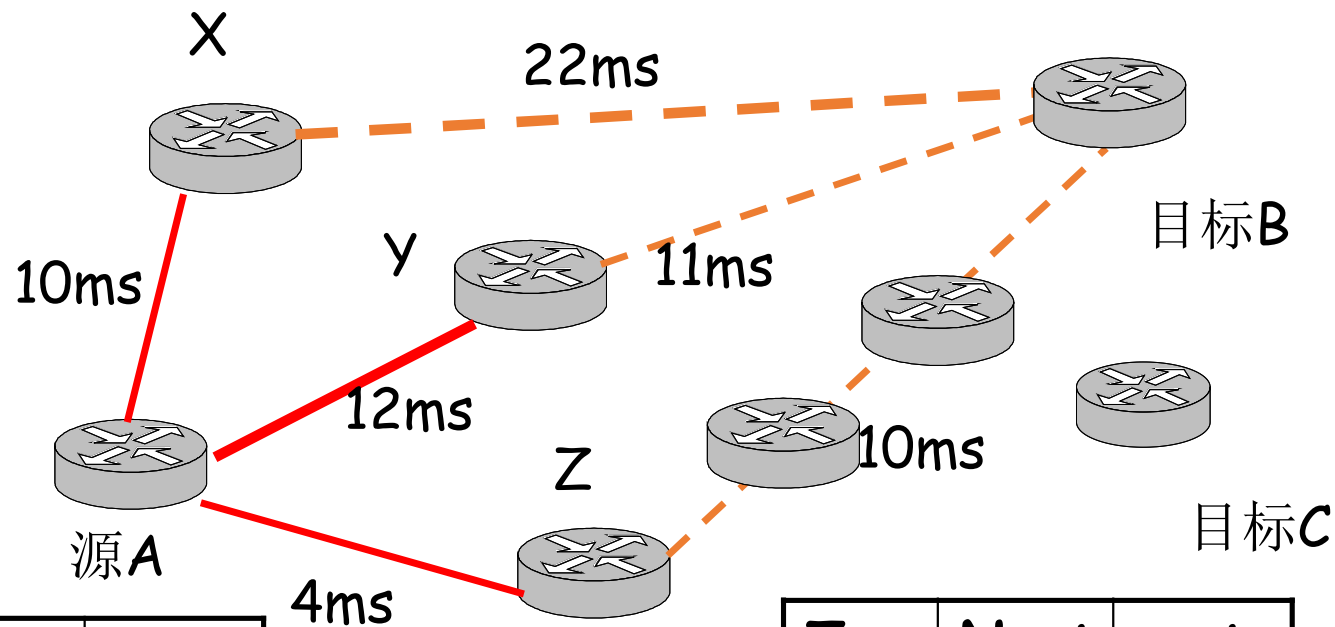


距离矢量路由选择(distance vector routing)

- 动态路由算法
- DV算法历史及应用情况
 - 1957 Bellman, 1962 Ford Fulkerson
 - 用于ARPANET, Internet(RIP)
DECnet, Novell, AppleTalk
- 基本思想
 - 各路由器维护一张路由表, 结构如图
 - 各路由器与相邻路由器交换路由表, 约定好节点的顺序即为距离矢量(待续)
 - 根据获得的路由信息, 更新路由表(待续)

<i>To</i> 目标	<i>Next</i> 下个	<i>cost</i>
A	Z	14
.....		

距离矢量路由选择(distance vector routing)

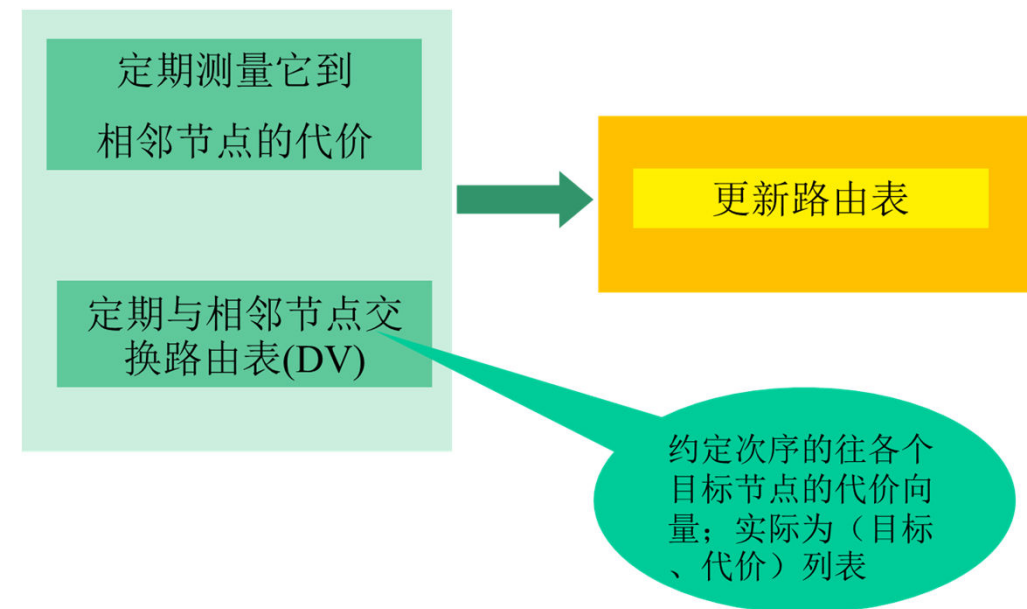


To	Next	cost
B	z	14
.....		

To	Next	cost
B	Q	10
.....		

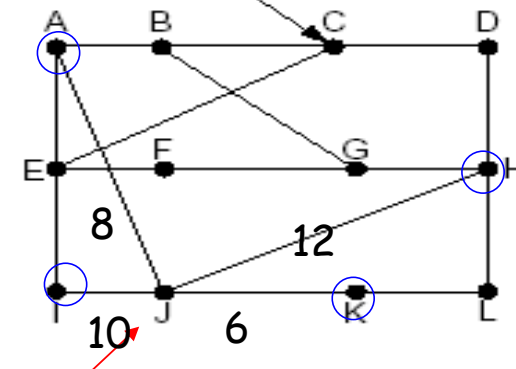
距离矢量路由选择(distance vector routing)

- 到相邻节点间代价和DV获得
 - ✓ 跳数(hops), 延迟(delay), 队列长度
 - ✓ 相邻节点间代价的获得: 通过实测
- 路由信息的更新, 对于所有目标B
 - ✓ 根据实测 得到本节点A到相邻站点的代价 (如:延迟)
 - ✓ 通过交换DV, 各相邻站点声称它们到目标站点B的代价
 - ✓ 计算出本站点A经过各相邻站点到目标站点B的代价
 - ✓ 找到一个最小的代价, 和相应的邻居作为该目标B的下一个节点Z, 到达节点B经过此节点Z, 并且代价为A-Z-B的代价



距离矢量路由：例子

- 以当前节点J为例,相邻节点A,I,H,K
- J测得到A,I,H,K的延迟为
8ms,10ms,12ms,6ms
- 通过交换DV, 从A,I,H,K获得到它们到G的延迟为18ms,31ms,6ms,31ms
- 因此从J经过A,I,H,K到G的延迟为
26ms,41ms,**18ms**, 37ms
- 将到G的路由表项更新为18ms, 下一跳为其中的一个邻居: H
- 其它目标一样计算, 除了本节点J



				New estimated delay from J	
				↓	Line
To	A	I	H	K	
A	0	24	20	21	8 A
B	12	36	31	28	20 A
C	25	18	19	36	28 I
D	40	27	8	24	20 H
E	14	7	30	22	17 I
F	23	20	19	40	30 I
G	18	31	6	31	18 H
H	17	20	0	19	12 H
I	21	0	14	22	10 I
J	9	11	7	10	0 -
K	24	22	22	0	6 K
L	29	33	9	9	15 K

JA	JL	JH	JK
delay	delay	delay	delay
is	is	is	is
8	10	12	6

New routing table for J			
26	41	18	37

Vectors received from J's four neighbors

距离矢量算法

基于 *Bellman-Ford* (BF) 方程（动态规划）：

Bellman-Ford 方程

让 $D_x(y)$ 表示: 从 x 到 y 的最小代价路径的代价
那么:

$$D_x(y) = \min_v \{ c_{x,v} + D_v(y) \}$$

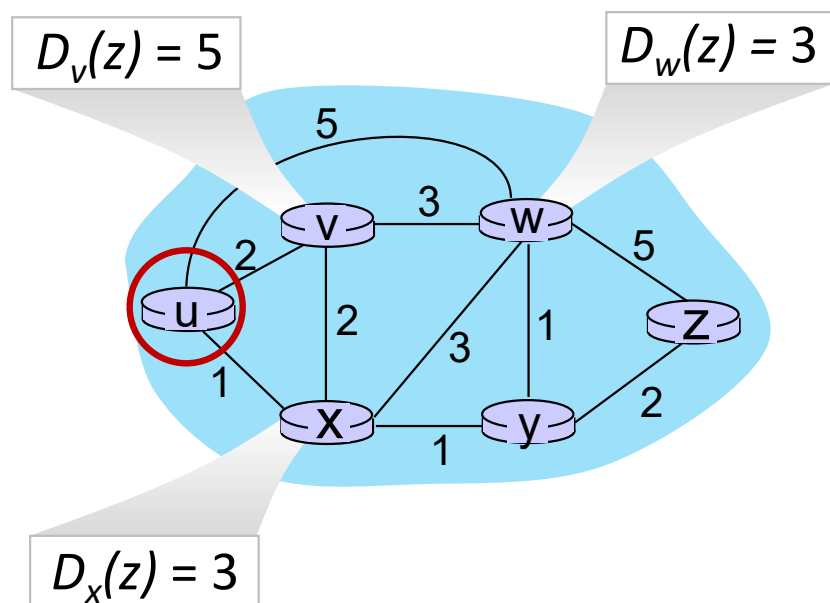
其他邻居节点估计出 v 到 y 最小路径的代价

x 到邻居 v 的直接链路代价

对所有的 x 的邻居 v 做 $\min$ 计算

Bellman-Ford 例子

假设u的邻居节点， x, v, w , 知道到目标z的最优路径代价:



由Bellman-Ford 方程:

$$\begin{aligned}
 D_u(z) &= \min \{ c_{u,v} + D_v(z), \\
 &\quad c_{u,x} + D_x(z), \\
 &\quad c_{u,w} + D_w(z) \} \\
 &= \min \{ 2 + 5, \\
 &\quad 1 + 3, \\
 &\quad 5 + 3 \} = 4
 \end{aligned}$$

让min (x)达成的邻居是到目标的下一跳

距离矢量算法

核心思想:

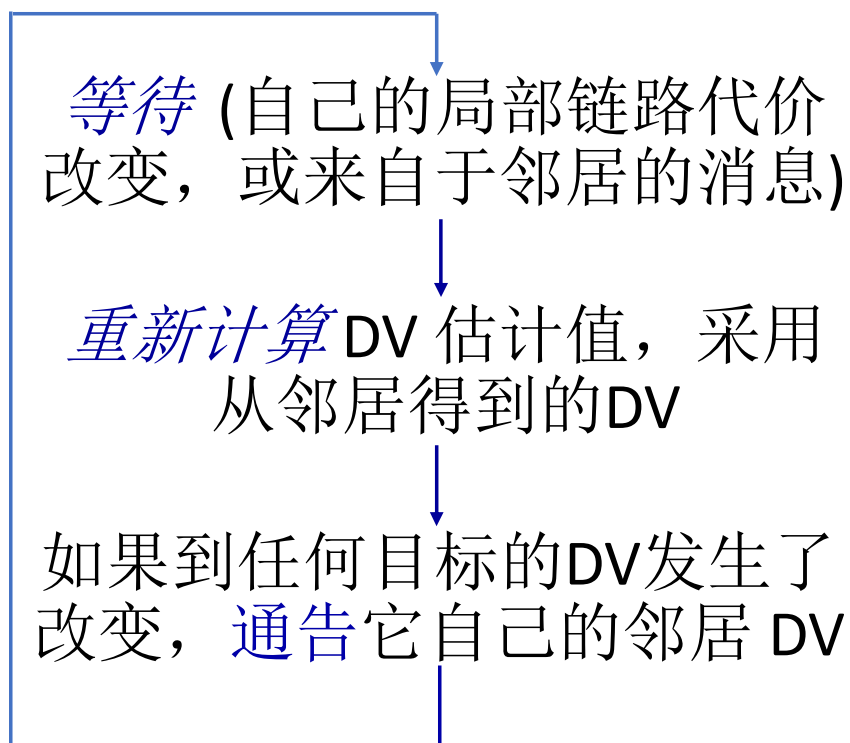
- 迭代式，每个节点将其DV估计发送给邻居
- 当x接收到其任何一个邻居的DV估计时，它采用B-F方程更新自己的DV:

$$D_x(y) \leftarrow \min_v \{c_{x,v} + D_v(y)\} \text{ 对于每个 } y \in N$$

- 在一定条件下，估计值 $D_x(y)$ 最终收敛到实际最小低价 $d_x(y)$

距离矢量算法：

每个节点：

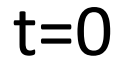


迭代，异步：每个节点局部、迭代计算，由以下事件触发：

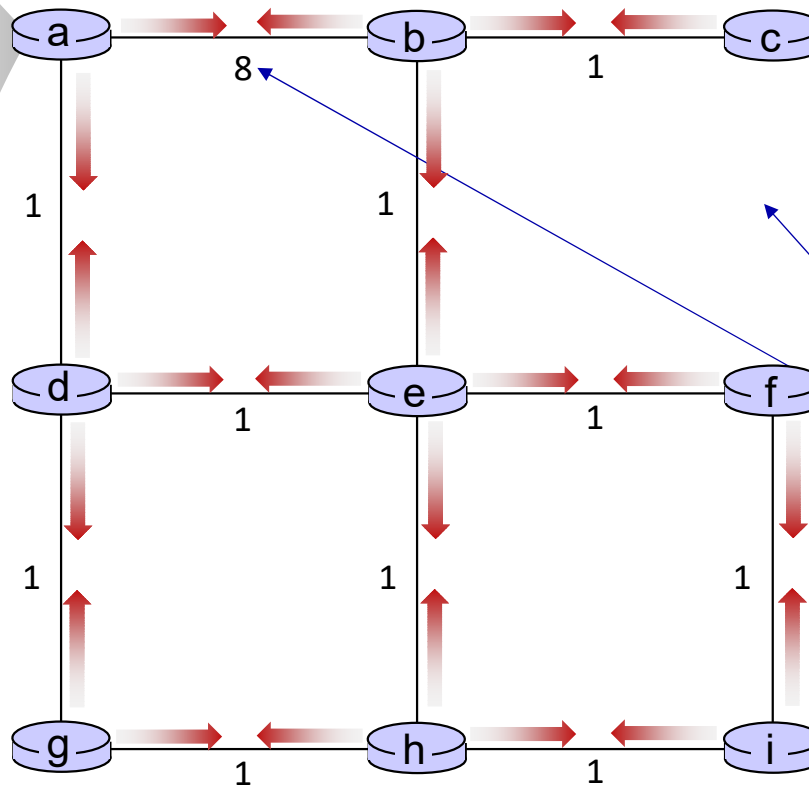
- 局部链路代价的改变
- 从邻居到来的DV更新

分布式，自终止：每个节点只是在它自己的DV发生改变时向邻居通告

- 之后邻居可能通告它们的邻居-在有需要时
- 没有通告收到，那么就无需采取行动！



- 所有节点仅有到其自己邻居的估计值
- 所有的节点发送自己的DV到其邻居

$$\begin{array}{l} D_a(a)=0 \\ D_a(b)=8 \\ D_a(c)=\infty \\ D_a(d)=1 \\ D_a(e)=\infty \\ D_a(f)=\infty \\ D_a(g)=\infty \\ D_a(h)=\infty \\ D_a(i)=\infty \end{array}$$


- 缺失的链路
- 大代价

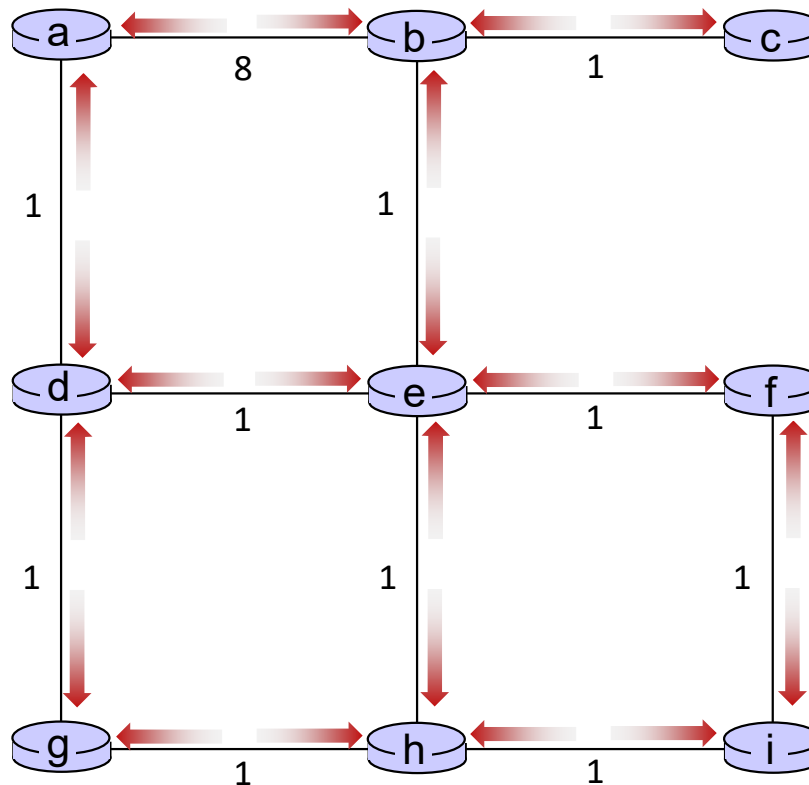
距离矢量例子：迭代



t=1

所有节点:

- 从邻居接收距离矢量
- 计算他们自己的新的本地距离矢量
- 发送它们新的局部距离矢量到它们的邻居



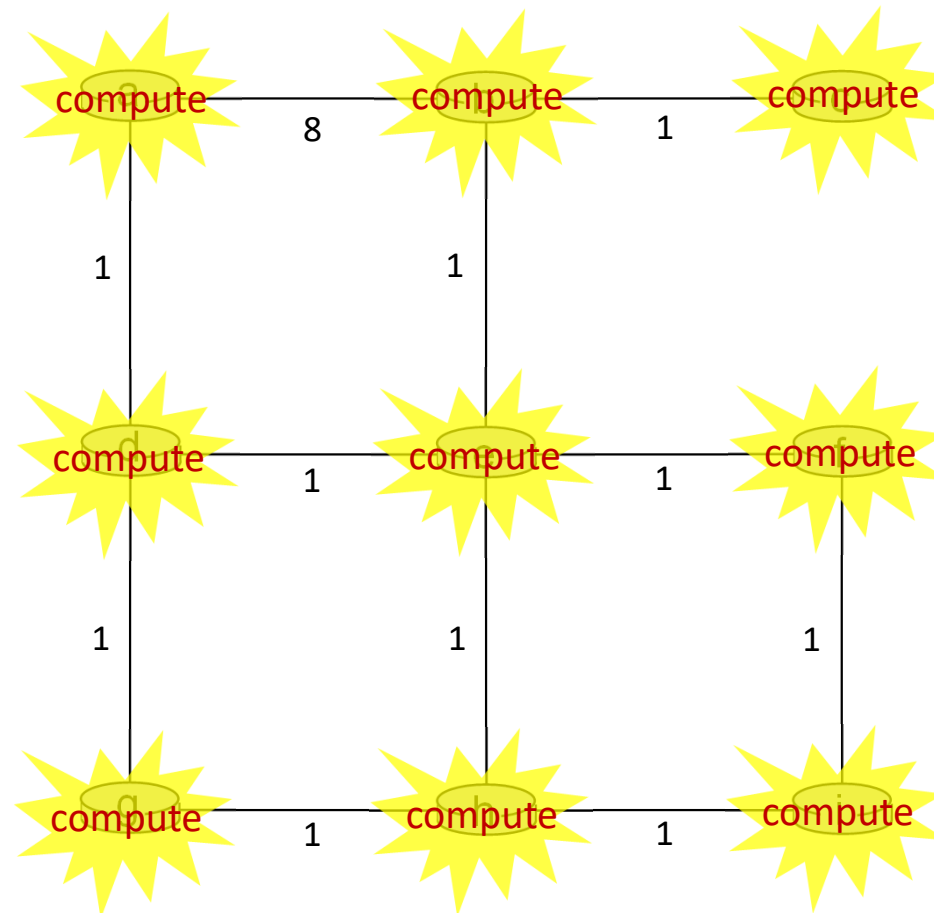
距离矢量例子：迭代



t=1

所有节点:

- 从邻居接收距离矢量
- 计算他们自己的新的本地距离矢量
- 发送它们新的局部距离矢量到它们的邻居



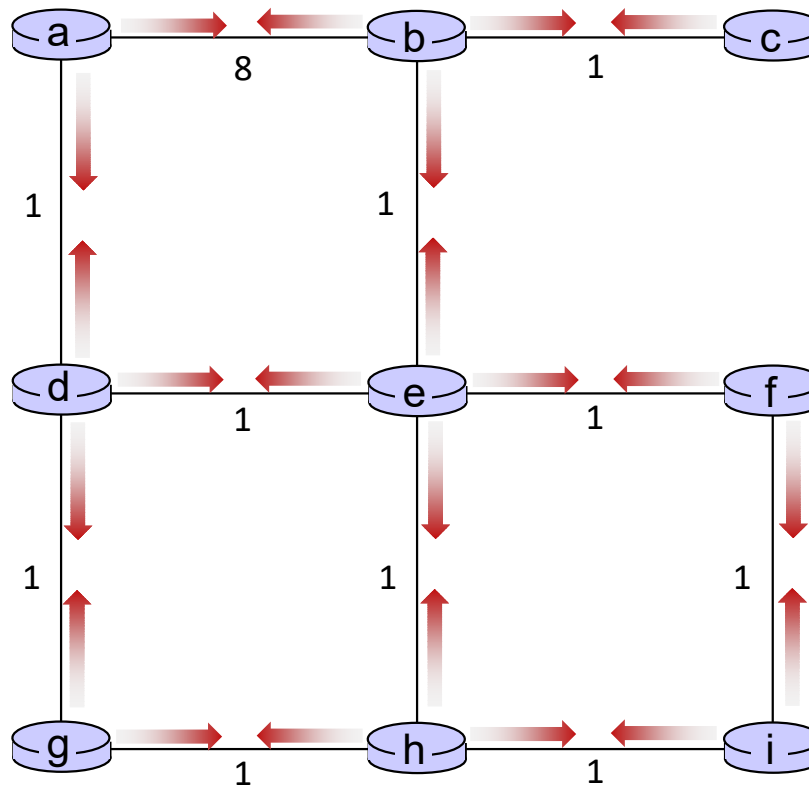
距离矢量例子：迭代



t=1

所有节点:

- 从邻居接收距离矢量
- 计算他们自己的新的本地距离矢量
- 发送它们新的距离矢量到它们的邻居



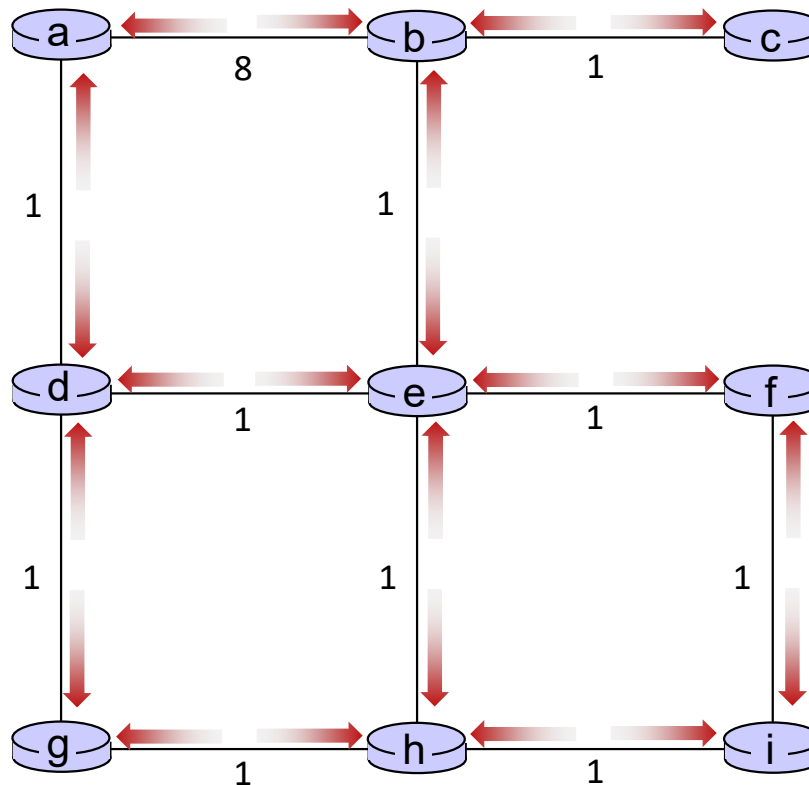
距离矢量例子：迭代



t=2

所有节点:

- 从邻居接收距离矢量
- 计算他们自己的新的本地距离矢量
- 发送它们新的距离矢量到它们的邻居



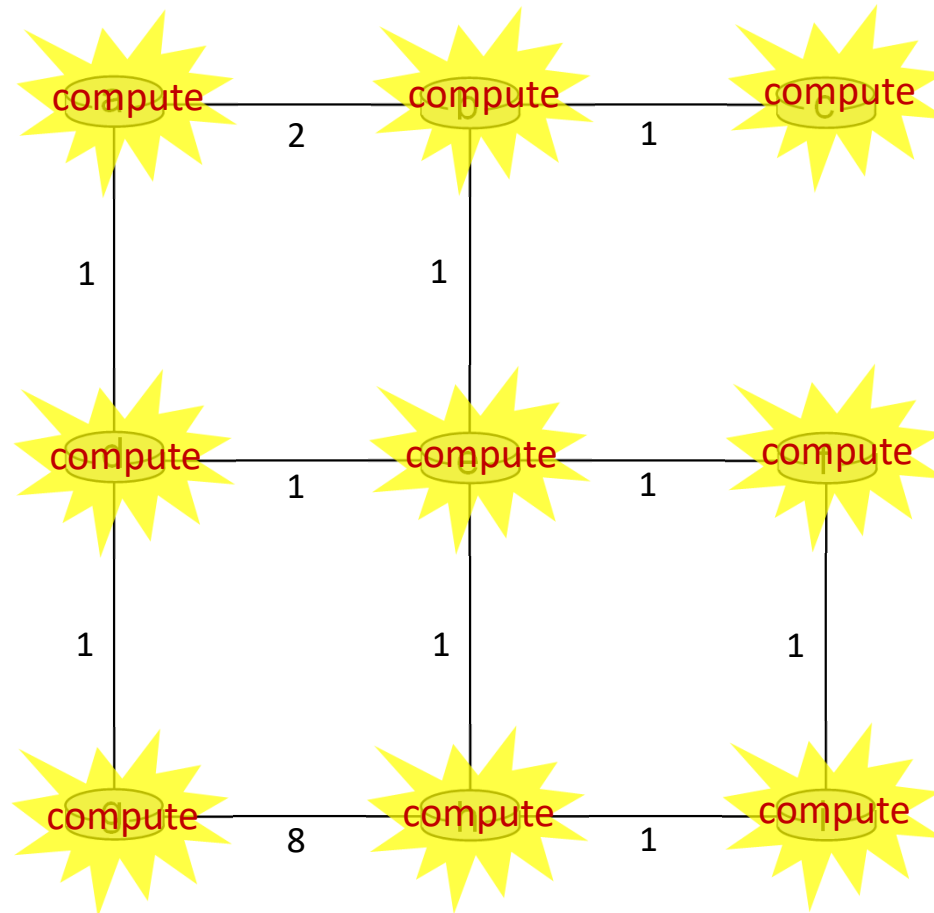
距离矢量例子：迭代



t=2

所有节点：

- 从邻居接收距离矢量
- 计算他们自己的新的本地距离矢量
- 发送它们新的距离矢量到它们的邻居



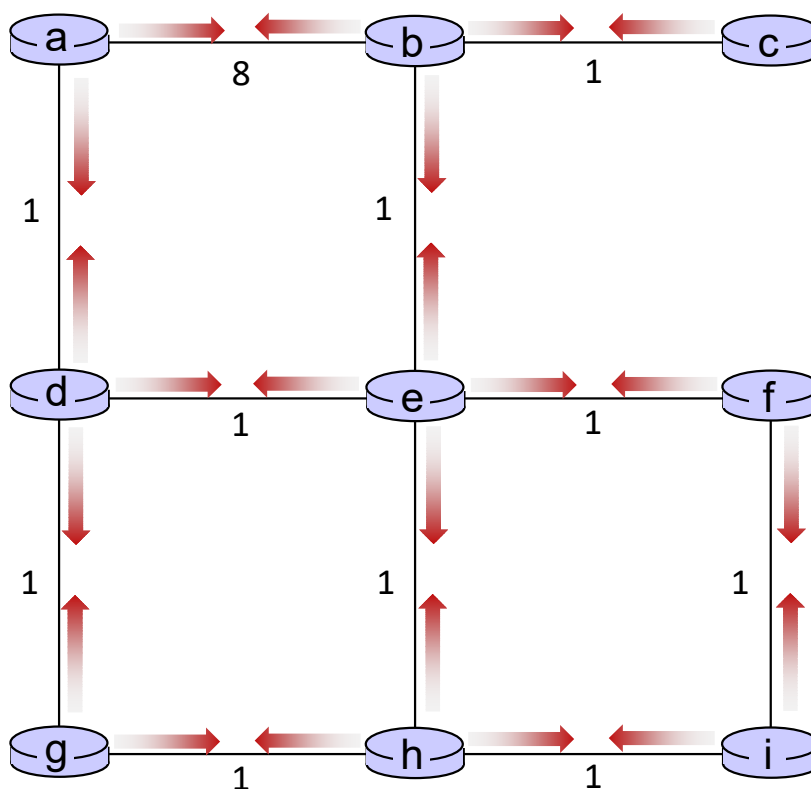
距离矢量例子：迭代



t=2

所有节点:

- 从邻居接收距离矢量
- 计算他们自己的新的本地距离矢量
- 发送它们新的距离矢量到它们的邻居



距离矢量例子：迭代

.... 一直迭代下去

接下来我们来看看节点上的局部迭代计算

距离矢量例子: 计算



t=1

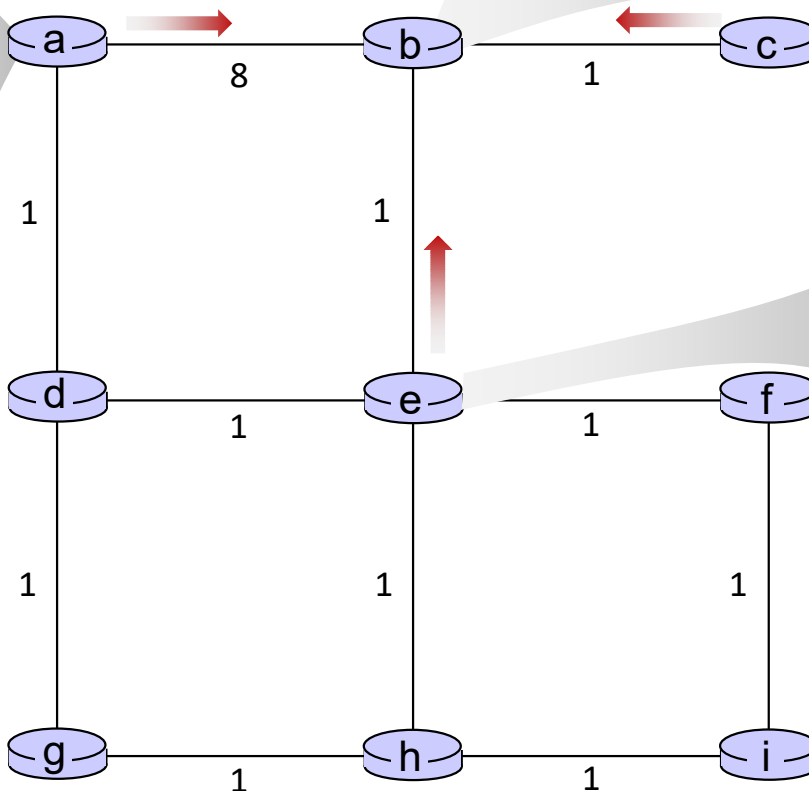
- B从a,c,e收到距离矢量

DV in a:
$D_a(a) = 0$
$D_a(b) = 8$
$D_a(c) = \infty$
$D_a(d) = 1$
$D_a(e) = \infty$
$D_a(f) = \infty$
$D_a(g) = \infty$
$D_a(h) = \infty$
$D_a(i) = \infty$

DV in b:	
$D_b(a) = 8$	$D_b(f) = \infty$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = \infty$	$D_b(h) = \infty$
$D_b(e) = 1$	$D_b(i) = \infty$

DV in c:
$D_c(a) = \infty$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = \infty$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$

DV in e:
$D_e(a) = \infty$
$D_e(b) = 1$
$D_e(c) = \infty$
$D_e(d) = 1$
$D_e(e) = 0$
$D_e(f) = 1$
$D_e(g) = \infty$
$D_e(h) = 1$
$D_e(i) = \infty$



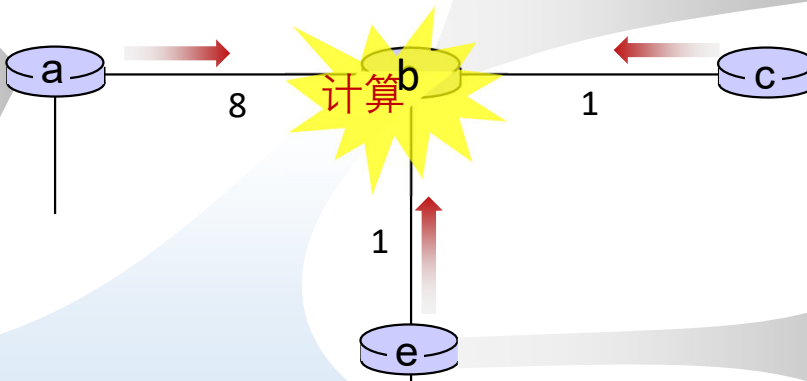
距离矢量例子:计算



t=1

- b 从a,c,e收到距离矢量, 计算:

DV in a:
$D_a(a)=0$
$D_a(b)=8$
$D_a(c)=\infty$
$D_a(d)=1$
$D_a(e)=\infty$
$D_a(f)=\infty$
$D_a(g)=\infty$
$D_a(h)=\infty$
$D_a(i)=\infty$



DV in b:	
$D_b(a) = 8$	$D_b(f) = \infty$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = \infty$	$D_b(h) = \infty$
$D_b(e) = 1$	$D_b(i) = \infty$

DV in c:
$D_c(a)=\infty$
$D_c(b)=1$
$D_c(c)=0$
$D_c(d)=\infty$
$D_c(e)=\infty$
$D_c(f)=\infty$
$D_c(g)=\infty$
$D_c(h)=\infty$
$D_c(i)=\infty$

DV in e:
$D_e(a)=\infty$
$D_e(b)=1$
$D_e(c)=\infty$
$D_e(d)=1$
$D_e(e)=0$
$D_e(f)=1$
$D_e(g)=\infty$
$D_e(h)=1$
$D_e(i)=\infty$

$$\begin{aligned}
 D_b(a) &= \min\{c_{b,a}+D_a(a), c_{b,c}+D_c(a), c_{b,e}+D_e(a)\} = \min\{8, \infty, \infty\} = 8 \\
 D_b(c) &= \min\{c_{b,a}+D_a(c), c_{b,c}+D_c(c), c_{b,e}+D_e(c)\} = \min\{\infty, 1, \infty\} = 1 \\
 D_b(d) &= \min\{c_{b,a}+D_a(d), c_{b,c}+D_c(d), c_{b,e}+D_e(d)\} = \min\{9, 2, \infty\} = 2 \\
 D_b(e) &= \min\{c_{b,a}+D_a(e), c_{b,c}+D_c(e), c_{b,e}+D_e(e)\} = \min\{\infty, \infty, 1\} = 1 \\
 D_b(f) &= \min\{c_{b,a}+D_a(f), c_{b,c}+D_c(f), c_{b,e}+D_e(f)\} = \min\{\infty, \infty, 2\} = 2 \\
 D_b(g) &= \min\{c_{b,a}+D_a(g), c_{b,c}+D_c(g), c_{b,e}+D_e(g)\} = \min\{\infty, \infty, \infty\} = \infty \\
 D_b(h) &= \min\{c_{b,a}+D_a(h), c_{b,c}+D_c(h), c_{b,e}+D_e(h)\} = \min\{\infty, \infty, 2\} = 2 \\
 D_b(i) &= \min\{c_{b,a}+D_a(i), c_{b,c}+D_c(i), c_{b,e}+D_e(i)\} = \min\{\infty, \infty, \infty\} = \infty
 \end{aligned}$$

DV in b:

$D_b(a) = 8$

$D_b(c) = 1$

$D_b(d) = 2$

$D_b(e) = 1$

$D_b(f) = 2$

$D_b(g) = \infty$

$D_b(h) = 2$

$D_b(i) = \infty$

距离矢量例子:计算



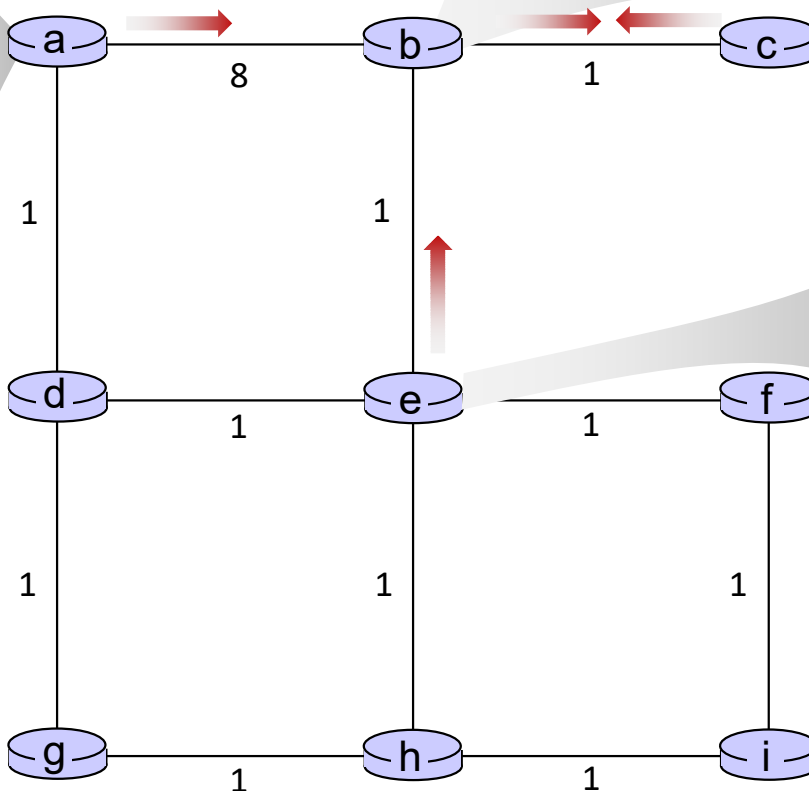
t=1

- c从b收到DV

DV in a:
$D_a(a) = 0$
$D_a(b) = 8$
$D_a(c) = \infty$
$D_a(d) = 1$
$D_a(e) = \infty$
$D_a(f) = \infty$
$D_a(g) = \infty$
$D_a(h) = \infty$
$D_a(i) = \infty$

DV in b:	
$D_b(a) = 8$	$D_b(f) = \infty$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = \infty$	$D_b(h) = \infty$
$D_b(e) = 1$	$D_b(i) = \infty$

DV in c:
$D_c(a) = \infty$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = \infty$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$



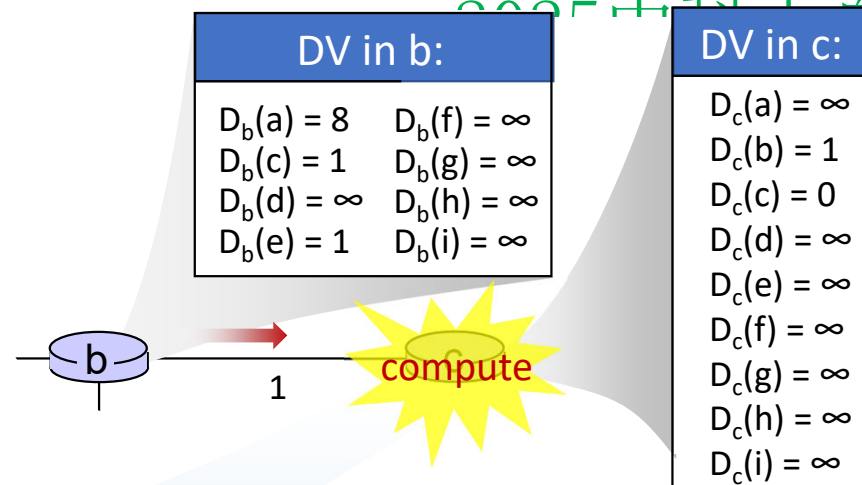
DV in e:
$D_e(a) = \infty$
$D_e(b) = 1$
$D_e(c) = \infty$
$D_e(d) = 1$
$D_e(e) = 0$
$D_e(f) = 1$
$D_e(g) = \infty$
$D_e(h) = 1$
$D_e(i) = \infty$

距离矢量例子:计算



t=1

- c收到来自于b的DV, 计算:



$$\begin{aligned}
 D_c(a) &= \min\{c_{c,b} + D_b(a)\} = 1 + 8 = 9 \\
 D_c(b) &= \min\{c_{c,b} + D_b(b)\} = 1 + 0 = 1 \\
 D_c(d) &= \min\{c_{c,b} + D_b(d)\} = 1 + \infty = \infty \\
 D_c(e) &= \min\{c_{c,b} + D_b(e)\} = 1 + 1 = 2 \\
 D_c(f) &= \min\{c_{c,b} + D_b(f)\} = 1 + \infty = \infty \\
 D_c(g) &= \min\{c_{c,b} + D_b(g)\} = 1 + \infty = \infty \\
 D_c(h) &= \min\{c_{c,b} + D_b(h)\} = 1 + \infty = \infty \\
 D_c(i) &= \min\{c_{c,b} + D_b(i)\} = 1 + \infty = \infty
 \end{aligned}$$

DV in c:
$D_c(a) = 9$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = 2$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$

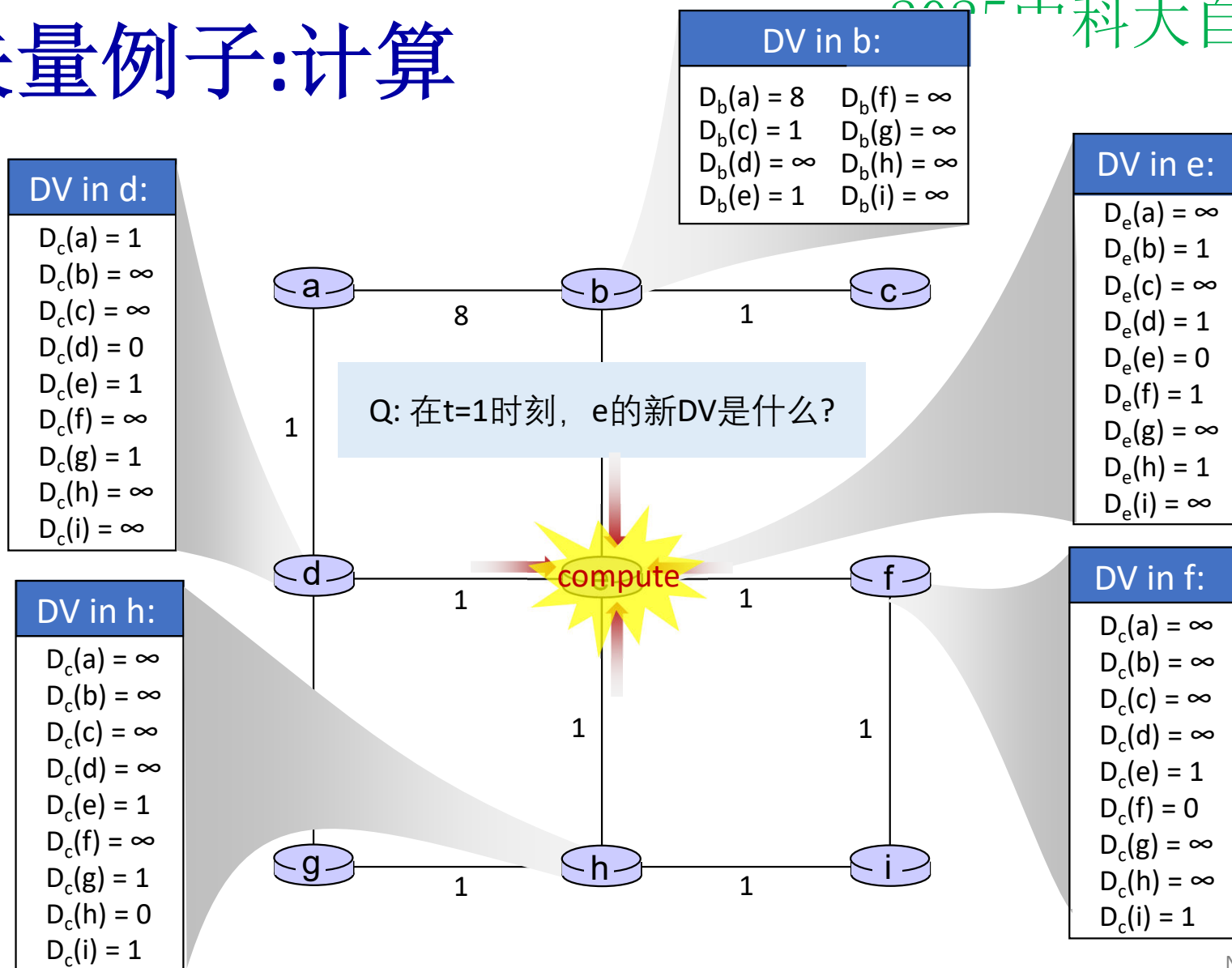
* Check out the online interactive exercises for more examples:
http://gaia.cs.umass.edu/kurose_ross/interactive/

距离矢量例子:计算



t=1

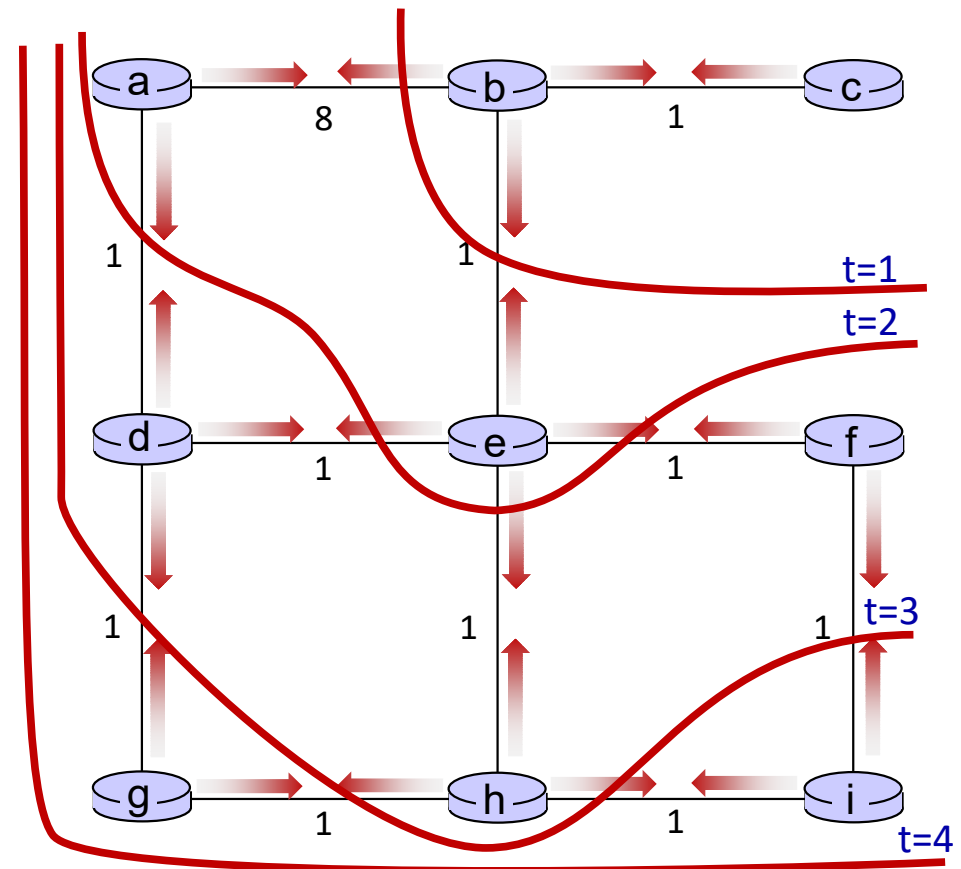
- e收到来自b, d, f, h的距离矢量



距离矢量：状态信息的扩散

迭代式的通告，计算步骤在网络中扩散路由信息：

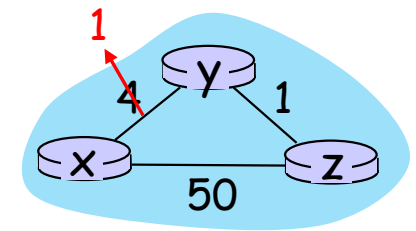
-  $t=0$ c 在 $t=0$ 时刻的状态仅仅在 c 处
-  $t=1$ c 在 $t=0$ 处的状态已经传播到 b ，并且可能影响最多1跳的节点距离矢量计算，即在 b 处
-  $t=2$ c 在 $t=0$ 处的状态现在可能影响最多2跳的节点之距离向量计算，即在 b 处以及现在在 a 、 e 处
-  $t=3$ c 在 $t=0$ 处的状态可能会影响到最多3跳之外的距离向量计算，即在 b 、 a 、 e 处以及现在在 c 、 f 、 h 处
-  $t=4$ c 在 $t=0$ 处的状态可能影响最多4跳之外的距离向量计算，即在 b 、 a 、 e 、 c 、 f 、 h 处以及现在在 g 、 i 处



距离矢量：链路代价改变

链路代价改变:

- 节点检测到局部的链路代价的改变
- 更新路由信息，重新计算其局部的DV
- 如果DV改变，通知邻居



“好消息
传得快”

t_0 : y检测到链路代价的变化，更新其DV，通知它的邻居.

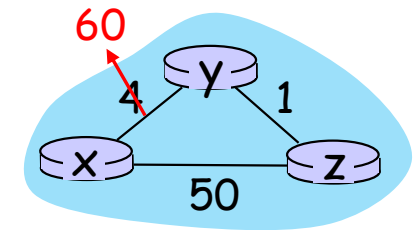
t_1 : z 接收到来自于y的更新，更新它自己的表格，计算到x的新最小代价2，发送DV给它的邻居y.

t_2 : y 接收来自于z的更新（2），试图更新它自己的距离表格，y的最小代价没有变化（1），因此y无需发送消息给z，收敛结束

距离矢量：链路代价改变

链路代价改变:

- 节点检测到链路代价的改变
- “坏消息传的慢” – 无穷计数问题:
 - *y* 检测到到*x*的新代价为60，但是*z*告诉它到*x*的代价为5，所以*y*计算“我到*x*的新代价为6，通过*z*到达；通告*z*其到达*x*的代价为6。
 - *z*获知到达通过*y*到达*x*的路径为6，所以*z*计算“我到*x*的新代价为7，通过*y*到达”。
 - *y*获知通过*z*到达*x*的新代价为7，因此*y*计算“我到达*x*的新代价为8，通过*y*”，通告*z*它到达*x*的新代价为8。
 - *z*或者通过*y*到达*x*的路径新代价为8，因此*z*计算“我到达*x*的新代价为9，通过*y*”，通告*y*其到达*x*的新代价为9
 - ...
- 看教材找解决方案，分布式算法很棘手！



LS和DV算法的对比

消息复杂度

LS: n 个路由器, $O(n^2)$ 消息发送

DV: s 在邻居之间交换信息; 收敛时间是变化的

收敛速度

LS: $O(n^2)$ 算法, $O(n^2)$ 消息复杂度

- 可能有振荡

DV: 收敛时间变化

- 可能存在路由环路
- 无穷计数问题

健壮性: 如果路由器发生了故障, 或者受到损害后会如何?

LS:

- 路由器会通告不正确的链路代价
- 每个路由器仅仅计算器自己的路由表

DV:

- DV路由器可能通告不正确的路径低价 (“我有一个相当低代价的到任何节点的路径”): 黑洞效应
- 每个路由器表被其他路由器使用: 在网络中散播错误

2种路由选择算法都有其优缺点, 而且在互联网上都有应用

网络层：“控制平面”提纲

- 引论
 - 路由算法
 - **ISP内部的路由协议: RIP, OSPF**
 - ISP之间的路由: BGP
 - SDN控制平面
 - Internet Control Message Protocol
- 
- 网络管理，配置
 - SNMP
 - NETCONF/YANG

让路由更加具备扩展性

之前学过的路由算法-太过理想化

- 互联网是通过路由器连接起来的一个个网络，所有路由器的地位是平等的
- 网络“扁平化”：互联网→网络，一个层级

... 如果是一个平面的路由，问题是：

扩展到几百万目标网络很难

- 需要在每个路由器中存储所有其他网络的可达信息，代价太大
- 路由信息的交换将会淹没网络！
- 不可扩展：规模大到不可行

管理自主性无法做到：

- Internet: 是网络的网络
- 每个网络的管理者都希望控制他自己网络的路由方式

互联网采用的可扩展路由方式

将互联网划分为一个个“自治区域autonomous systems” (AS) (a.k.a. “domains”)

AS内 (aka “intra-domain”)路由:
在一个AS内部路由

- 一个AS内部: 所有路由器采用同样的AS内路由协议
- 不同的AS: 可能运行着不同的AS内路由协议
- **网关路由器:** 在自己AS的“边缘”，通过链路连接其他AS网关
- 对于外部AS子网，内部网关协议的意义在于: 决定如何到达网关

AS间inter-AS (aka “inter-domain”)路由: 在AS之间进行路由

- 网关执行AS（域）间的路由（也参与AS内路由的计算），交换和计算AS间的网络可达信息
- 对于外部AS子网，外部网关协议决定通过哪个网关可以到达对于其他AS的网络

互联网的路由协议：层次化路由

外部网关协议（**AS**间路由协议）：

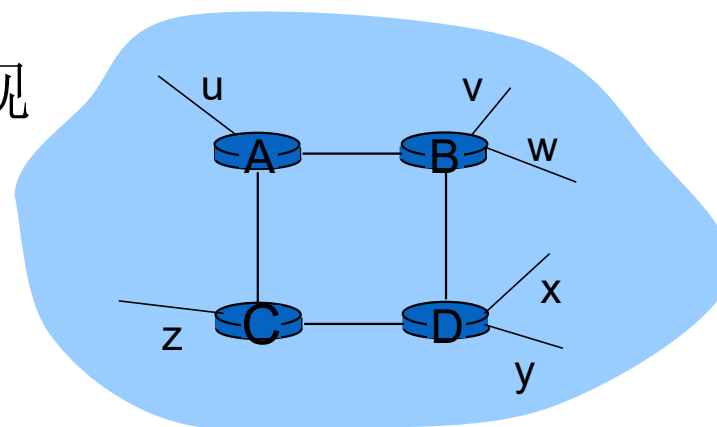
- **BGP: Border Gateway Protocol** [RFC 4271]

内部网关协议（**AS**内路由协议）：

- **RIP: Routing Information Protocol** [RFC 1723]
 - 经典DV：每30秒DS进行交换；不再被广泛应用
- **EIGRP: Enhanced Interior Gateway Routing Protocol**
 - 基于DV，数十年的思科私有协议 (2013年开放[RFC 7868])
- **OSPF: Open Shortest Path First** [RFC 2328]
 - 链路状态路由，IS-IS 协议 (ISO 标准, 不是RFC标准)本质上和OSPF一样

RIP (Routing Information Protocol)

- 在 1982年发布的BSD-UNIX 中被实现
- Distance vector 算法
 - 距离矢量:每条链路cost=1, # of hops (max = 15 hops) 跳数
 - DV每隔30秒和邻居交换DV, 通告
 - 每个通告包括: 最多25个目标子网

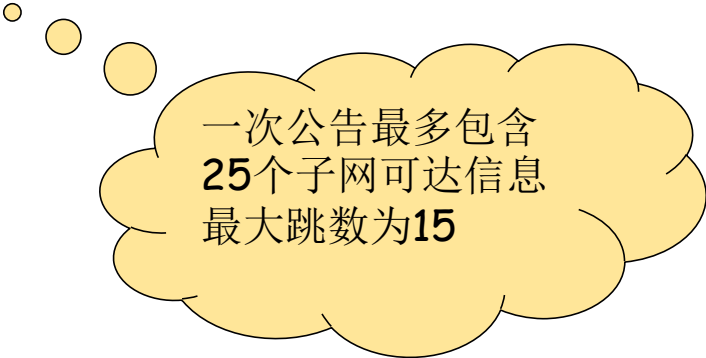


从路由器A到目标子网:

subnet	hops
u	1
v	2
w	2
x	3
y	3
z	2

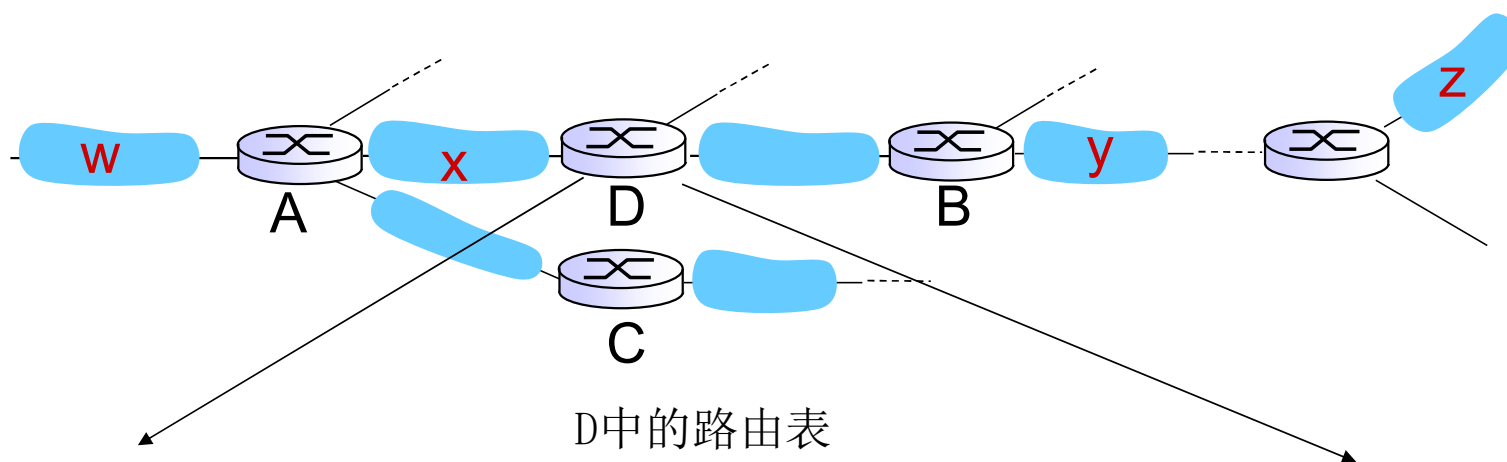
RIP 通告(advertisements)

- DV: 在邻居间每30秒交换通告报文
 - 定期
 - 在改变路由的时候发送通告报文
 - 在对方请求下可以发送通告报文
- 每一个通告: 至多AS内部的25个目标网络的DV
 - 目标网络+跳数



一次公告最多包含
25个子网可达信息
最大跳数为**15**

RIP: 例子



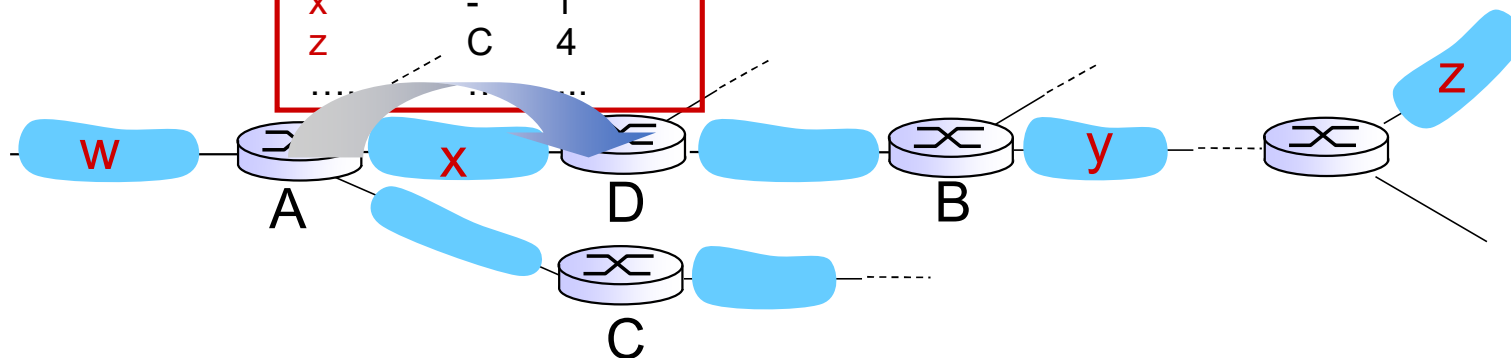
D中的路由表

destination subnet	next router	# hops to dest
W	A	2
Y	B	2
Z	B	7
X	--	1
....		

RIP: 例子

A发给D距离矢量

dest	next	hops
W	-	1
X	-	1
Z	C	4
....		



routing table in router D

destination subnet	next router	# hops to dest
W	A	2
y	B	2
Z	B A	7 5
X	--	1
....		

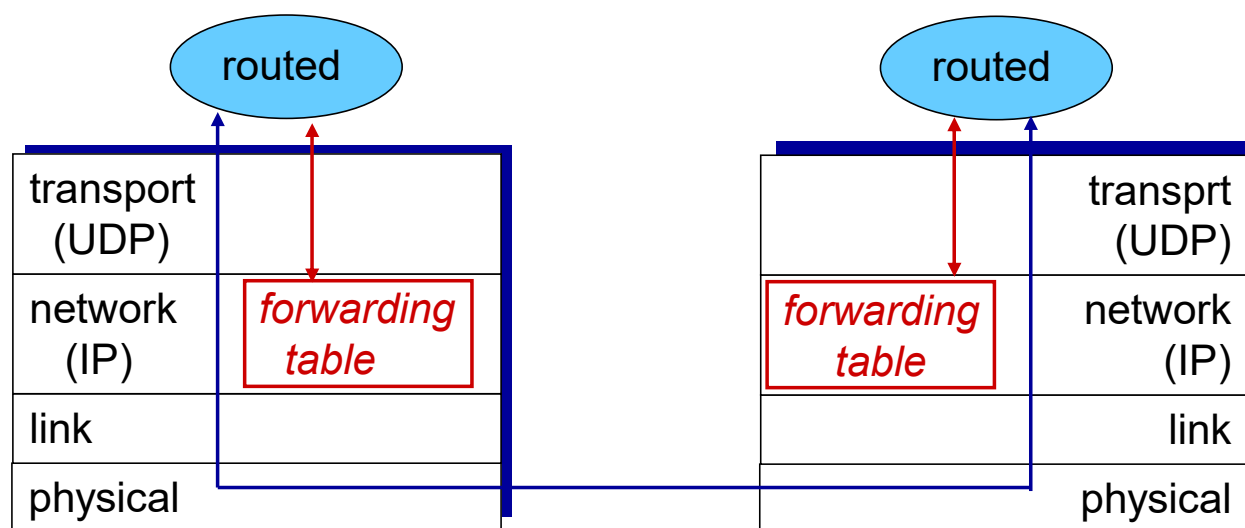
RIP: 链路失效和恢复

如果180秒没有收到通告信息-->邻居或者链路失效

- 发现经过这个邻居的路由已失效
- 新的通告报文会传递给邻居
- 邻居因此发出新的通告 (如果路由变化的话)
- 链路失效快速地在整网中传输
- 使用毒性逆转 (poison reverse) 阻止ping-pong回路 (不可达的距离: 跳数无限 = 16 段)

RIP 由进程实现

- RIP 以应用进程的方式实现：route-d (daemon)
- 通告报文通过UDP报文传送，周期性重复
- 网络层协议使用了传输层的服务，以应用层进程的方式实现



网络层：“控制平面”提纲

- 引论
- 路由算法
- **ISP内部的路由协议: RIP, OSPF**
- ISP之间的路由: BGP
- SDN控制平面
- Internet Control Message Protocol
- 网络管理，配置
 - SNMP
 - NETCONF/YANG



OSPF (Open Shortest Path First) 路由

- “开放”: 公开可获得
- 经典的链路状态LS算法
 - 每个路由器泛洪OSPF链路状态通告（直接通过IP协议，而不是采用TCP或者UDP）给所有AS其他路由器
 - 多个链路代价指标可用：带宽，延迟
 - 每个路由器获悉全局拓扑和边代价，采用Dijkstra算法计算转发表
- 安全: 所有的OSPF报文都是经过认证的（防止恶意攻击）

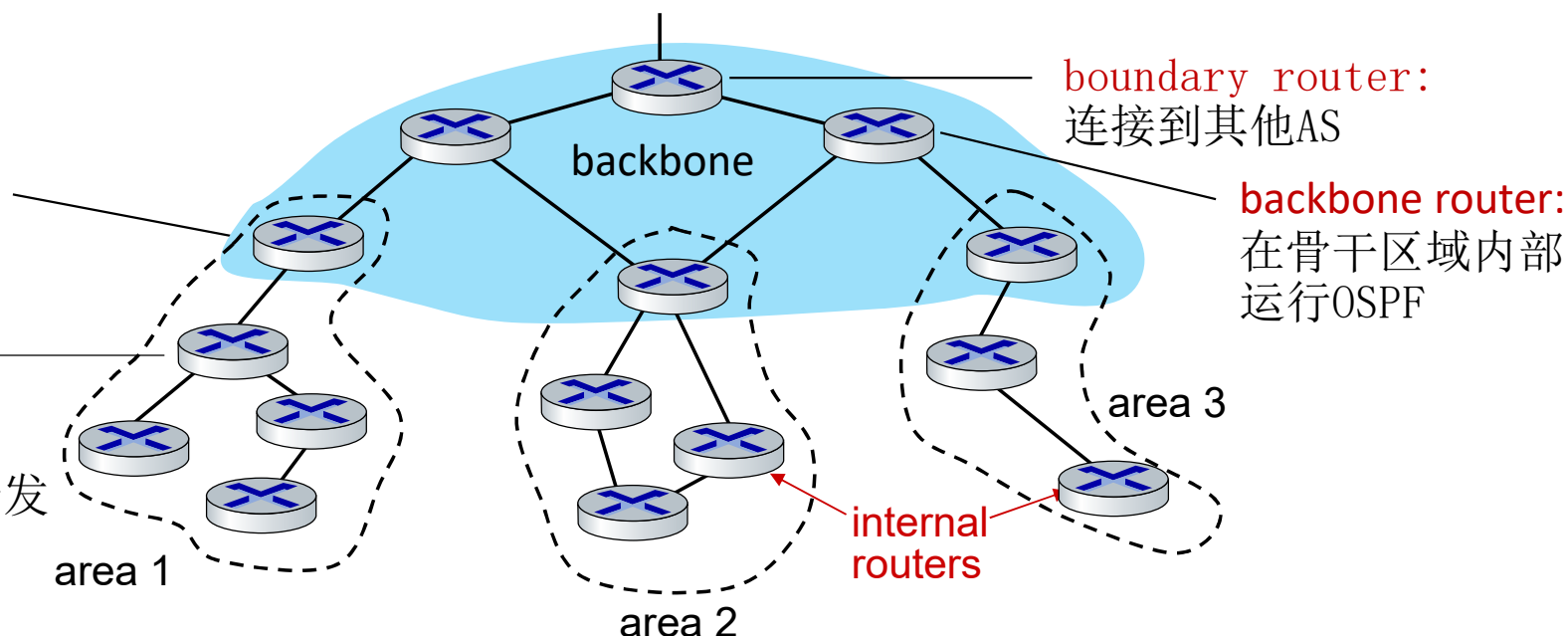
AS内部的层次化 OSPF

- 2个层次的层次性：本地区域，骨干区域
 - 链路状态通告在本区域泛洪，或者骨干区域泛洪
 - 每个节点具备所在区域area详细的拓扑；仅仅知道到其他区域的大致方向

area border routers: “汇总” 到本区域目标的距离，通告给骨干区域

local routers:

- 在区域内部泛洪LS
- 计算区域内部的路由
- 通过区域边界路由器转发分组到其他区域



网络层：“控制平面”提纲

- 引论
- 路由算法
 - 链路状态link state
 - 距离矢量distance vector
- ISP内部的路由协议: RIP,OSPF
- **ISP之间的路由: BGP**
- SDN控制平面
- Internet Control Message Protocol
- 网络管理，配置
 - SNMP
 - NETCONF/YANG



互联网的层次路由

■ 一个平面的路由

- 互联网中所有路由器的地位一样
- 通过LS, DV, 或者其他路由算法, 所有路由器都要知道其他所有路由器（子网）如何走
- 所有路由器在一个平面

□ 平面路由的问题

○ 规模巨大的网络中, 路由信息的存储、传输和计算代价巨大

- ⑩ DV: 距离矢量很大, 且不能够收敛
- ⑩ LS: 几百万个节点的LS分组, 其泛洪传输、存储以及最短路径算法的计算

○ 管理问题:

- ⑩ 不同的网络所有者希望按照自己的方式管理网络
- ⑩ 希望对外隐藏自己网络的细节
- ⑩ 当然, 还希望和其它网络互联

互联网的层次路由

- 层次路由：将互联网分成一个个AS
 - 某个区域内的路由器集合，自治系统 “autonomous systems” (AS)
 - 一个AS用AS Number (ASN)唯一标示
 - 通常一个ISP可能包括1个(或者为数不多的几个AS)

□ 路由变成了：2个层次路由

- AS内部路由 “intra-AS” routing protocol: 内部网关协议
 - 同一个AS内路由器运行相同路由协议
 - 不同AS可运行着不同的内部网关协议
 - 网关路由器：AS边缘路由器可以连接到其他AS
 - 解决AS内部各网络路由器（包括网关）路由的问题
- AS间运行外部网关协议 “inter-AS” routing protocol:
 - ⑩ 解决AS之间的路由问题，完成AS之间（各子网）的互联互通

层次路由的优点

■ 解决了规模问题

- 内部网关协议解决：AS内部数量有限的路由器相互到达的问题，AS内部规模可控
 - 如AS节点太多，可分割该AS，使得AS内部的路由节点数量有限
- AS间的路由规模性问题
 - 增加一个AS，对于AS之间的路由从总体上来说，只是增加了一个节点=子网（每个AS可以用一个点来表示）
 - 对于其他AS来说只是增加了一个表项，就是这个新增AS如何走的问题
 - 扩展性强：规模增大，性能不会减得太多

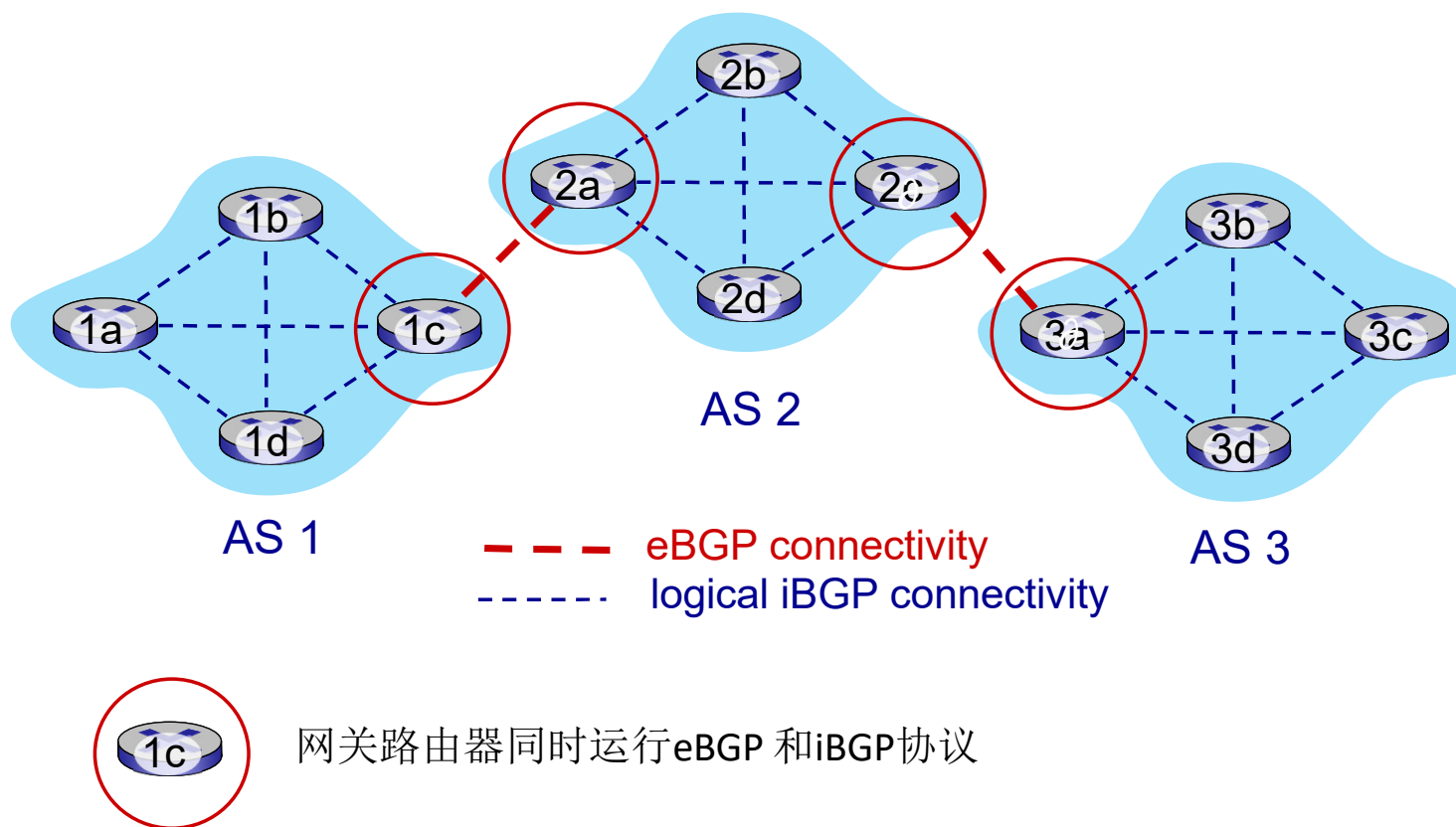
□ 解决了管理问题

- 各个AS可以运行不同的内部网关协议
- 可以使自己网络的细节不向外透露

互联网AS间路由： BGP

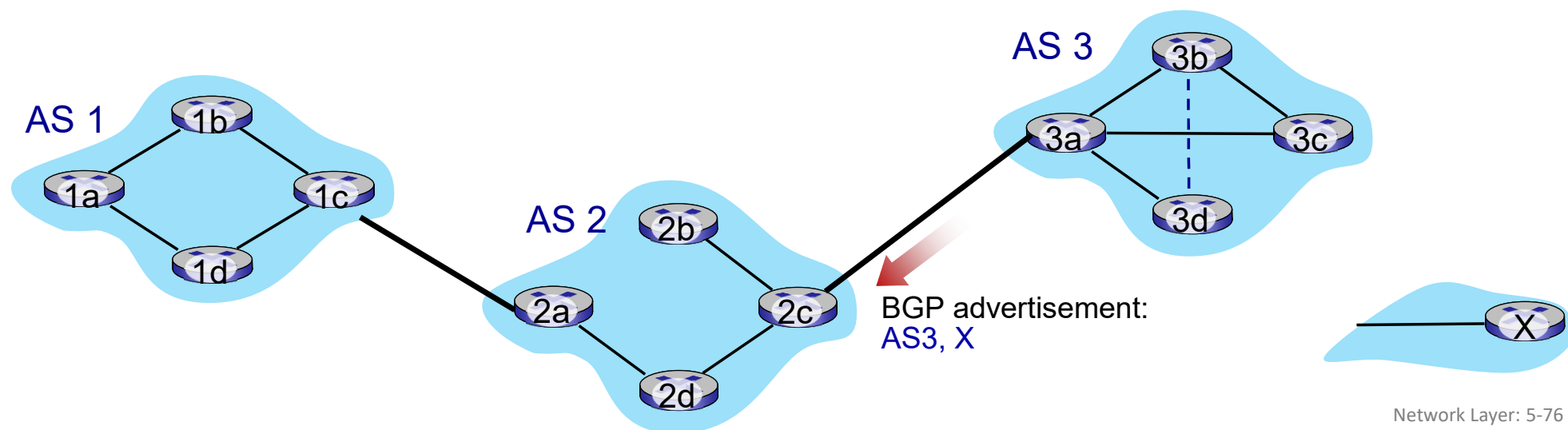
- **BGP (Border Gateway Protocol):** AS间路由事实上的标准
 - “把互联网粘在一起的胶水”
- 允许AS通告其所**拥有网络的可达信息**，以及**通过它可以到达的其他AS目标**
 - 告诉互联网其他部分：“我在这里，通过我可以到达这些目标（本AS内网络和所知道的其他AS网络）”
- BGP提供给每个AS方法，用于
 - **eBGP:** 邻居间AS子网的可达性信息交换
 - **iBGP:** 向AS内路由器传播其获知的其他AS网络可达信息
 - 基于策略，以及获得的子网可达信息以及，决定“好的”到达其他网络的路径
- 基于距离矢量算法（路径矢量）
 - 不仅是距离矢量，还包括到达各个目标网络的详细路径（AS 序列）
 - 能够避免简单DV算法的路由环路问题

eBGP, iBGP 连接



BGP 基础

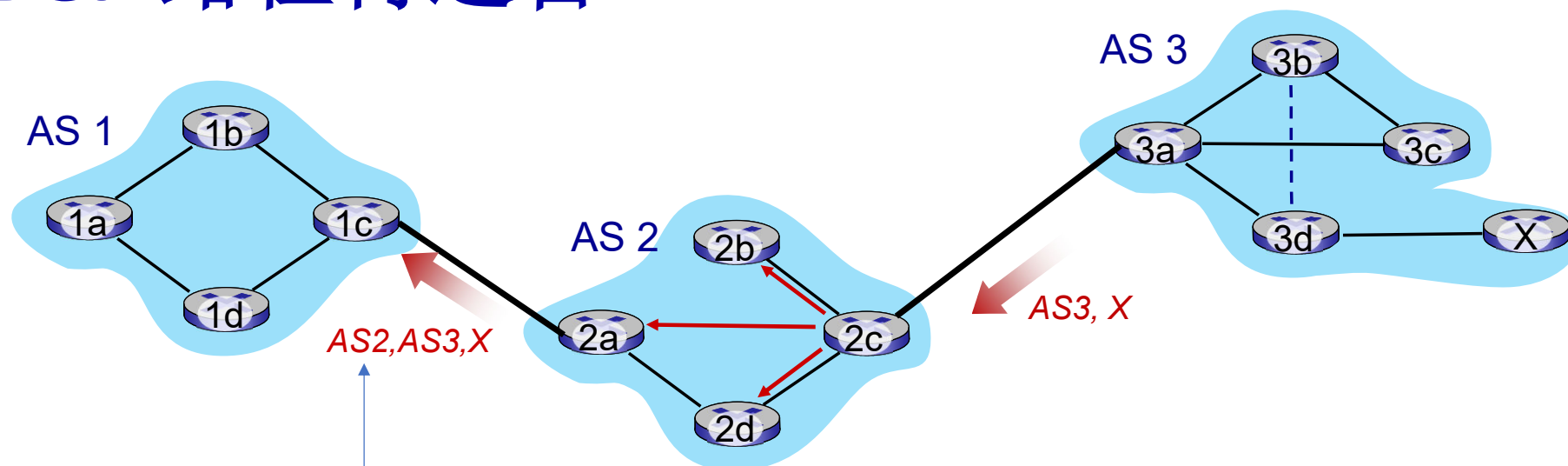
- **BGP会话**：2个BGP路由器(对等体)通过半永久的TCP连接交换BGP报文
 - 通告到不同目标网络前缀的路径（“BGP是一种路径矢量协议，不是DV”）
- 当 AS3的网关3a通告**AS3, x**（目标），给AS2的网关2c：
 - 3a参与AS内路由运算，知道本AS所有子网X信息
 - AS3**承诺**给AS2，它可以转发分组给这些目标x
 - 3a是2c关于X的下一跳（next hop）



路径属性和BGP路由

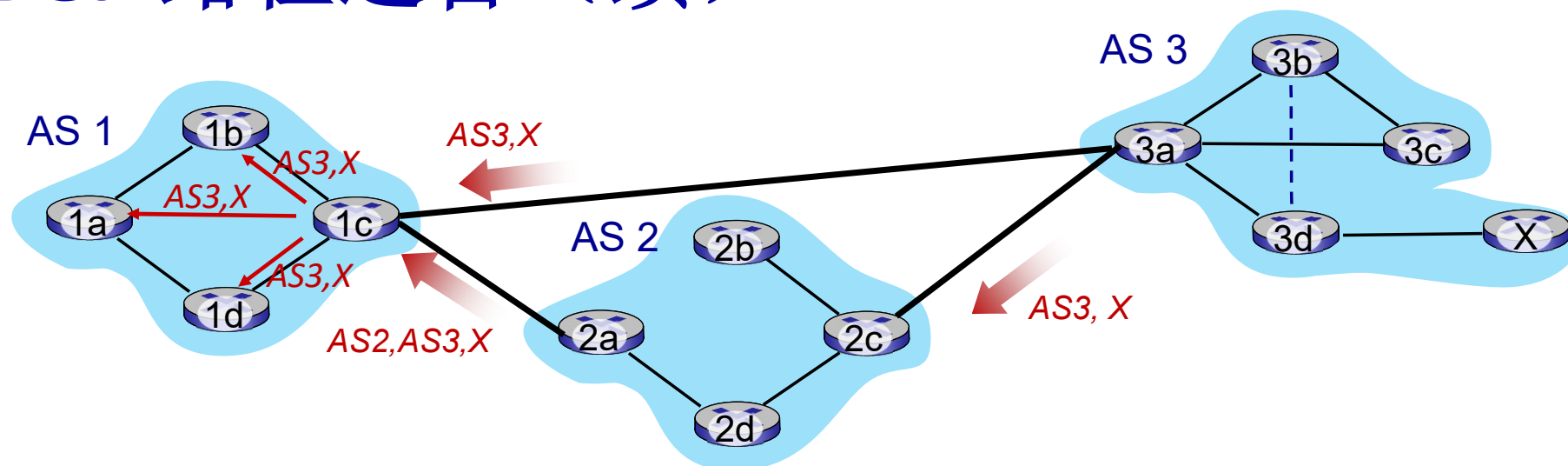
- BGP 通告的路由信息： prefix + attributes
 - prefix: 正在通告能够到达的目标
 - 2个重要的属性:
 - AS-PATH: 通告所经过的AS列表形成的路径
 - NEXT-HOP: 指示到下一跳AS的特定内部AS路由器
 - 其它属性: 路由偏好指标, 如何被插入的属性
- 基于策略的路由:
 - 网关采用输入策略决定收到的路由通告是否被接收还是被丢弃
 - 过滤原因例1: 不想经过某个AS, 转发某些前缀的分组
 - 过滤原因例2: 已经有了一条往某前缀的偏好路径
 - AS策略也决定了是否去再通告该路径到其他邻居AS

BGP 路径再通告



- AS2 路由器 2c从AS3路由器3a收到路由通告 **AS3,X** (通过eBGP)
- 基于AS2的输入策略，AS2路由器2c接受了路径AS3,X，并且通告（通过iBGP）给了所有的AS2路由器
- 根据AS2转发策略，AS2路由器2a（通过eBGP）向AS1路由器1c通告路径**AS2 , AS3 ,X**
 - 路径上加上了 AS2自己作为AS序列的一跳

BGP 路径通告（续）



网关路由器可能学习到到目标的多条路径:

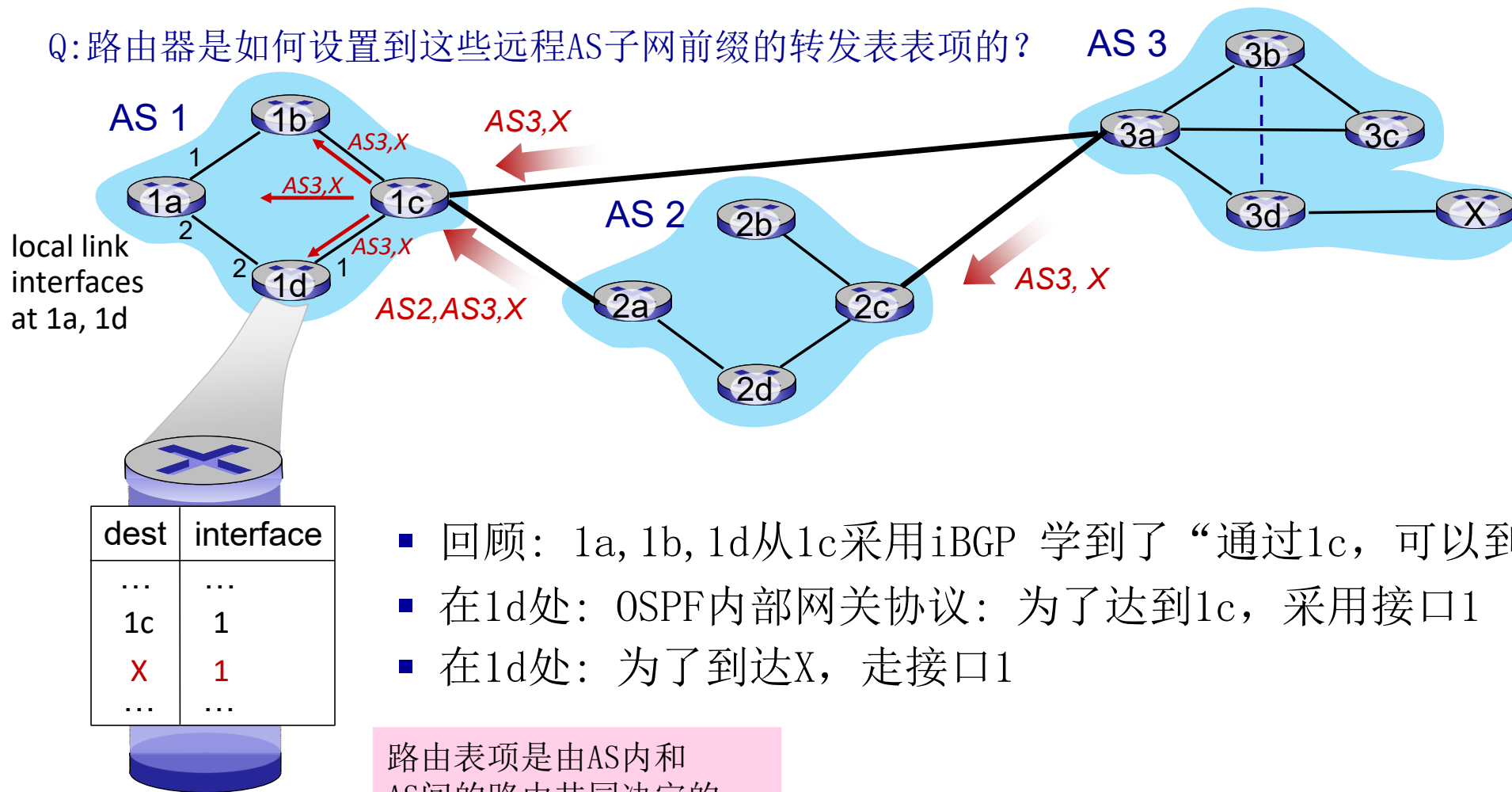
- AS1 网关路由器1c 从2a学到路径 **AS2,AS3,X**
- AS1 网关路由器1c从3a学到路径: **AS3,X**
- 基于策略, AS1网关路由器1c选择**AS3,X**, 而且采用iBGP通告给AS1所有的内部路由器

BGP 报文

- BGP 报文在对等路由器之间交换，采用TCP连接
- BGP 报文：
 - **OPEN**: 打开到远程BGP对等路由器的TCP连接，而且对发送BGP对等体进行身份认证
 - **UPDATE**: 通告新路径（或者收回老的路径）
 - **KEEPALIVE**: 当缺少UPDATE时保持连接；也用于给OPEN请求以确认
 - **NOTIFICATION**: 报告前一个报文的错误；也用来关闭连接

BGP 路径通告

Q: 路由器是如何设置到这些远程AS子网前缀的转发表表项的?

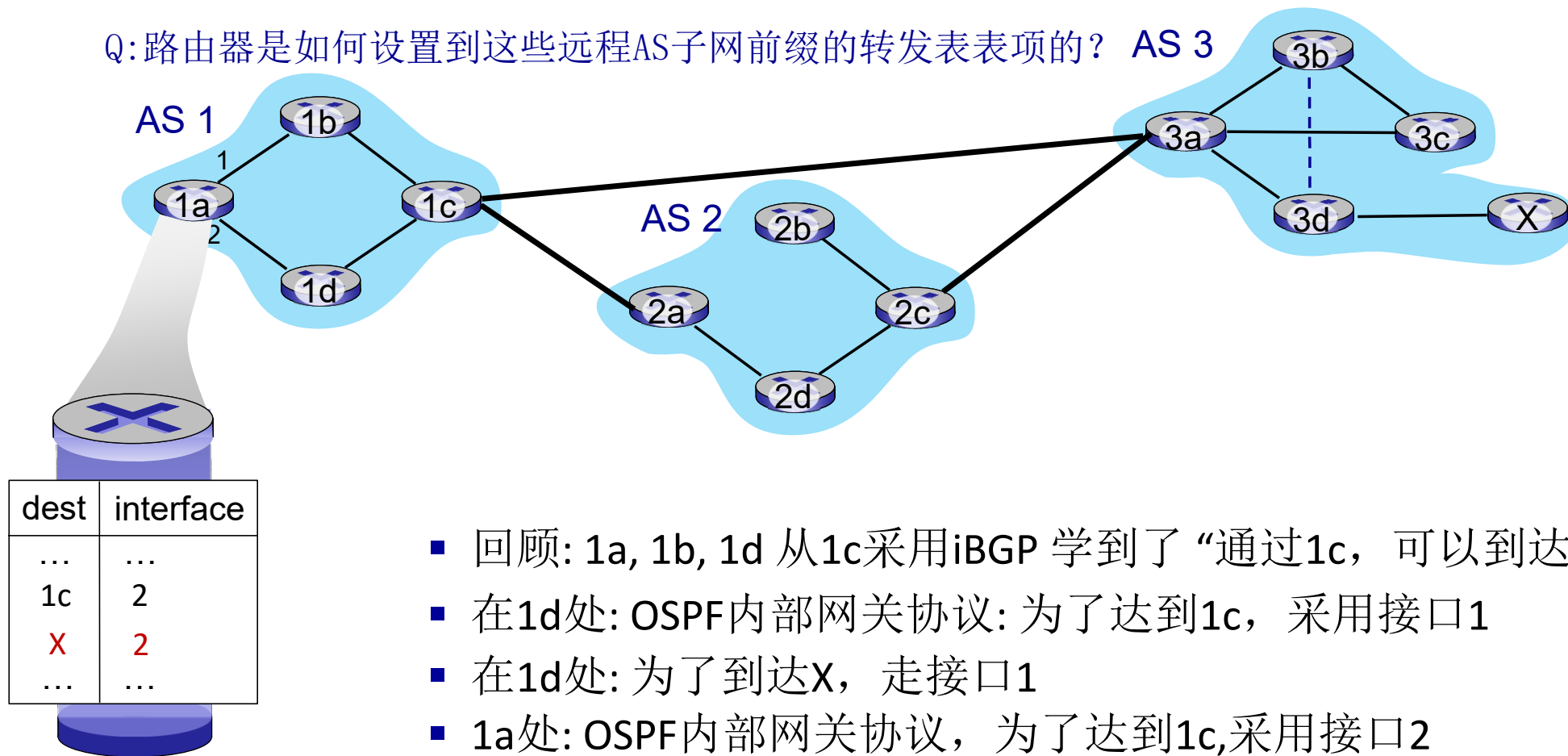


- 回顾：1a, 1b, 1d从1c采用iBGP 学到了“通过1c，可以到达x”
- 在1d处：OSPF内部网关协议：为了达到1c，采用接口1
- 在1d处：为了到达X，走接口1

路由表项是由AS内和AS间的路由共同决定的

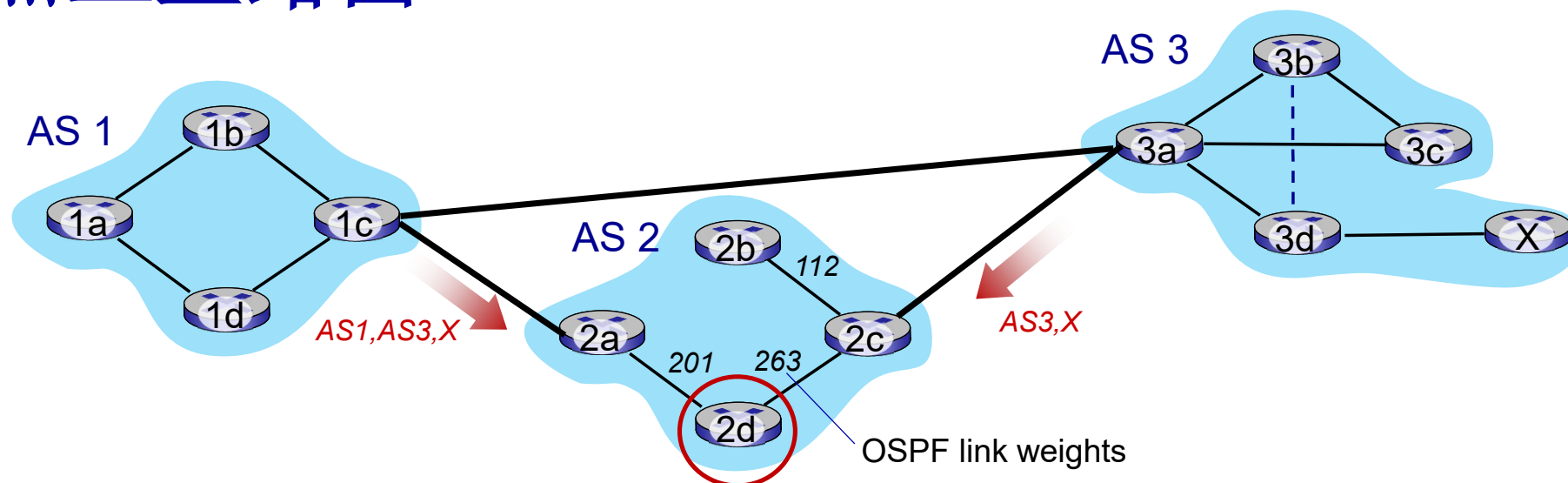
BGP 路径通告

Q: 路由器是如何设置到这些远程AS子网前缀的转发表表项的? AS 3



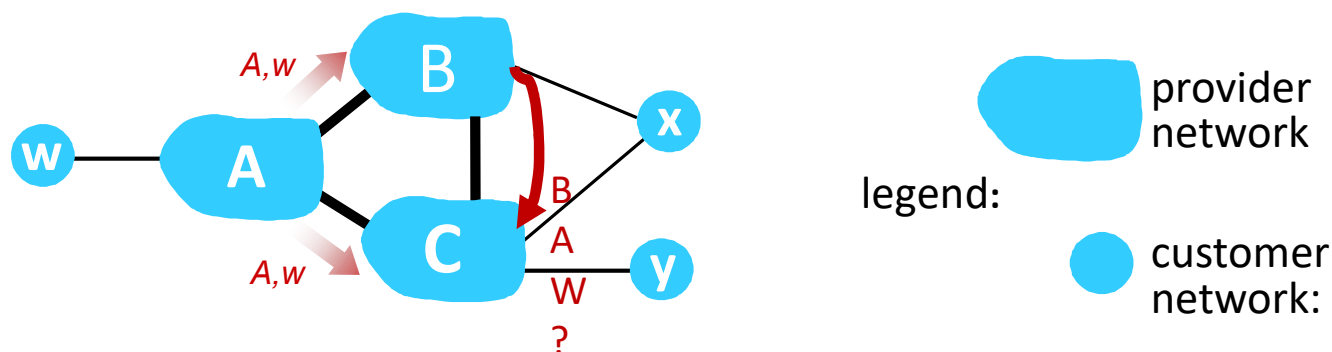
- 回顾: 1a, 1b, 1d 从1c采用iBGP 学到了“通过1c, 可以到达x”
- 在1d处: OSPF内部网关协议: 为了达到1c, 采用接口1
- 在1d处: 为了到达X, 走接口1
- 1a处: OSPF内部网关协议, 为了达到1c, 采用接口2
- 1a处: 为了达到X, 采用接口2

热土豆路由



- 2d 学到 (通过iBGP)它可以通过2a或者2c到达子网X
- **热土豆策略**: 选择一个具备AS内代价最低的本地网关（例如：2d 选择了2a，即使需要较长的AS序列才能够到达X）：不考虑AS间路由代价！

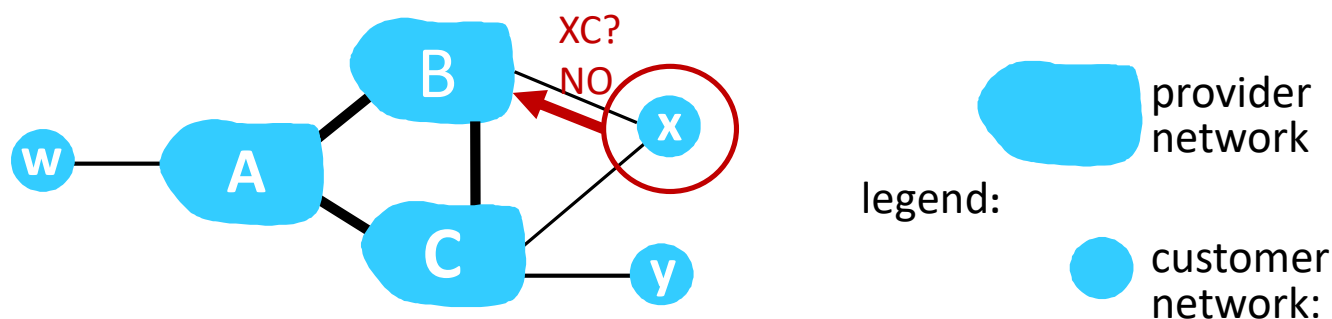
BGP: 通过通告来实现策略



ISP希望仅仅路由进出于它客户的流量 (不希望承载其他ISP的过境流量-一个典型的“现实世界”的策略)

- A 通告路径 Aw 给 B和 C
- B选择不通告 BA_w给C!
 - B 从路由CBA_w无法获得“收入”，因为不管是C, A, w的哪一位都不是B的客户
 - C 无法学习到 CBA_w 路径
- C将会路由CA_w到w （而不是采用B）

BGP: 通过通告实现策略（续）



ISP希望仅仅路由进出于它客户的流量 (不希望承载其他ISP的过境流量-一个典型的“现实世界”的策略)

- A,B,C 是**提供商网络 provider networks**
- x,w,y 是**客户 customer** (提供商网络的客户)
- x 是**双接入 dual-homed**: 接入到2个网络
- **执行的策略**: x不想B通过它路由到C
 - ..所以x不会向B通告它可以路由到C

BGP 路由选择

- 路由器可能学到到目标AS的不止一条路由，基于以下因素进行选择：
 1. 局部偏好值属性：对AS序列进行策略函数的打分
 2. 最短的AS路径
 3. 最近的下一跳路由器：热土豆路由
 4. 附加评判标准

为什么AS内，AS间路由如此不同？

策略：

- 资源属AS间：管理者希望控制其路由，以及谁通过其网络进行路由
- AS内：网络于同一个管理者，因此更加关注性能而不是策略
 - AS内部各子网的路由器尽可能地利用资源进行快速路由

可扩展性：

- AS间路由必须考虑规模问题，以便支持全网的数据转发
- AS内部路由规模不是一个大的问题
 - 如果AS太大，可将此AS分成小的AS；规模可控
 - 对于AS间路由只不过增加了一个点而已

性能：

- AS内：聚焦于性能
- AS间：策略远重要于性能

网络层：“控制平面”提纲

- 引论
- 路由算法
- ISP内部的路由协议: RIP, OSPF
- ISP之间的路由: BGP
- **SDN控制平面**
- Internet Control Message Protocol
- 网络管理，配置
 - SNMP
 - NETCONF/YANG

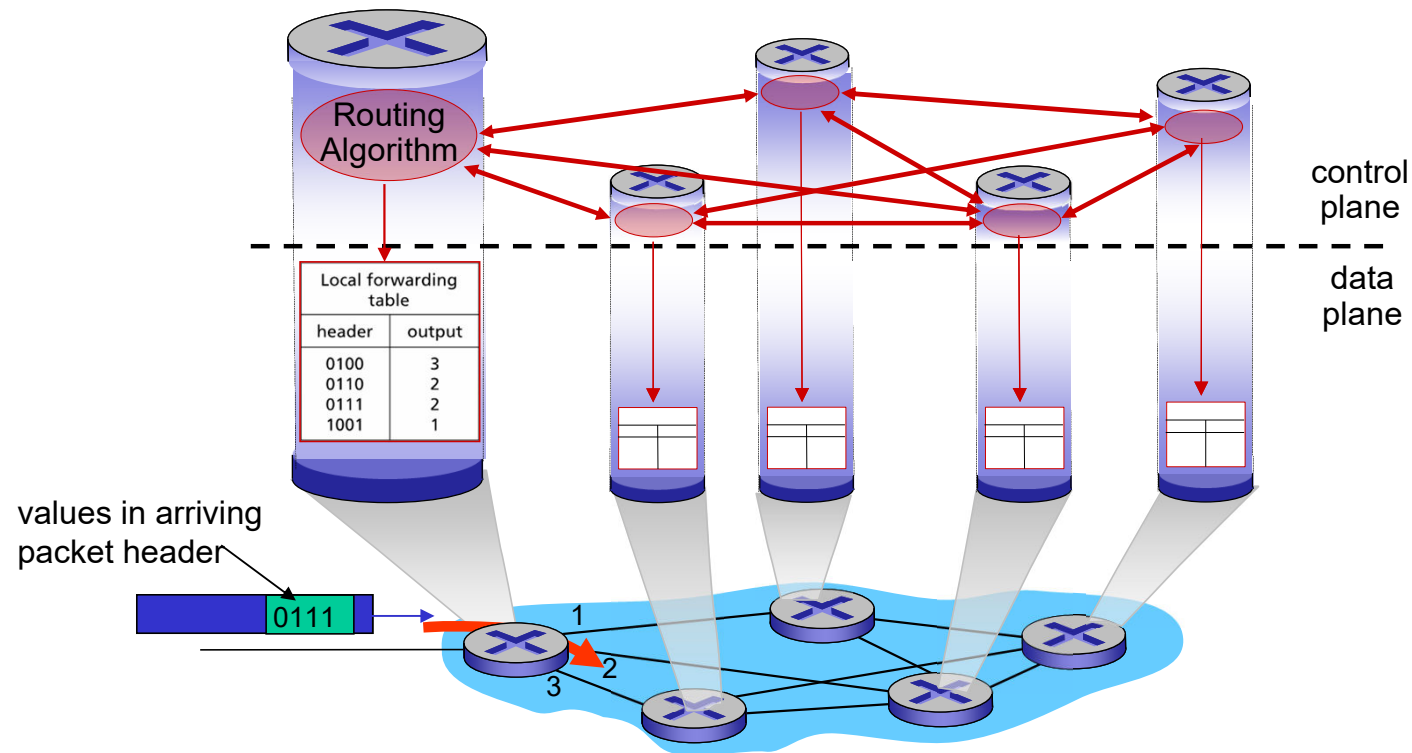


Software defined networking (SDN)

- 互联网网络层：历史上通过分布式，每路由器控制方法实现：
 - 单个路由器包含交换硬件，在专有路由器操作系统（如Cisco IOS）中运行互联网标准协议（IP、RIP、IS-IS、OSPF、BGP）的私有实现
 - 不同的“中间盒”实现不同的网络层功能：防火墙，负载均衡，NAT盒子.....
- ~2005:重新兴起再思考网络控制平面的兴趣

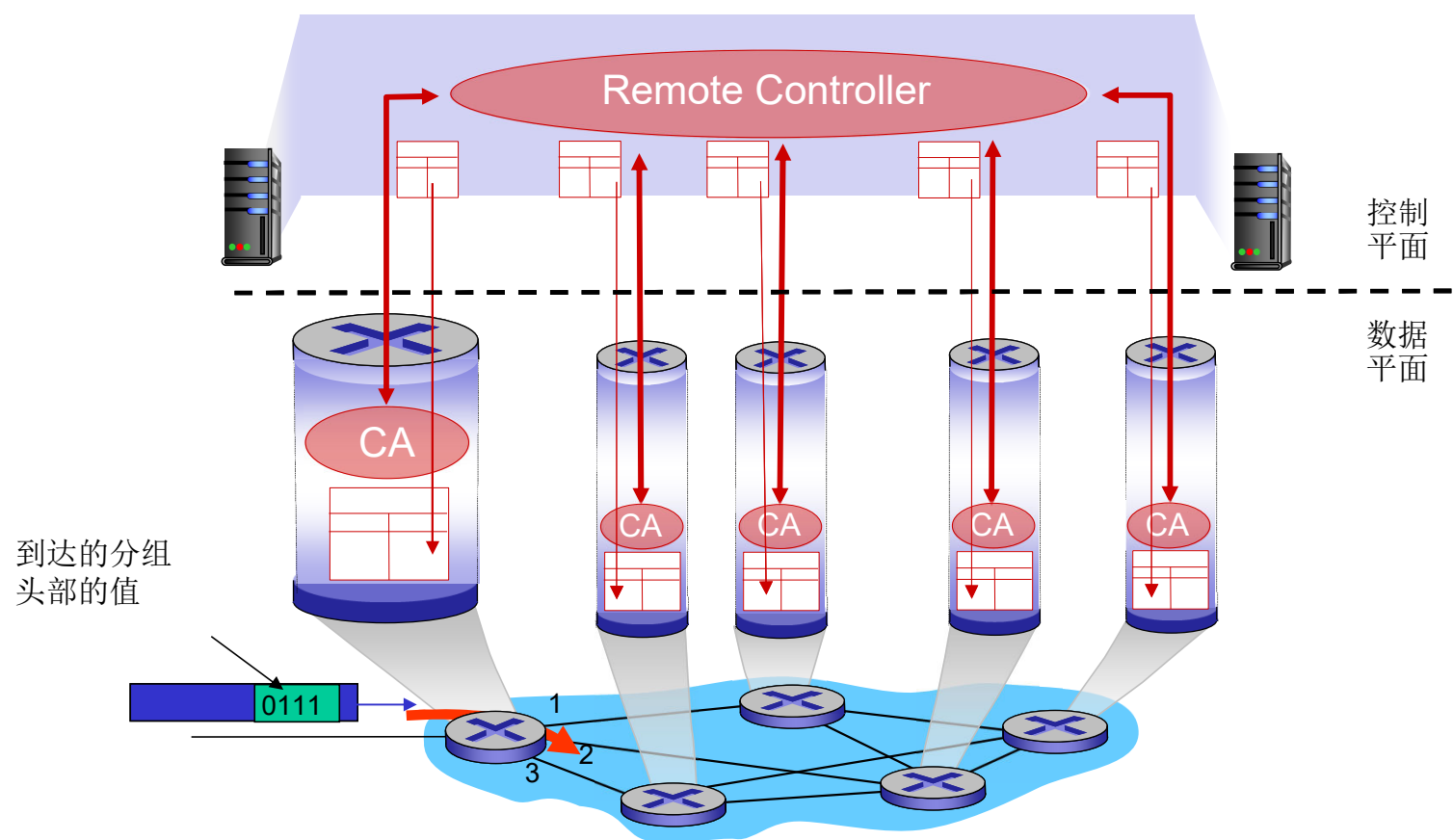
每路由器控制平面

独立的路由算法元件运行在**每个路由器**中，路由器在控制平面进行交互



Software-Defined Networking (SDN) 控制平面

远程的控制器计算，安装交换设备上的转发表



Software defined networking (SDN)

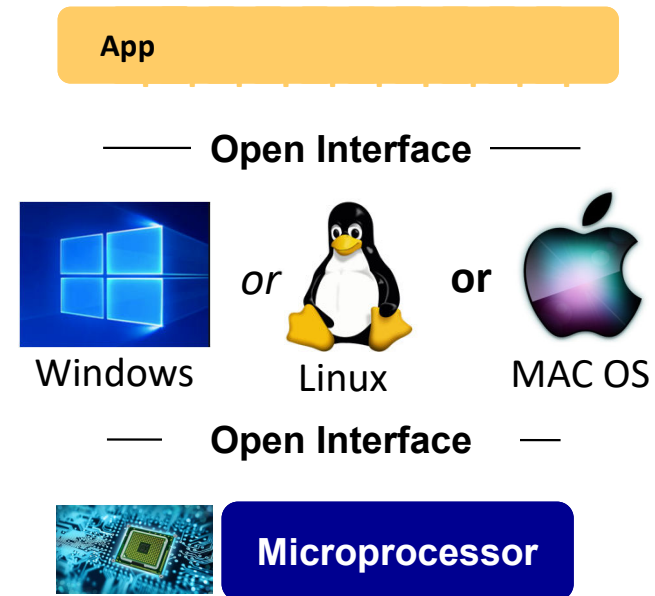
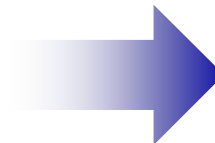
为什么需要一个逻辑上集中的控制平面?

- 更简单的网络管理：避免如有器错误配置，具有更大的流量灵活性
- 基于表的转发（回顾OpenFlow API）允许“可编程”路由器
 - 集中化的“可编程”更加容易：中心化计算表项，分发表项
 - 分布式“可编程”更加困难：每个路由器上实现分布式算法（协议），其计算结果为表项
- 控制平面的开放（而不是专有）实现
 - 快速创新：百花齐放

SDN 类比: 主框架计算机和PC革命

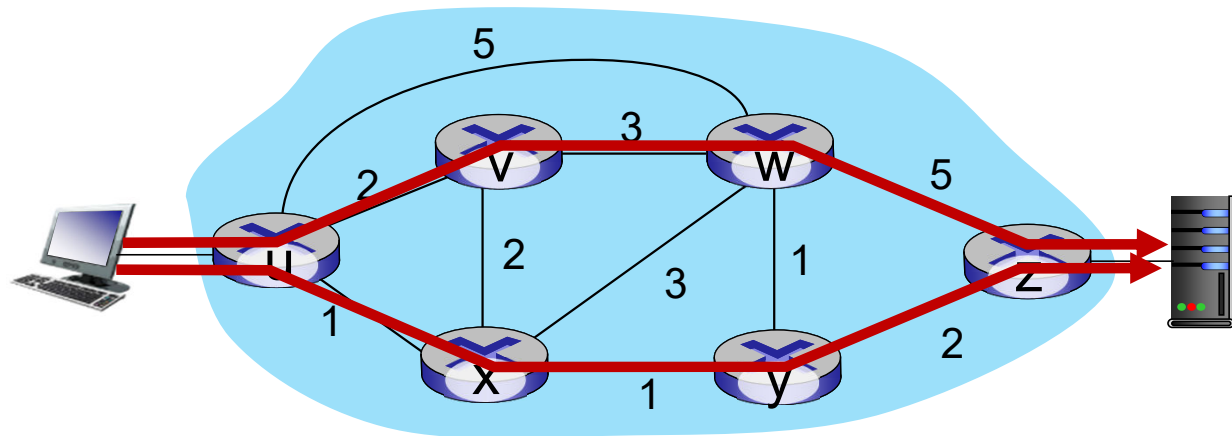


垂直集成
封闭，是有
创新速度慢
产业规模小



水平集成
开放接口
快速创新
产业规模巨大

流量工程：传统路由方法比较困难

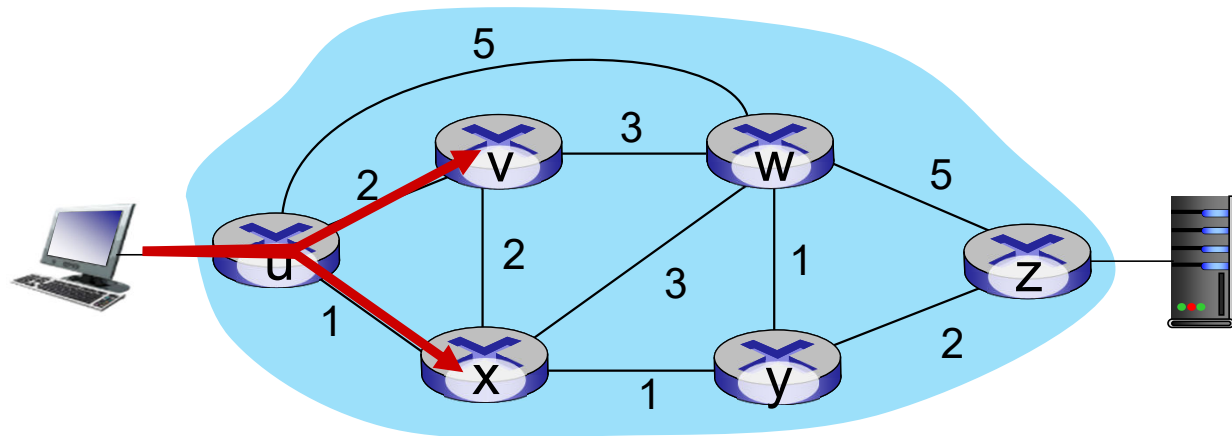


Q: 网络运行着希望u-z的流量通过uvwz，而不是uxyz，如何进行控制？

A: 需要重新定义链路的权重，这样流量路由算法计算相应的路由（或者需要一个新的路由选择算法）！

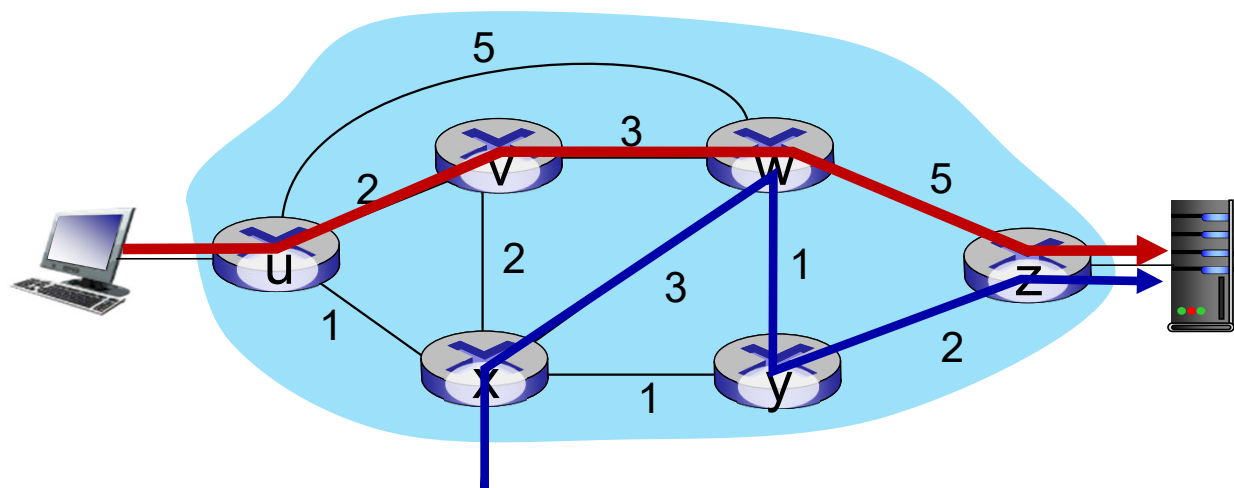
链路权重是仅有的控制“旋钮”，没有太多的控制方法！

流量工程：传统路由方法比较困难



Q: 如果网络运行着希望将u-z的流量分岔（负载均衡）沿着uvwz和uxzy进行，如何操作？

A: 没办法做到（或者需要一个新的路由选择算法）



A: 没办法做到（采用基于目标的转发，和LS,DV路由）

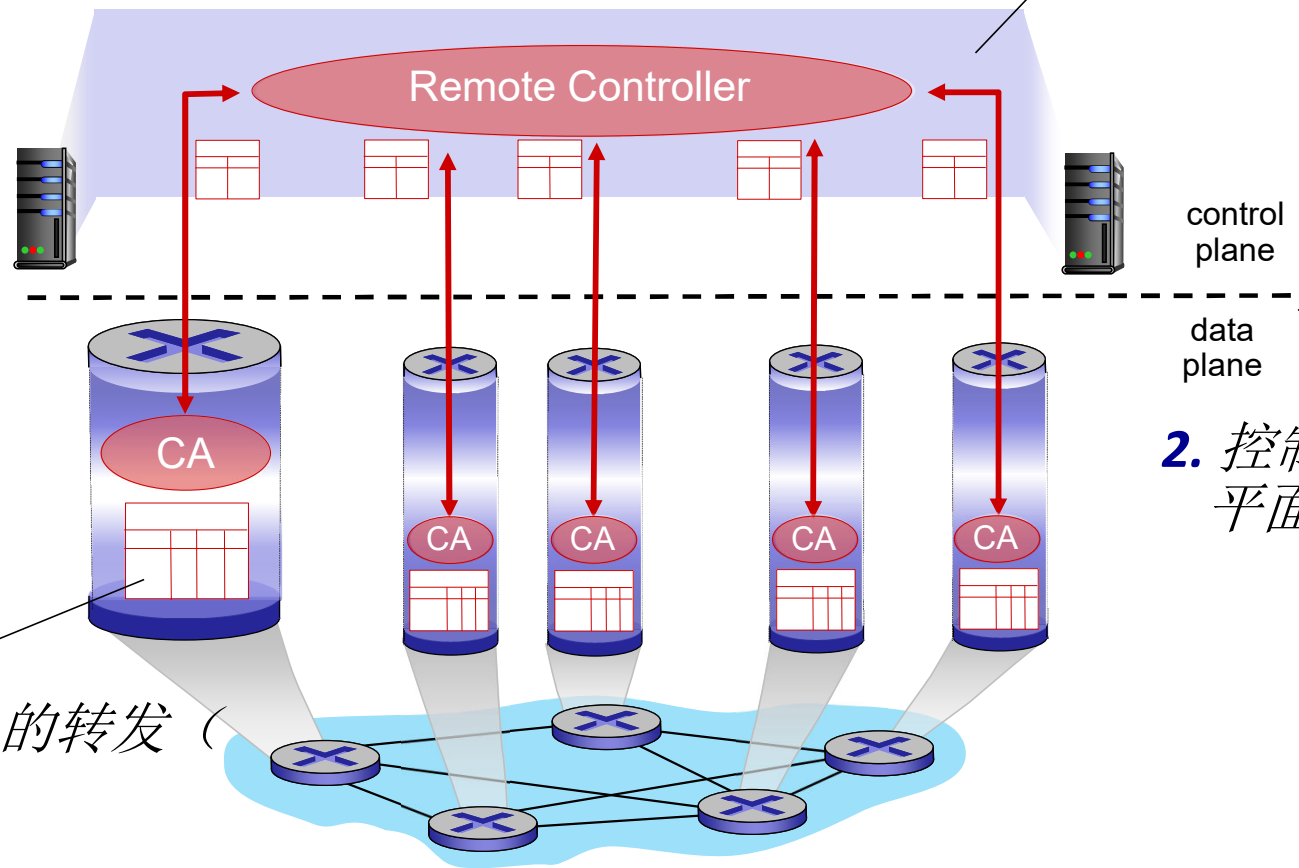
我们在第4章学过通用化的转发和SDN可以用于实现所需要的任何路由

Software defined networking (SDN)

4. 可编程控制应用



3. 控制平面功能在数据平面交换机之外



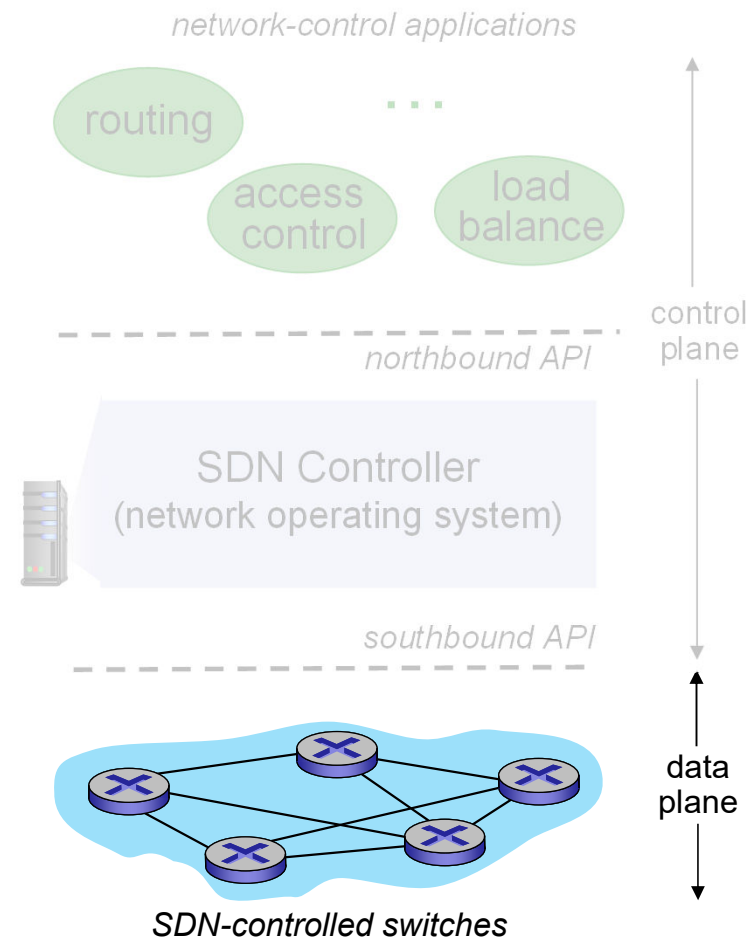
2. 控制平面和数据平面分离

1. 通用的“基于流”的转发 (例如OpenFlow)

Software defined networking (SDN)

数据平面交换机:

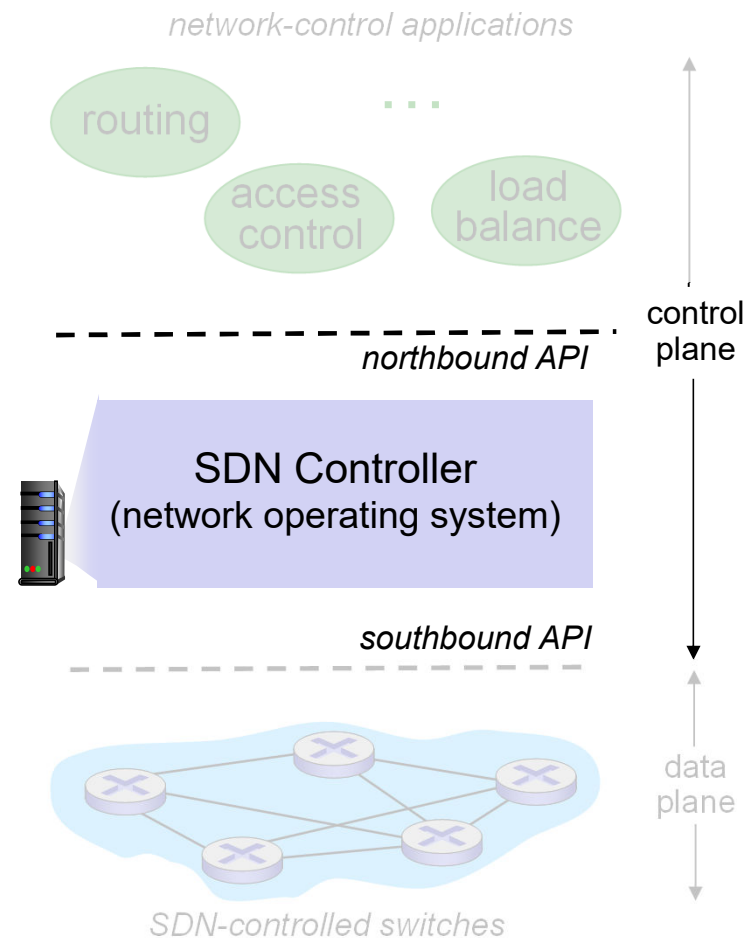
- 快速，简单，商业化的交换机采用硬件实现了通用数据平面转发（见4.4）
- 在控制器的监督下，流（转发）表被计算和安装
- API用于基于表的交换机控制（例如 OpenFlow）
 - 定义哪些是可控的，哪些不是
- 与控制器的通信协议（例如：OpenFlow）



Software defined networking (SDN)

SDN 控制器 (network OS):

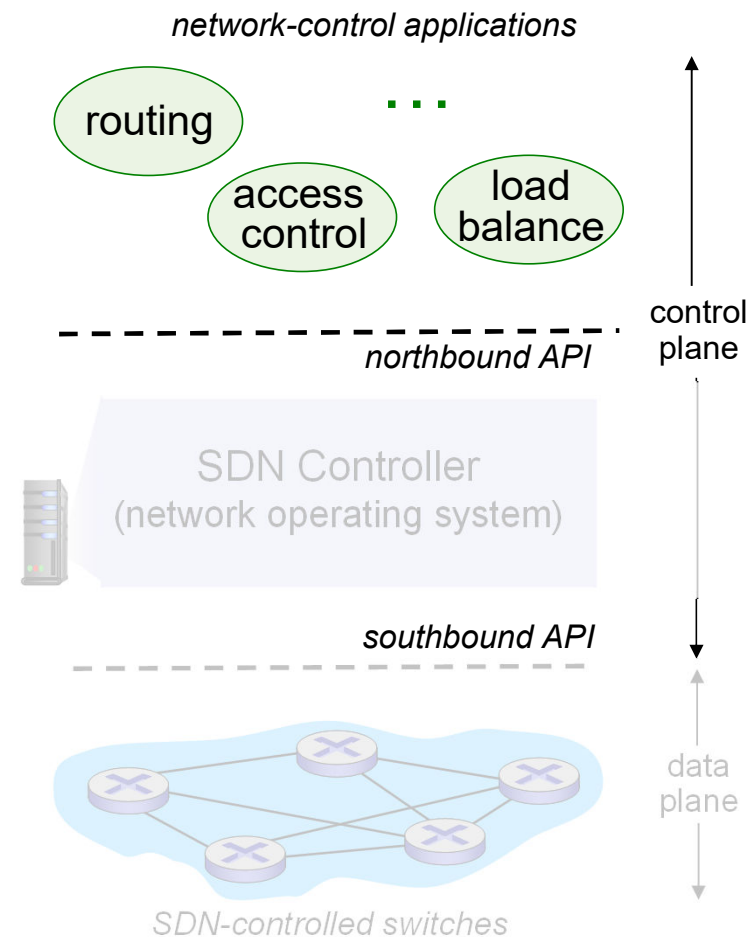
- 维护网络状态信息
- 通过北向接口和其“上”的网络控制应用交互
- 通过南向接口和其“下”的网络交换机交互
- 由于性能、可扩展性、容错以及健壮原因可以被分布式方式实现



Software defined networking (SDN)

网络控制应用:

- 控制的“大脑”：通过SDN控制器提供的API，利用低层的服务实现控制功能
 - 路由器 交换机
 - 接入控制 防火墙
 - 负载均衡
 - 其他功能
- 未绑定：可由第三方提供：不同于路由供应商或SDN控制器提供商

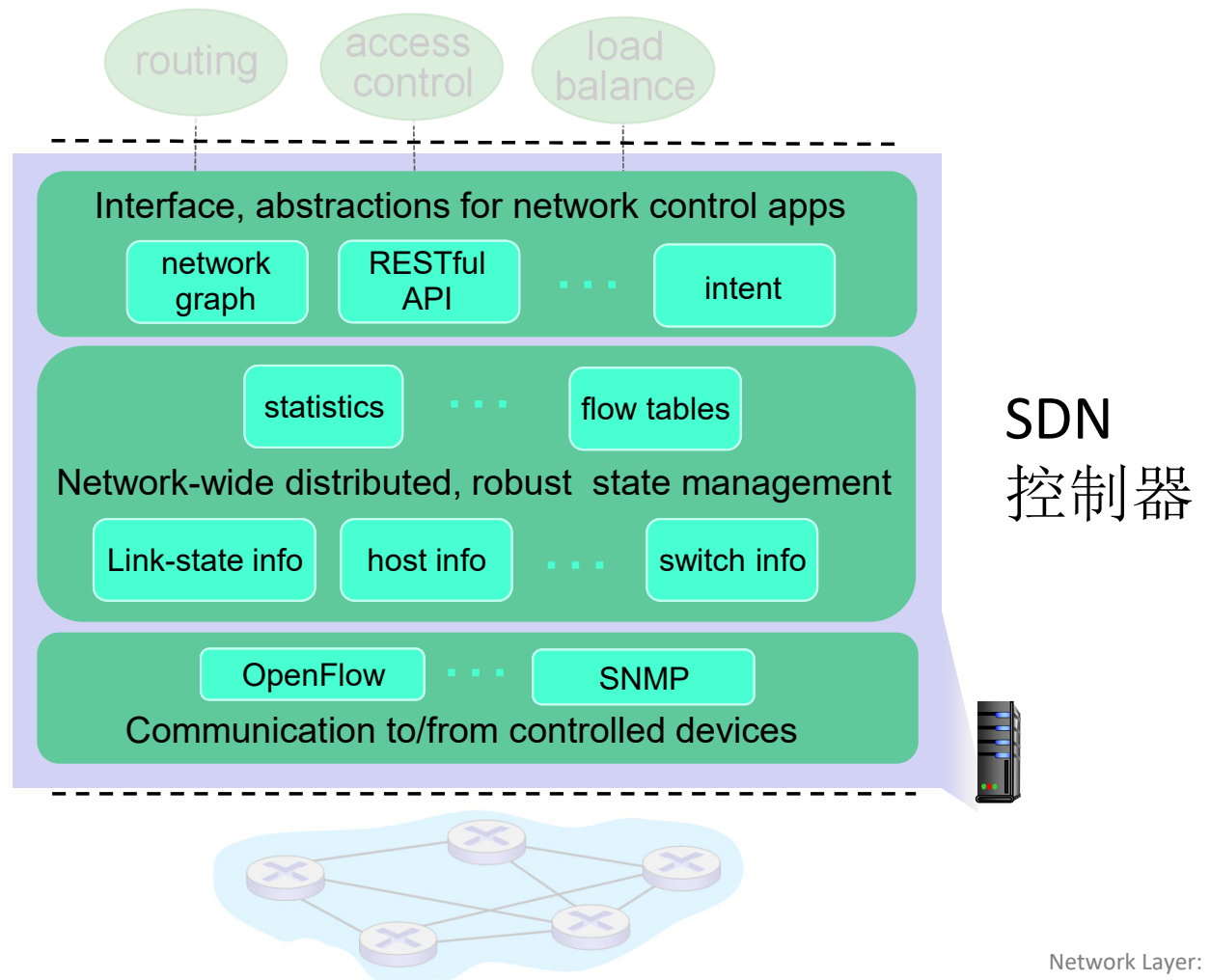


SDN控制器元件

网络控制应用接口层：API抽象

网络层状态管理：网络链路，交换机和服务的状态
：一个分布式数据库

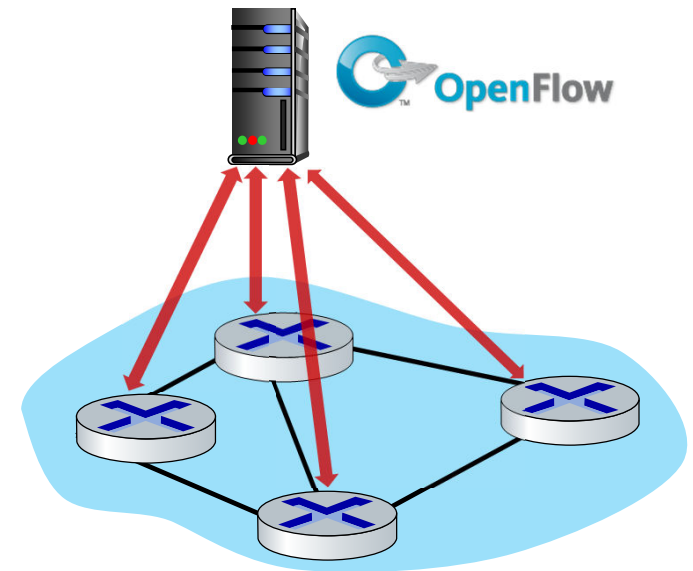
通信层：SDN控制器和被控的交换机交互



OpenFlow 协议

- 在控制器和交换机之间互操作
- 使用TCP进行报文交换
 - 加密可选
- 3中OpenFlow报文：
 - 控制器-交换机
 - 异步消息（交换机到控制器）
 - 对称消息(misc.)
- 与OpenFlow API不同
 - AIP被用于制定的通用转发行为

OpenFlow Controller

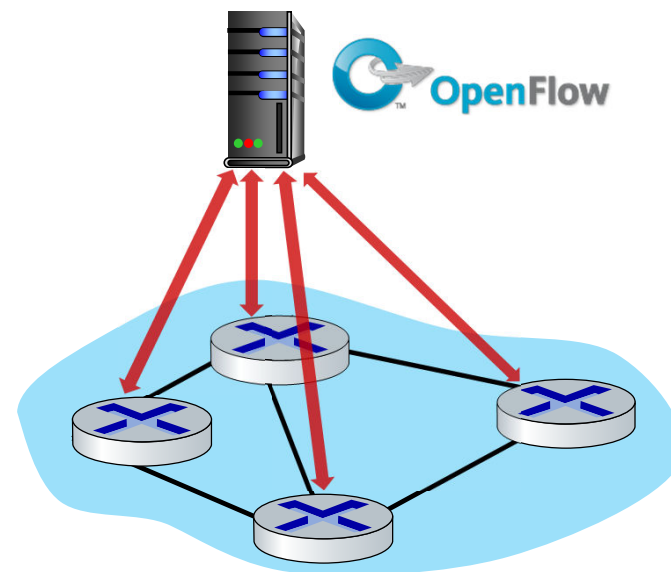


OpenFlow: 控制器-交换机报文

关键的控制器-交换机报文

- **特性:** 控制器查询交换机特性, 交换机回应
- **配置:** 控制器查询/设置交换机配置参数
- **修改状态:** 增加, 删除, 修改 OpenFlow 中流表的表项
- **分组输出:** 控制器可以通过特定交换机端口将分组发走

OpenFlow Controller

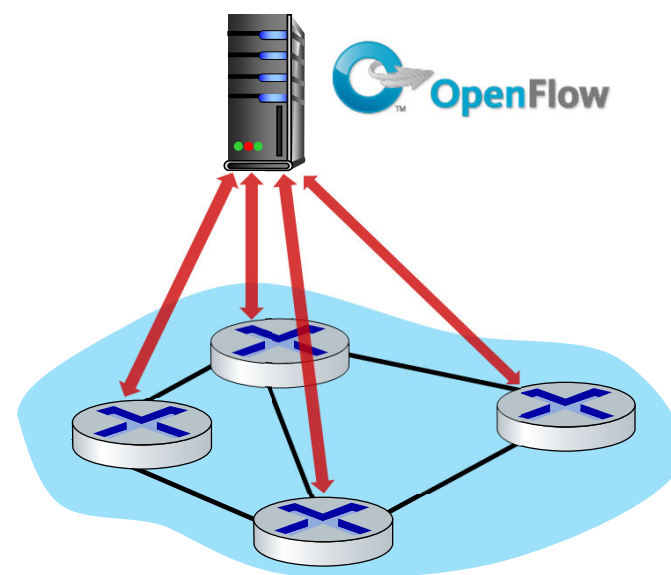


OpenFlow: 交换机-控制器报文

关键交换机到控制器报文

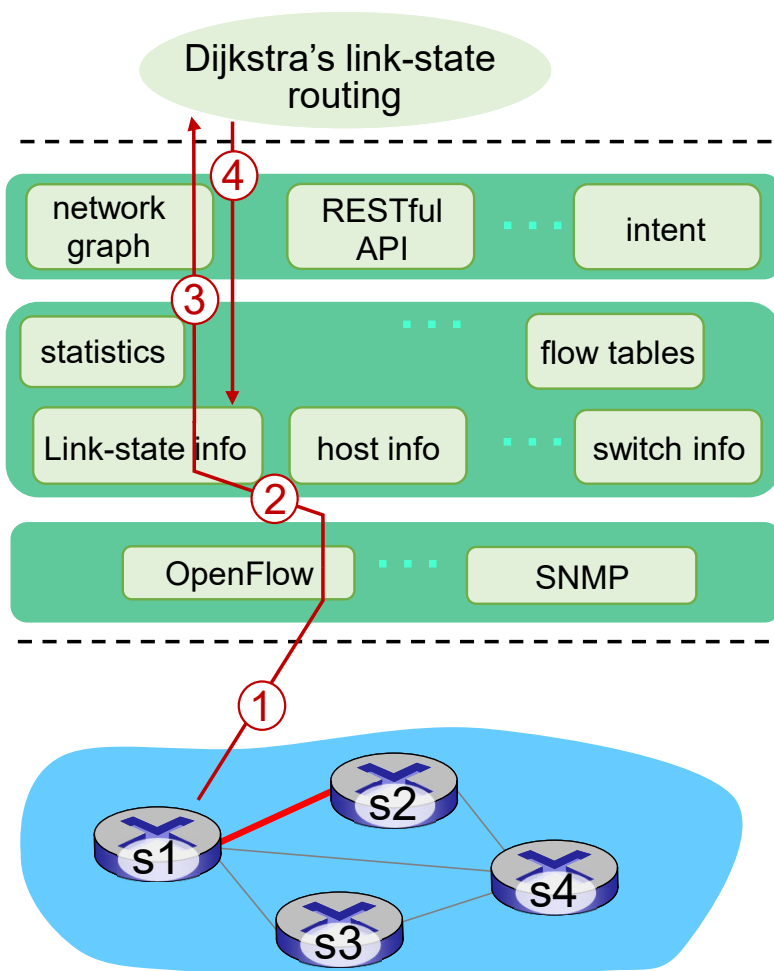
- **分组进入**: 将分组（和它的控制）发给交换机，见交换机的**分组输出**报文
- **流移除**: 在交换机上删除流表的表项
- **端口状态**: 通告控制器一个端口的改变

OpenFlow Controller



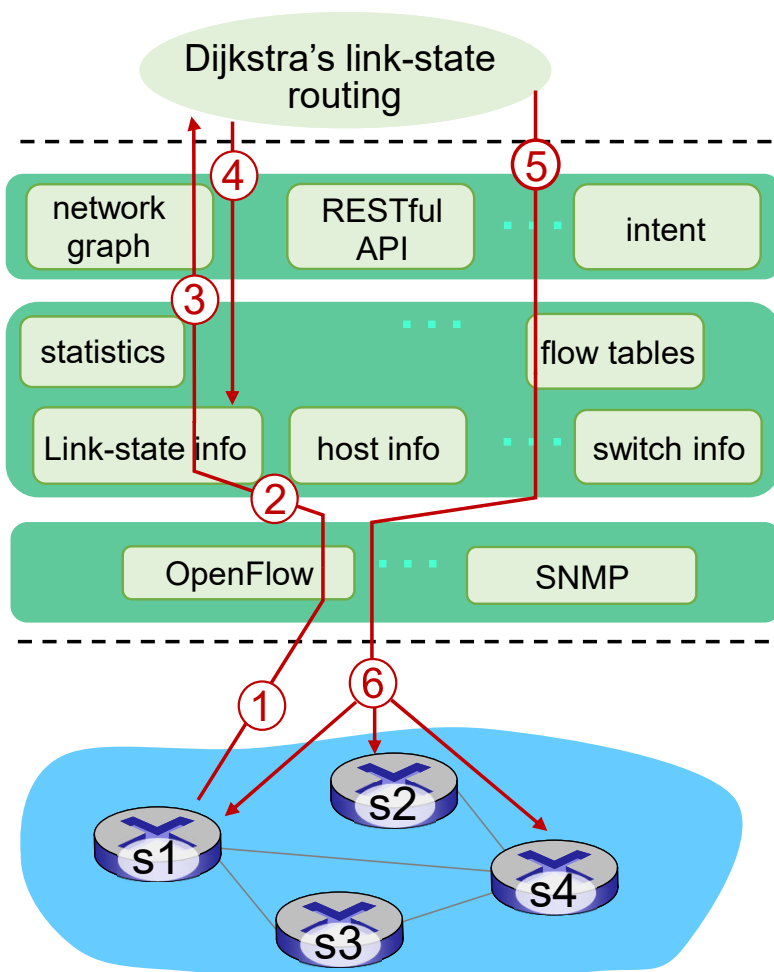
幸运的是，网络运营商不会通过直接创建/发送OpenFlow消息来“编程”交换机。相反，使用控制器上的更高级别的抽象

SDN: 控制/数据平面交互的实例



- ① S1, 经历了链路失效, 采用OpenFlow端口状态报文通告控制器
- ② SDN 控制器接收OpenFlow报文, 更新链路状态信息
- ③ Dijkstra路由算法应用被调用 (前面注册过这个状态变化消息)
- ④ Dijkstra路由算法访问控制器中的网络拓扑信息, 链路状态信息计算新路由

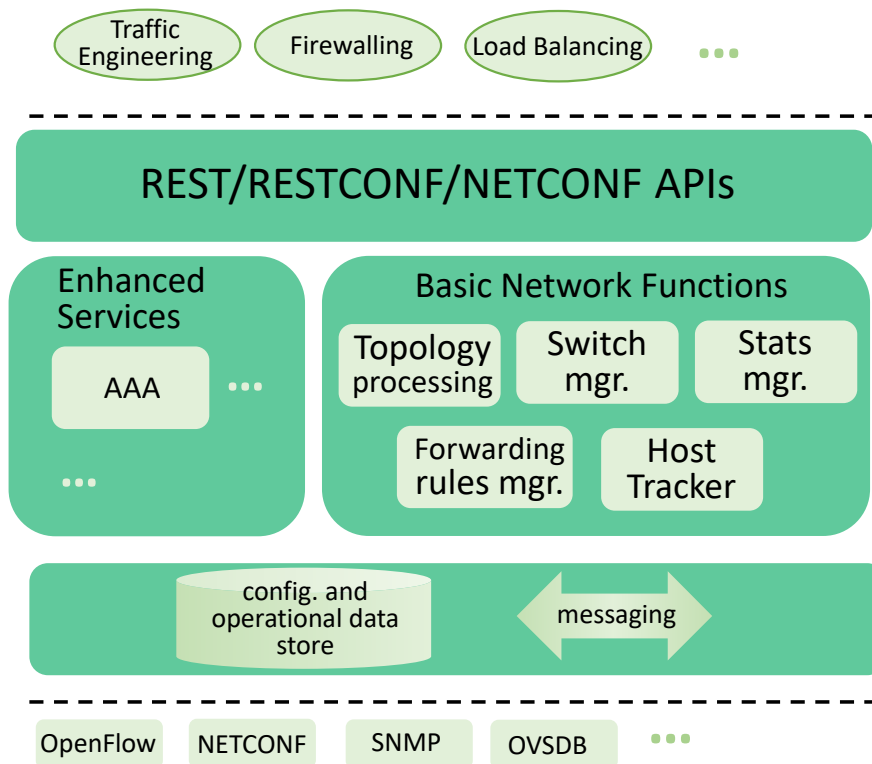
SDN: 控制/数据平面交互的实例



⑤ 链路状态路由app和SDN控制器中流表计算元件交互，计算出新的所需流表

⑥ 控制器采用OpenFlow在交换机上安装新的需要更新的流表

OpenDaylight (ODL) 控制器



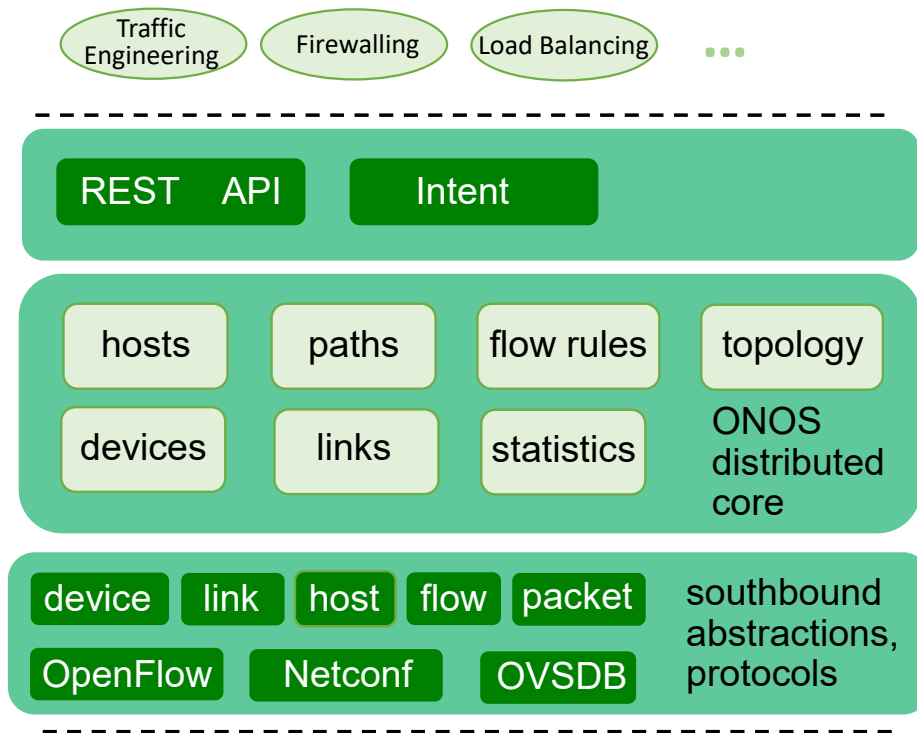
网络编排和应用
北向 API

服务抽象层
连接内部的，外部的应用和服务

服务抽象层 (SAL)

南向 API

ONOS 控制器



网络应用

北向 API

南向 API

2025中科大自动化系

- 控制应用和控制器分离
- 意图框架：更高层次的服务描述：描述什么而不是如何
- 相当多的重点聚焦在分布式核心上，以提高服务的可靠性，复制性能的扩展性

SDN: 一些挑战

- 强化控制平面：可信、可靠、性能可扩展性、安全的分布式系统
 - 对于失效的鲁棒性：利用为控制平面可靠分布式系统的强大理论
 - 可信任，安全：从开始就进行铸造
- 网络、协议满足特殊任务的需求
 - e.g., 实时性、超高可靠性、超高安全性
- 互联网络范围内的扩展性：不仅仅在一个AS内部署，全网部署
- SDN在5G网络中非常关键

SDN 和传统网络协议的未来

- SDN-计算 对比 路由器计算的转发表：
 - 仅仅是一个逻辑上集中计算对比于 分布式协议计算的例子
- 可以想象一下SDN计算的拥塞控制：
 - 控制器基于路由器报告（分组交换机给控制器）的拥塞水平设置发送方的速率



网络层：“控制平面”提纲

- 引论
- 路由算法
- ISP内部的路由协议: RIP, OSPF
- ISP之间的路由: BGP
- SDN控制平面
- Internet Control Message Protocol



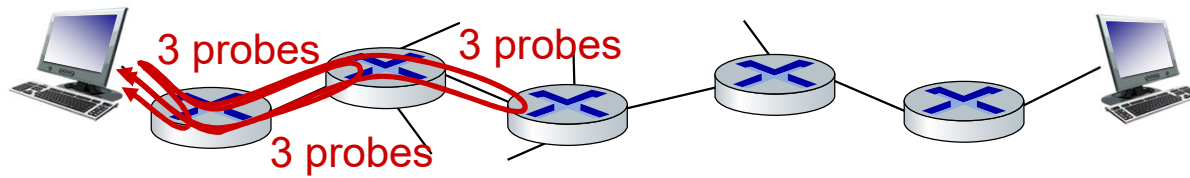
- 网络管理，配置
 - SNMP
 - NETCONF/YANG

ICMP: internet control message protocol

- 被主机和路由器用于传送网络层面的控制信息
 - 错误报告：主机、网络、端口和协议不可达
 - 回声请求/响应（被ping使用）
- 在IP协议之“上”
 - ICMP报文被携带于IP数据报中
- *ICMP 报文*: 类型，代码加上触发错误的IP数据报的头8个字节

Type	Code	description
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

Traceroute 和 ICMP



- 源主机发送一系列UDP段给目标主机
 - 1st 数据报设置 TTL =1, 2nd数据报设置 TTL=2, 等等.
- 第n轮的数据报到达第n个路由器:
 - 路由器将会丢弃该数据报, 并且发送一个 ICMP报文给源主机 (类型11, 代码0)
 - ICMP报文包括路由器的名字和IP地址
- 当ICMP报文到达源主机时: 记录RTT

停止判据:

- UDP段最终到达目标主机
- 目标主机回送ICMP “端口不大达” 的报文 (类型3, 代码3)
- 源主机停止测试

什么是网络管理?

- 自治系统（autonomous systems: aka “network”）: 大约1000左右的互操作的软硬件元件
- 其他复杂的系统同样需要被检测，配置和控制：
 - 喷气机，核反应堆，其他设备？

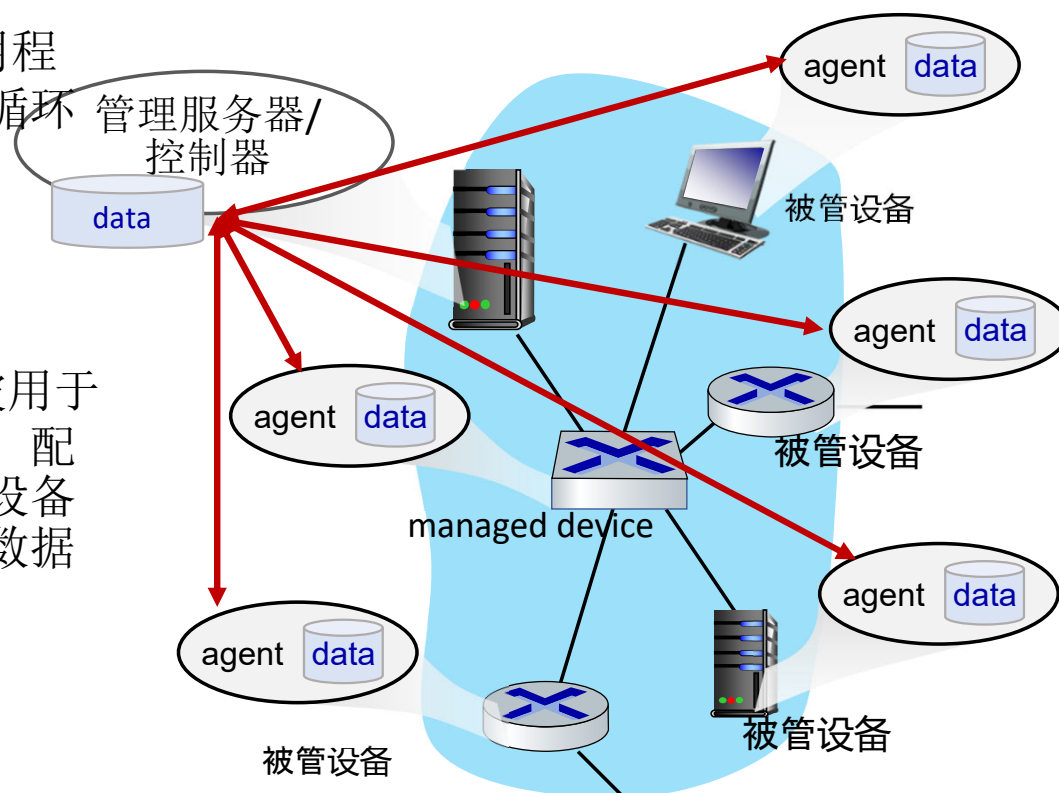


“网络管理包括了硬件、软件和人类元素的设置、综合和协调，以便监测、测试、轮询、配置、分析、评价和控制网络和网元资源，用合理的成本满足实时性、运行性和服务质量的要求”

网络管理的元件

管理服务器: 应用程序，通常带有网络循环中的管理者（人）

网络管理协议: 被用于给管理服务器查询，配置管理设备；用于设备报告给管理服务器数据和事件



被管设备:

具备可管理，可配置硬件和软件组件软件 的设备

数据: 设备的“状态”，配置参数，运行参数和设备统计信息

网络管理员管理网络的方法

CLI (命令行界面)

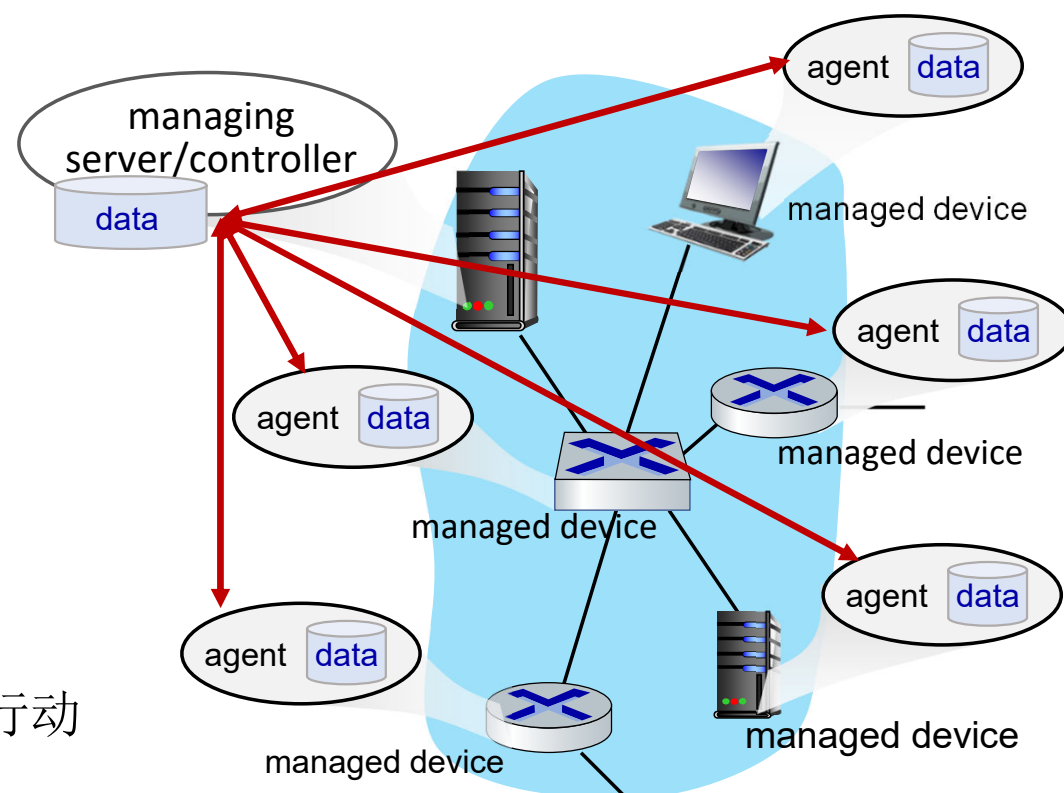
- 管理员向单独设备直接发送（键入，脚本）命令（例如：vis ssh）

SNMP/MIB

- 管理员采用SNMP协议查询/设置MIB库中的数据

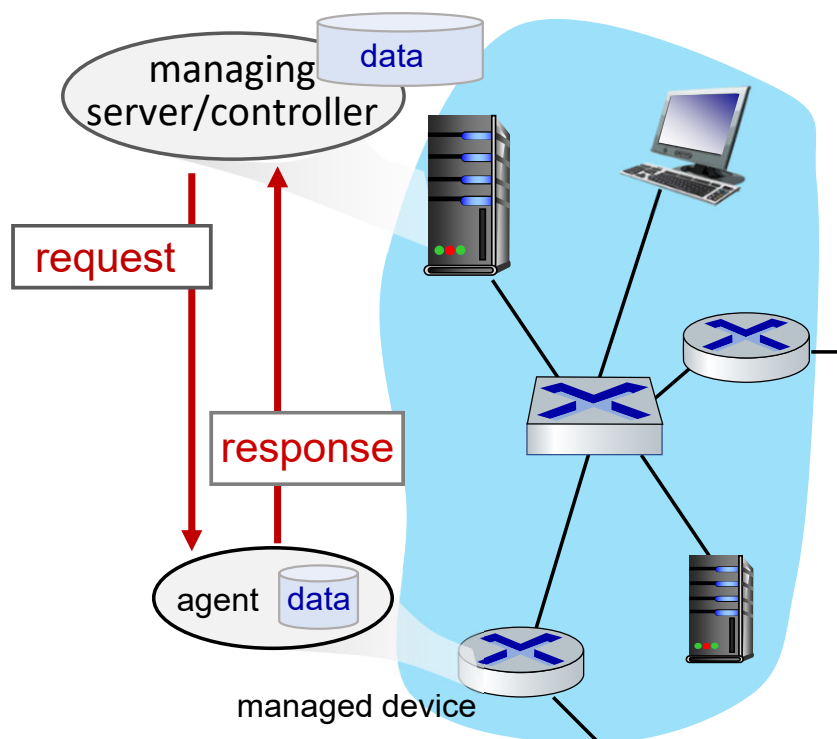
NETCONF/YANG

- 更抽象，网络范围，全面
- 注重多设备配置管理
- YANG: 数据建模语言
- NETCONF: 在YANG兼容的远程设备间进行动作/通信

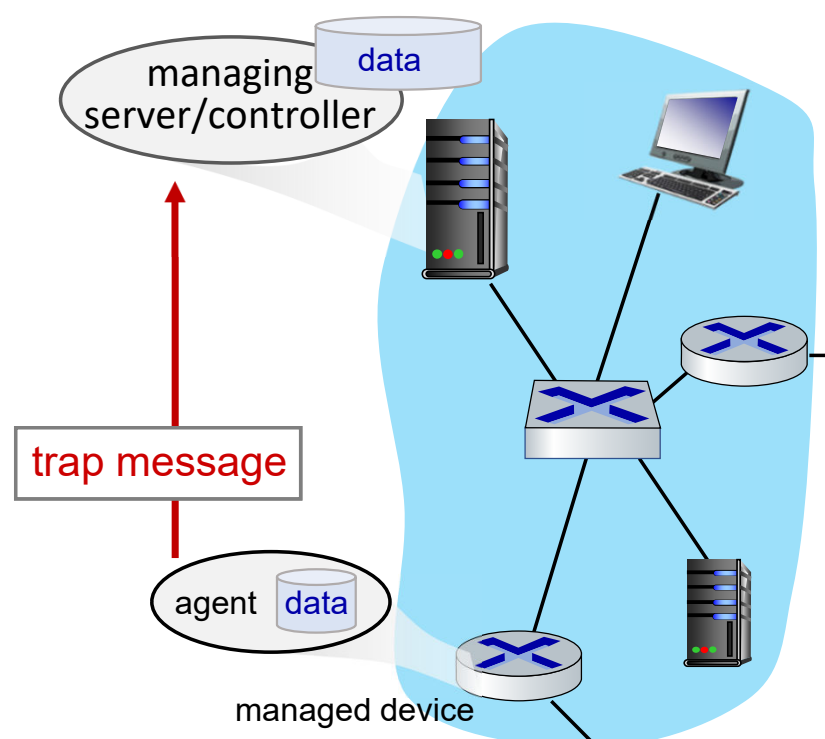


SNMP 协议

2个查询MIB库信息方法，命令：



request/response mode

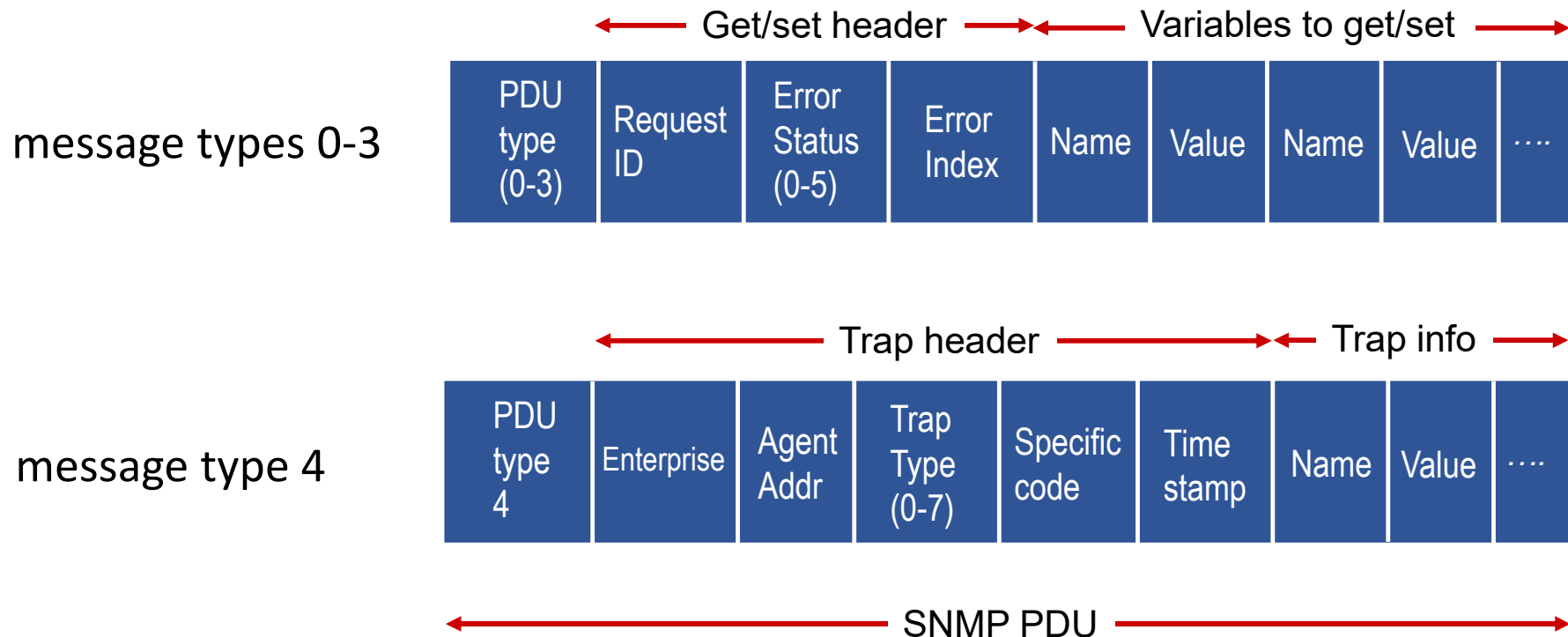


trap mode

SNMP 协议: 报文类型

报文类型	功能
GetRequest GetNextRequest GetBulkRequest	manager-to-agent: “get me data” (data instance, next data in list, block of data).
SetRequest	manager-to-agent: set MIB value
Response	Agent-to-manager: value, response to Request
Trap	Agent-to-manager: inform manager of exceptional event

SNMP 协议: 报文格式



SNMP: Management Information Base (MIB)

- 被管设备的运行（以及一些配置）数据
- 被收集于设备中的**MIB 模块**
 - 400 MIB 模块被RFC定义; 很多都是供应商规范的MIB库
- **Structure of Management Information (SMI):** 数据定义语言
- 用于UDP协议的MIB变量的样例:

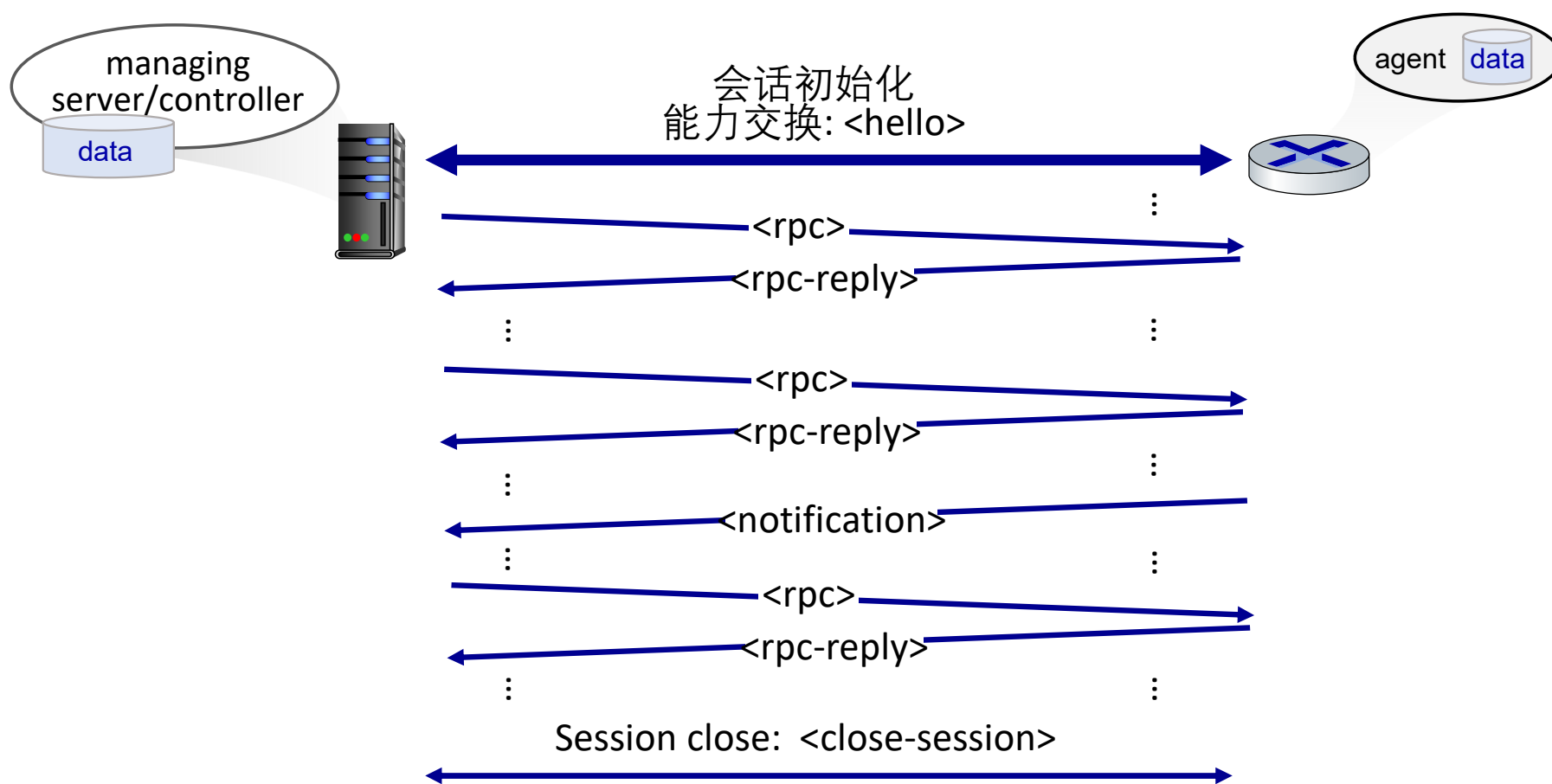


Object ID	Name	Type	Comments
1.3.6.1.2.1.7.1	UDPInDatagrams	32-bit counter	total # datagrams delivered
1.3.6.1.2.1.7.2	UDPNoPorts	32-bit counter	# undeliverable datagrams (no application at port)
1.3.6.1.2.1.7.3	UDInErrors	32-bit counter	# undeliverable datagrams (all other reasons)
1.3.6.1.2.1.7.4	UDPOutDatagrams	32-bit counter	total # datagrams sent
1.3.6.1.2.1.7.5	udpTable	SEQUENCE	one entry for each port currently in use

NETCONF 概述

- **目标:** 主动管理和配置网络范围内的设备
- 在管理服务器和被管网络设备之间运行
 - 动作：请求，设置，修改，激活配置
 - 多个设备上的原子操作动作
 - 查询运行数据和统计数据
 - 订阅来自设备的通知
- 远程过程调用(Remote procedure call :RPC)范式
 - NETCONF 协议报文被编码与XML
 - 在安全，可靠传输协议（如： TLS）上进行交换

NETCONF 初始化, 交换和关闭



选定的NETCONF操作

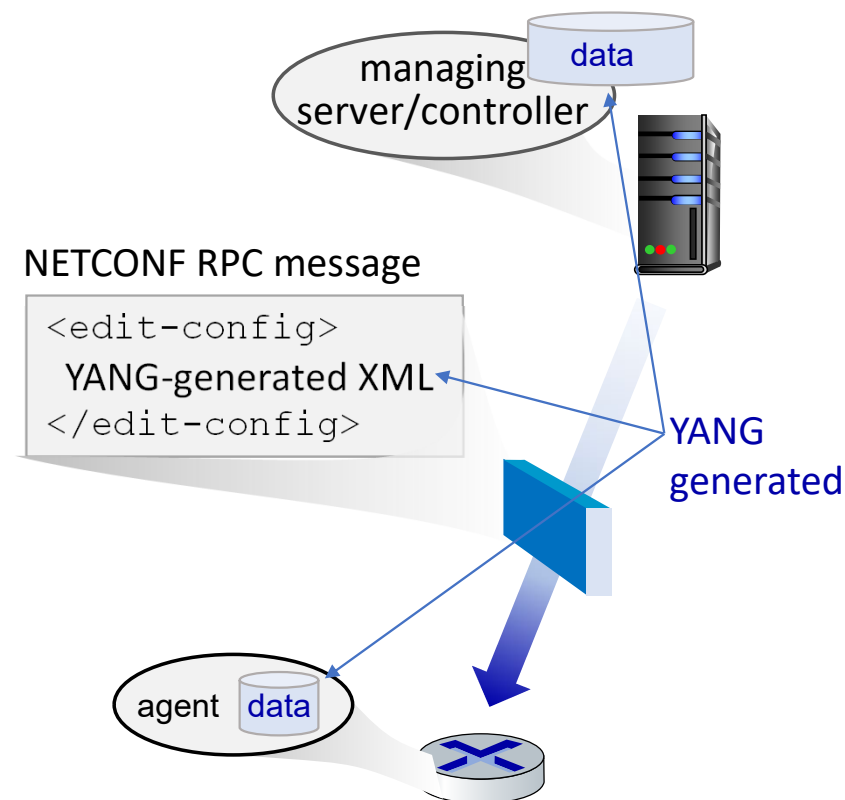
NETCONF	操作描述
<code><get-config></code> <code><get></code> <code><edit-config></code> <code><lock></code> , <code><unlock></code> <code><create-subscription></code> <code><notification></code>	检索给定配置的全部或部分。一个设备可能有多种配置 检索全部或部分配置状态和操作状态数据. 在被管设备上改变指定的（可能正在运行的）配置，被管设备 <code><rpc-reply></code> 包含 <code><ok></code> 或者 具有回滚操作的<code><rpcerror></code> 锁定（解锁）被管设备上的配置数据存储（以锁定来自其他源的NETCONF、SNMP或CLI命令）。 启用来自托管设备的事件通知订阅

NETCONF RPC 消息样例

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <rpc message-id="101"  note message id
03   xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
04   <edit-config>      change a configuration
05     <target>
06       <running/> change the running configuration
07     </target>
08     <config>
09       <top xmlns="http://example.com/schema/
10         1.2/config">
11         <interface>
12           <name>Ethernet0/0</name>  change MTU of Ethernet 0/0 interface to 1500
13           <mtu>1500</mtu>
14         </interface>
15       </top>
16     </config>
17 </edit-config>
18 </rpc>
```

YANG

- 数据定义语言用于规范NETCONF网络管理数据的结构，语法和语义
 - 内置数据类型，类似于SMI
- 描述设备的XML文档，可以从YANG描述中生成功能
- 可以表示有效NETCONF配置必须满足的数据间的约束
 - 确保NETCONF配置满足准确性，一致性约束



网络层：总结

我们已经学了很多！

- 网络控制平面的方法
 - 每路由器控制 (传统方法)
 - 逻辑上集中的控制 (software defined networking)
- 传统路由算法
 - 在互联网中的实现： OSPF , BGP
- SDN 控制器
 - 实践中的实现： ODL, ONOS
- Internet Control Message Protocol
- 网络管理

下一站：链路层！

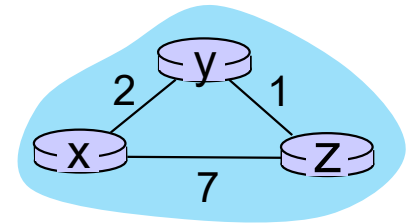
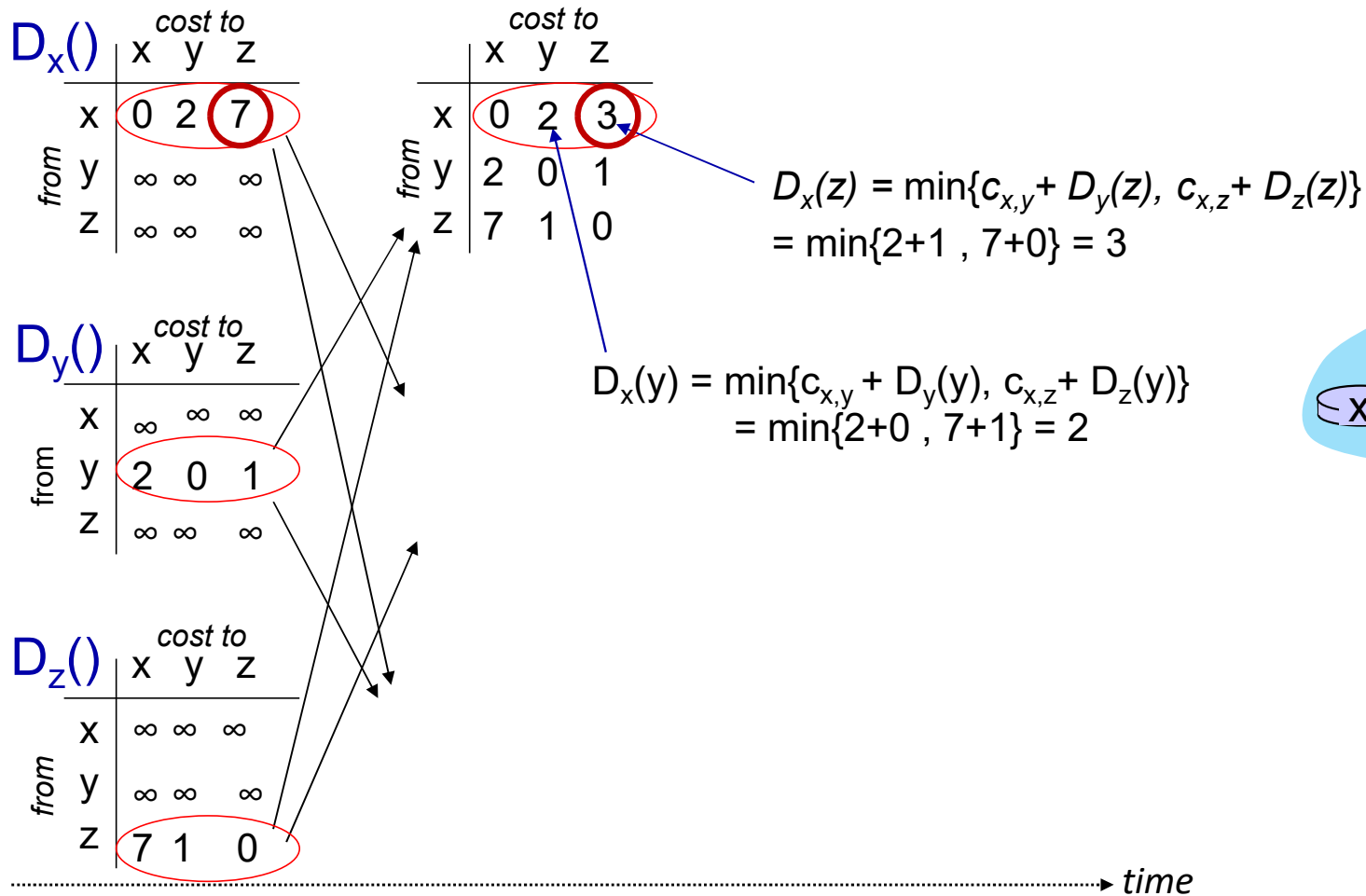
网络层-控制平面：学完了！

- 引论
- 路由算法
 - 链路状态link state
 - 距离矢量distance vector
- ISP内部的路由协议: RIP,OSPF
- ISP之间的路由: BGP
- SDN控制平面
- Internet Control Message Protocol
- 网络管理，配置
 - SNMP
 - NETCONF/YANG



第5章附件ppt

距离矢量：另外一个例子



距离矢量：另外一个例子

